Department of Computer Science

Lab Manual

of

MACHINE LEARNING

MCA521-4

Class Name: IV MCA

Master of Computer Application

2026

Prepared by:	Verified by:
Nirupama Vincent 2547236 4MCAB	Dr. Rupali Sunil Wagh Dr. Helen K Joy Dr. Shivangi Singh

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University "**Excellence and Service"**. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

"To develop IT professionals with ethical and human values". To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Master of Computer Applications (MCA) is a post-graduate programme of CHRIST (Deemed to be University) and approved by the All India Council of Technical Education (AICTE). It is a two-year programme spread over six trimesters to bridge the chasm between computer studies and applications, bringing within reach of enthusiastic youngsters an excellent group of experienced and dedicated staff and experts, and an enviable infrastructure. MCA at CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training programme aims to prepare young minds for challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field. The Department is committed to the motto "Excellence and Service," and this philosophy pervades every aspect of its functioning.

Programme Objective

This programme is conceptualised and designed based on the strong commitment of the department to provide better quality education to the students. The principal objectives of this course are to:

- Heighten technological awareness
- Train future industry specialists
- Encourage effective software development
- Provide research training
- Involve students in innovative research
- Produce graduates with a two-year professional education in Computer Science with technical, professional, and communications skills.

Ethics and Human Values

1. Only proprietary or open-source software would be used for academic teaching and learning purposes.
2. Copying of programs from the internet, friends or from other sources is strictly discouraged since it impairs development of programming skills.
3. Unique Practical (Domain based) exercises ensure that the students don't get involved in code plagiarism.
4. Projects undertaken by students during the course are done in teams to improve collaborative work and synergy between team members.
5. Projects involve modularization which initiates students to take individual responsibility for common goals.
6. Passion for excellence is promoted among the students, be it in software development or project documentation.
7. Giving due credit to sources during the seminar and research assignment is promoted among the students
8. The course and its design enforce the practice of good referencing technique to improve the sense of integrity.
9. Courses involving group discussions and debates on ethical practices and human values are designed to sensitize the students in dealing with customers and members within the Organization.

Programme Outcomes:

P01: Foundation Knowledge: Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.
P02: Problem Analysis: Identify, review, formulate, and analyse problems primarily focussing on customer requirements using critical thinking frameworks.
P03: Development of Solutions: Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
PO4: Modern Tool Usage: Select, adapt, and apply modern computational tools such as the development of algorithms with an understanding of the limitations including human biases.
P05: Individual and Teamwork: Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
P06: Project Management and Finance: Use the principles of project management such as scheduling, and work breakdown structure and be conversant with the principles of Finance for profitable project management.
P07: Ethics: Commit to professional ethics in managing software projects with financial aspects.
Learn to use new technologies for cyber security and insulate customers from malware
P08: Life-long learning: Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Programme Educational Objectives(PEO's)

PEO1: Develop innovative and effective software solutions through research that addresses real-world challenges, grounded in a commitment to continuous learning and adaptability.
PEO2: Advance the knowledge and practices in the field of computer applications through lifelong learning and rigorous research, supporting professional growth and academic excellence.
PEO3: Exhibit ethical leadership, social responsibility to contribute and enhance community well-being by innovating solutions.

MCA521-4 – MACHINE LEARNING
Course Name: MACHINE LEARNING Code: MCA521-4

Total Hours: 75
L + P + T: 3 + 4 + 0
Max Marks: 100
Credits: 4
Course Type: Core Course
Course Category: Theo&Prac

Course Description

Machine Learning is a key area in computer applications that focuses on building models capable of making predictions or informed decisions based on data. A strong understanding of machine learning enables students to analyze and interpret complex datasets, develop predictive models, and extract meaningful insights. This course covers a wide range of machine learning concepts and techniques, including supervised and unsupervised learning. It provides a solid theoretical foundation while emphasizing practical, hands-on experience with algorithms and real-world data. The course equips students with the knowledge and skills required to apply machine learning techniques across various domains using modern tools and methodologies.

Course Objectives

This course enables students to understand the fundamental concepts of Machine Learning, including its core principles, terminology, and methodologies. Students will learn to differentiate between various types of learning algorithms such as supervised, unsupervised, and reinforcement learning, and recognize their appropriate applications.

Course Outcomes

After successful completion of the course, students will be able to:

- Explain the fundamental principles and scope of Machine Learning
- Implement modelling practices in machine learning for better generalisation
- Interpret predictive modelling results and performance of supervised machine learning techniques
- Assess the outcomes and effects of unsupervised machine learning processes
- Deploy robust machine learning models for interdisciplinary applications

Unit-1: INTRODUCTION TO MACHINE LEARNING Teaching Hours: 15 Hours

Introduction to AI – Knowledge Representation – Data – Representation of Data, Data Processing, Data Storage, Data Processing, Types of Data – Exploratory Data Analysis, Visualisations and Interpretation, Outliers

Machine Learning: Definition, Objectives, Components, Features, Applications – Traditional Programming v/s Machine Learning, Types of Machine Learning – Predictive Models, Statistical Inferences – Applications of Machine Learning

Challenges in ML: Insufficient Quantity of Data, Non-representative and Poor-Quality Data, Irrelevant Features, Estimation Estimation, Reducing Human Biases in Model Building, Ethical Use of Technology

Lab Exercises:

1. Data Preprocessing and visualisation
2. Data exploration and inferences

Unit-2: PROCESSES IN MACHINE LEARNING Teaching Hours: 15 Hours

Data Preparation and Model Selection: Effect of Data Preparation: Encoding, Nominal and Ordinal Representation, Feature Transformation and Feature Engineering, Generalisation, Normalisation, Sampling, Using Saved Weights, Model Selection Strategies: Overfitting, Underfitting, Bias-Variance Trade-off, Testing and Validation: Performance measures - Confusion matrix, Precision-Recall Trade off, F1 Score, ROC Curve, AUC, Error Analysis, Model Parameters, Hyperparameters, Hyperparameter Tuning, Cross Validation, Decision Functions: Introduction to Supervised Machine Learning Methods, Decision Functions, Decision Thresholds, Distance Measures, Hyperplanes, Regression & Classification Problems, Loss and Cost Functions, Linear Regression using OLS, Multi-class vs Multi-label, KNN Classification

Lab Exercises:

3. Saving weights of a generalised model.
4. Regression and Classification Evaluation Metrics: Illustration through Linear Regression and KNN Classification

Unit-3: SUPERVISED LEARNING ALGORITHMS Teaching Hours: 15 Hours

Learning through Optimisation: Gradient Descent, Linear Regression, Logistic Regression Computational Learning: Decision Trees, Naive-bayes classification, Support Vector Machines, Ensemble Learners: Learning, Voting, Bagging, Stacking, Boosting, Random Forests, GridSearchCV

Lab Exercises:

5. Regression through Gradient Descent
6. Comparison of Different Classifiers
7. Illustration of Ensemble Learning Methods

Unit-4: UNSUPERVISED LEARNING AND DIMENSIONALITY REDUCTION Teaching Hours: 15 Hours

Introduction: Types of Unsupervised Learning, Challenges in Unsupervised Learning, Concept of Cost Functions in Unsupervised Learning, Clustering Methods: Distance based, Centroid Based, Elbow method to find Optimal Number of Clusters, Silhouette Method, K-Means Clustering, Hierarchical Clustering: Agglomerative and Bottom Up, Applying Supervised Learning after Clustering, Dimensionality Reduction Methods: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

Lab Exercises:

8. KMeans and Elbow Methods
9. Hierarchical Clustering and Dendrograms
10. Dimensionality Reduction
11. Image Compression using Dimensionality Reduction.

Unit-5 Teaching Hours: 15 Hours

MACHINE LEARNING IN REAL-WORLD

Emerging Techniques: Role of ML in attaining Sustainable Development Goals, Time Series Preprocessing, Blackbox Methods: Neural Networks & Deep Learning, Reinforcement Learning/QLearning, Self Supervised Learning, Quantum Machine Learning, Keras and Tensorflow for

designing Multi Layer Perceptron. Case Studies: CNN, RNN, Advances in Deep Learning, Natural Language Processing, Real Time Systems, Computer Vision, Robotics, Expert Systems, Generative Networks, Large Language Models. Model Deployment: Model Deployment: Connecting Models to User Interface, Deployment of ML Models.

Lab Exercises:

12. Training a Multi-layer perceptron
13. Deploying Trained ML-models

Text and Reference Books

[1] Campesato, O. (2020). Artificial Intelligence, Machine Learning and Deep Learning. Mercury Learning and Information.
[2] Andreas C. Müller and Sarah Guido (2017), "Introduction to Machine Learning with Python A Guide For Data Scientists" O'Reilly book
[3] Ethem Alpaydin (2005), "Introduction to Machine Learning", Prentice Hall of India
[4] Max Kuhn and Kjell Johnson (2018), "Applied Predictive Modelling", Springer

Essential Reading

[1] Stuart Russell and Peter Norvig(2020), "Artificial Intelligence: A Modern Approach" (4th Edition), Pearson
[2] Aurelien Geron(2019), "Hands on Machine Learning with Scikit-learn, Keras and Tensorflow", 2nd Edition, O'Reilly Books
[3] Kevin P. Murphy(2012), "Machine Learning: A Probabilistic Perspective", MIT Press
[4] Hastie, Tibshirani, Friedman(2008), "The Elements of Statistical Learning" (2nd ed), Springer
[5] Ian Goodfellow, Aaron Courville, Yoshua Bengio (2019), "Deep Learning (Adaptive Computation And Machine Learning Series)", MIT Press

Additional Information (Web Resources):

1. Andrew Ng, Deeplearning.ai, Machine Learning Specialisation, offered through Coursera: https://www.deeplearning.ai/courses/machine-learning-specialization/

Evaluation Pattern: CIA - 50% ESE - 50%

LIST OF PROGRAMS

MSCAIML

Sl. no	Title of lab Experiment	Page number	RBT	CO
1	India Air Quality & Crop Yield — EDA Lab Data Preprocessing, Visualisation & Exploration		L3	CO1, 3
2			L3	CO2, 3
3	Student Survey: Simple Linear Regression using Scikit Learn, OLS, and Parameter Saving		L3	CO3
4	Regression and Classification Evaluation Metrics		L3	CO3
5	Linear Regression through Gradient Descent		L3	CO3
6	Comparison of Logistic Regression and K Nearest Neighbors (KNN) Classifiers		L3	CO3, 4
7	Lab Activity: Iris Dataset Analysis		L4	CO3
8	Implementation and Performance Evaluation of Categorical Naive Bayes Classifier		L3	CO3
9	Implementation of SVM and PCA using Real-World Datasets		L3	CO4
10	Learning the XOR Boolean Function Using an MLP		L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <ML lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2…>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.

- Upload the PDF to Google Classroom before the deadline.

Lab 1 & 2

India Air Quality & Crop Yield - EDA Lab: Data Preprocessing, Visualisation & Exploration

Lab Exercise I & II:

Aim

- To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualisation techniques on raw, messy, real-world datasets.

Evaluation Rubrics

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

Code with Results

Task 1

Dataset Profiling and Initial Data Inspection

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

city = pd.read_csv('city_day.csv')
crop= pd.read_csv('crop_production.csv')
```

✓ 1.7s

```
city.head()
```

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	133.36	0.00	0.02	0.00	NaN	NaN
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	34.06	3.68	5.50	3.77	NaN	NaN
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	30.70	6.80	16.40	2.25	NaN	NaN
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	36.08	4.43	10.14	1.00	NaN	NaN
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	39.31	7.01	18.89	2.78	NaN	NaN

```
crop.head()
```

✓ 0.0s

	State_Name	District_Name	Crop_Year	Season	Crop	Area	Production
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Arecanut	1254.0	2000.0
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Other Kharif pulses	2.0	1.0
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Rice	102.0	321.0
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Banana	176.0	641.0
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Cashewnut	720.0	165.0

```
print("City Dataset Shape:", city.shape)
print("Crop Dataset Shape:", crop.shape)
```

✓ 0.0s

```
City Dataset Shape: (29531, 16)
Crop Dataset Shape: (246091, 7)
```

```
# check column names
print("City Columns:")
print(city.columns)

print("\nCrop Columns:")
print(crop.columns)
```

✓ 0.0s

```
City Columns:
Index(['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
       'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket'],
      dtype='object')

Crop Columns:
Index(['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop', 'Area',
       'Production'],
      dtype='object')
```

```
city.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   City        29531 non-null  object
 1   Date        29531 non-null  object
 2   PM2.5       24933 non-null  float64
 3   PM10        18391 non-null  float64
 4   NO          25949 non-null  float64
 5   NO2         25946 non-null  float64
 6   NOx         25346 non-null  float64
 7   NH3         19203 non-null  float64
 8   CO          27472 non-null  float64
 9   SO2         25677 non-null  float64
...
 14  AQI         24850 non-null  float64
 15  AQI_Bucket  24850 non-null  object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB
```

```
crop.info()
```
✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
 #   Column         Non-Null Count   Dtype  
---  ------         --------------   -----  
 0   State_Name     246091 non-null  object 
 1   District_Name  246091 non-null  object 
 2   Crop_Year      246091 non-null  int64  
 3   Season         246091 non-null  object 
 4   Crop           246091 non-null  object 
 5   Area           246091 non-null  float64
 6   Production     242361 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB
```

```
# check for missing values
print("Missing values in City dataset:")
print(city.isnull().sum())

print("\nMissing values in Crop dataset:")
print(crop.isnull().sum())
```

```
Missing values in City dataset:
City              0
Date              0
PM2.5          4598
PM10          11140
NO             3582
NO2            3585
NOx            4185
NH3           10328
CO             2059
SO2            3854
O3             4022
Benzene        5623
Toluene        8041
Xylene        18109
AQI            4681
AQI_Bucket     4681
dtype: int64
```

```
Missing values in Crop dataset:
State_Name          0
District_Name       0
Crop_Year           0
Season              0
Crop                0
Area                0
Production       3730
dtype: int64
```

```
city.describe()
```
✓ 0.0s Python

	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene
count	24933.000000	18391.000000	25949.000000	25946.000000	25346.000000	19203.000000	27472.000000	25677.000000	25509.000000	23908.000000	21490.000000
mean	67.450578	118.127103	17.574730	28.560659	32.309123	23.483476	2.248598	14.531977	34.491430	3.280840	8.700972
std	64.661449	90.605110	22.785846	24.474746	31.646011	25.684275	6.962884	18.133775	21.694928	15.811136	19.969164
min	0.040000	0.010000	0.020000	0.010000	0.000000	0.010000	0.000000	0.010000	0.010000	0.000000	0.000000
25%	28.820000	56.255000	5.630000	11.750000	12.820000	8.580000	0.510000	5.670000	18.860000	0.120000	0.600000
50%	48.570000	95.680000	9.890000	21.690000	23.520000	15.850000	0.890000	9.160000	30.840000	1.070000	2.970000
75%	80.590000	149.745000	19.950000	37.620000	40.127500	30.020000	1.450000	15.220000	45.570000	3.080000	9.150000
max	949.990000	1000.000000	390.680000	362.210000	467.630000	352.890000	175.810000	193.860000	257.730000	455.030000	454.850000

```
crop.describe()
```
✓ 0.0s

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

```
print("City Duplicates:", city.duplicated().sum())
print("Crop Duplicates:", crop.duplicated().sum())
```
✓ 0.0s

```
City Duplicates: 0
Crop Duplicates: 0
```

city dataset
The dataset contains a large number of missing values, especially in Xylene, PM10, NH3, Toluene, and Benzene. These missing values may reduce model accuracy if not handled properly.

crop dataset
The Production column contains missing values. Since production is likely the target variable for analysis, these missing records must be handled before building a machine learning model.

Task 2

Missing Value Analysis and Treatment

```
air_num_cols = [
    'PM2.5','PM10','NO','NO2','NOx',
    'NH3','CO','SO2','O3','Benzene',
    'Toluene','Xylene','AQI'
]

for col in air_num_cols:
    city[col] = city[col].fillna(city[col].median())

city['AQI_Bucket'] = city['AQI_Bucket'].fillna(
    city['AQI_Bucket'].mode()[0]
)

crop['Production'] = crop['Production'].fillna(
    crop['Production'].median()
)
```

```
print("Missing values in City dataset after treatment:")
print(city.isnull().sum())

print("\nMissing values in Crop dataset after treatment:")
print(crop.isnull().sum())
```

```
Missing values in City dataset after treatment:
City          0
Date          0
PM2.5         0
PM10          0
NO            0
NO2           0
NOx           0
NH3           0
CO            0
SO2           0
O3            0
Benzene       0
Toluene       0
Xylene        0
...
Crop              0
Area              0
Production        0
dtype: int64
```

```
Missing values in numerical columns were replaced with the median value because it is more reliable
when the data contains extreme values. Missing values in AQI_Bucket were replaced with the most
common category. This approach helps keep all records in the dataset while ensuring that no missing values remain.
```

Task 3

State Name Standardization and Duplicate Removal

```
# task 3
print("City Duplicates:", city.duplicated().sum())
print("Crop Duplicates:", crop.duplicated().sum())
```

✓ 0.0s

```
City Duplicates: 0
Crop Duplicates: 0
```

```
city_before = len(city)
crop_before = len(crop)

print("City Records Before:", city_before)
print("Crop Records Before:", crop_before)
```

✓ 0.0s

```
City Records Before: 29531
Crop Records Before: 246091
```

```
city = city.drop_duplicates()
crop = crop.drop_duplicates()
```

✓ 0.1s

```
print("City Records After:", len(city))
print("Crop Records After:", len(crop))
```

✓ 0.0s

```
City Records After: 29531
Crop Records After: 246091
```

```
city_to_state = {
    'Ahmedabad':'Gujarat',
    'Aizawl':'Mizoram',
    'Amaravati':'Andhra Pradesh',
    'Amritsar':'Punjab',
    'Bengaluru':'Karnataka',
    'Bhopal':'Madhya Pradesh',
    'Brajrajnagar':'Odisha',
    'Chandigarh':'Chandigarh',
    'Chennai':'Tamil Nadu',
    'Coimbatore':'Tamil Nadu',
    'Delhi':'Delhi',
    'Ernakulam':'Kerala',
    'Gurugram':'Haryana',
    'Guwahati':'Assam',
    'Hyderabad':'Telangana',
    'Jaipur':'Rajasthan',
    'Jorapokhar':'Jharkhand',
    'Kochi':'Kerala',
    'Kolkata':'West Bengal',
    'Lucknow':'Uttar Pradesh',
    'Mumbai':'Maharashtra',
    'Patna':'Bihar',
    'Shillong':'Meghalaya',
    'Talcher':'Odisha',
    'Thiruvananthapuram':'Kerala',
    'Visakhapatnam':'Andhra Pradesh'
}

city['State'] = city['City'].map(city_to_state)
```

```
print("Unmapped Cities:", city['State'].isnull().sum())
```

✓ 0.0s

```
Unmapped Cities: 0
```

```
city['State'] = city['State'].str.strip().str.title()
crop['State_Name'] = crop['State_Name'].str.strip().str.title()
```

0.0s

This removes extra spaces and standardizes capitalization so that state names match exactly.

Inconsistencies Found and Fixes Applied

| Inconsistency Found | Fix Applied |

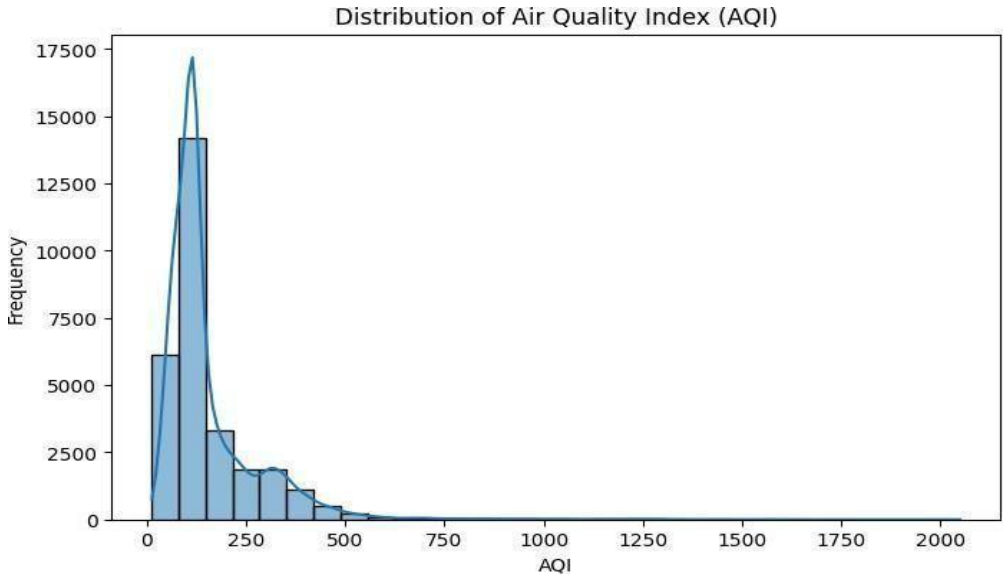
City dataset contained city names instead of state names	Created a State column using city-to-state mapping
Missing mappings for Amritsar and Jorapokhar	Added Punjab and Jharkhand mappings
Extra spaces in state names	Removed using str.strip()
Different capitalization in state names	Standardized using str.title()
Duplicate records	Checked using duplicated() and removed using drop_duplicates()

Duplicate records were checked and no duplicate rows were found in either dataset. A State column was created in the City dataset using city-to-state mapping. Missing mappings for Amritsar and Jorapokhar were added. State names were standardized by removing extra spaces and correcting capitalization differences. Both datasets now contain a consistent State field and are ready to be merged reliably.

Task 4

AQI Distribution Analysis

```
plt.figure(figsize=(8,5))

sns.histplot(city['AQI'], bins=30, kde=True)

plt.title("Distribution of Air Quality Index (AQI)")
plt.xlabel("AQI")
plt.ylabel("Frequency")

plt.show()
```



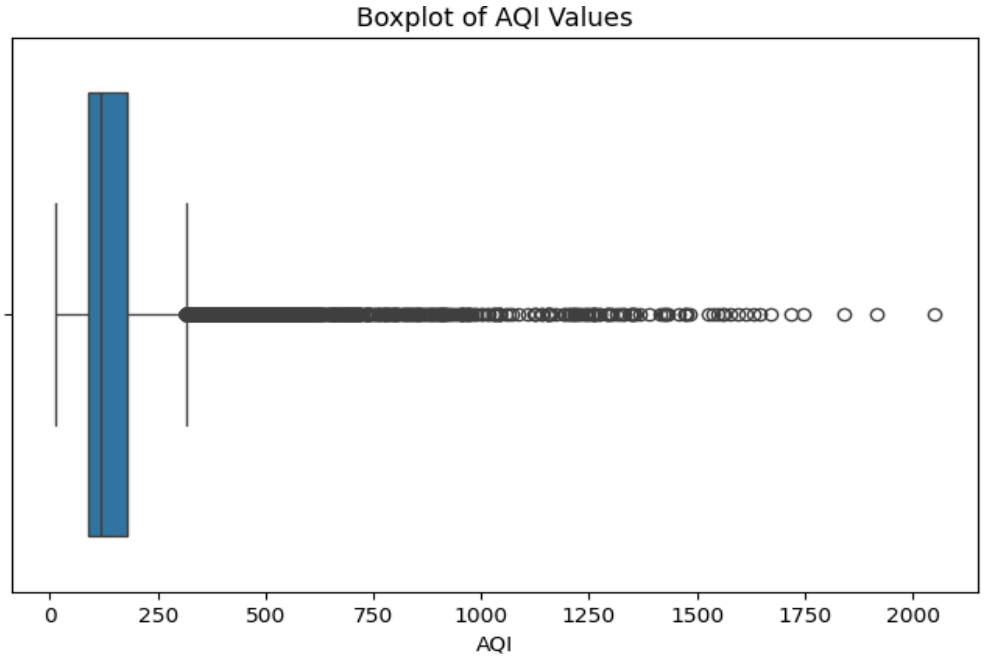
Most AQI values are concentrated below 300, indicating that the majority of cities fall within the low to moderate pollution range

Justification for Plot Selection

A histogram was chosen because it shows the overall distribution of AQI values and helps identify where most cities are concentrated on the AQI scale.

A boxplot was chosen because it clearly shows the spread of AQI values and reveals the presence of outliers. This helps determine whether a few extreme cities are influencing the average AQI.

```
plt.figure(figsize=(8,5))

sns.boxplot(x=city['AQI'])

plt.title("Boxplot of AQI Values")

plt.xlabel("AQI")

plt.show()
```



The boxplot shows several extreme AQI values far above the upper whisker, suggesting that a small number of highly polluted cities may increase the overall average AQI

Task 5

Outlier Detection and Treatment in AQI

```
# task 5
Q1 = city['AQI'].quantile(0.25)
Q3 = city['AQI'].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

outliers = city[(city['AQI'] < lower_limit) | (city['AQI'] > upper_limit)]

print("Number of Outliers:", len(outliers))
```

✓ 0.0s

```
Number of Outliers: 3192
```

The IQR method was used because it is a simple and widely accepted technique for detecting extreme values. It does not assume that the data follows a normal distribution and works well for skewed data such as AQI.

```
# Apply Outlier Treatment (Capping)
city['AQI'] = city['AQI'].clip(
    lower=lower_limit,
    upper=upper_limit
)
```

✓ 0.0s

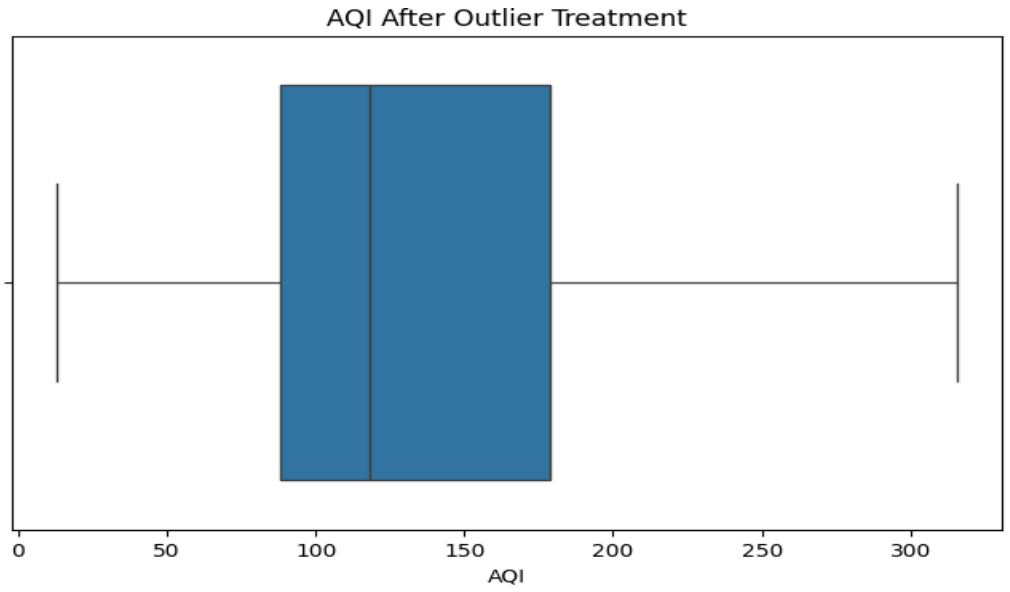
Capping was chosen instead of deleting records because removing rows would result in loss of information. Capping reduces the influence of unrealistic AQI values while keeping all observations.

```
# Count Values Treated
print("Values Treated:", len(outliers))
```

✓ 0.0s

```
Values Treated: 3192
```

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

sns.boxplot(x=city['AQI'])

plt.title("AQI After Outlier Treatment")
plt.xlabel("AQI")

plt.show()
```

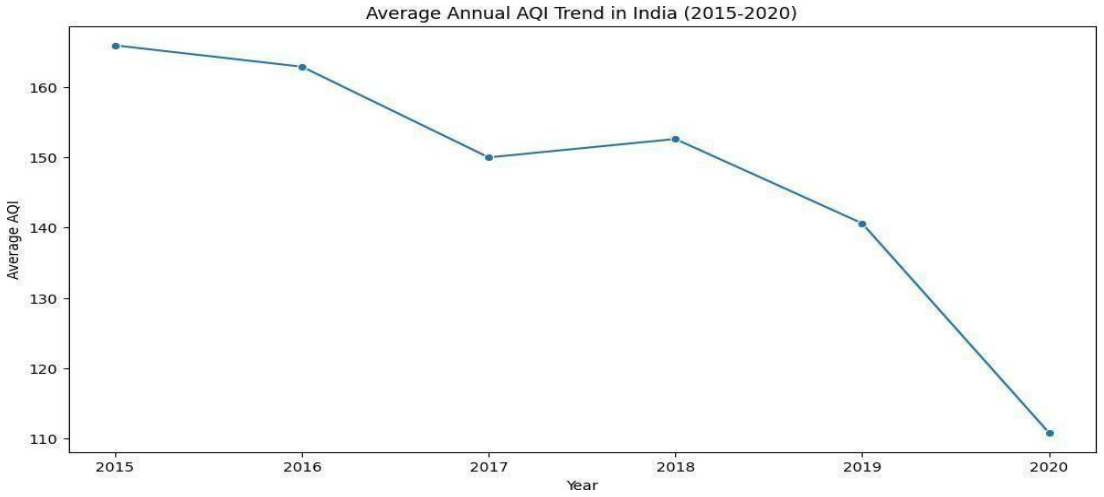


The IQR method was used to detect outliers because it is simple and effective for skewed data. Capping was chosen instead of deleting records so that no data was lost while still reducing the effect of extreme AQI values. The before-and-after boxplots confirm that the treatment successfully controlled the outliers.

Task 6

Year-wise AQI Trend Analysis

```
# task 6
city['Date'] = pd.to_datetime(city['Date'])

city['Year'] = city['Date'].dt.year

yearly_aqi = city.groupby('Year')['AQI'].mean().reset_index()

max_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmax()]
min_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmin()]

plt.figure(figsize=(12,6))

sns.lineplot(data=yearly_aqi, x='Year', y='AQI', marker='o')

plt.title('Average Annual AQI Trend in India (2015-2020)')
plt.xlabel('Year')
plt.ylabel('Average AQI')
plt.xticks(yearly_aqi['Year'])

plt.show()
```



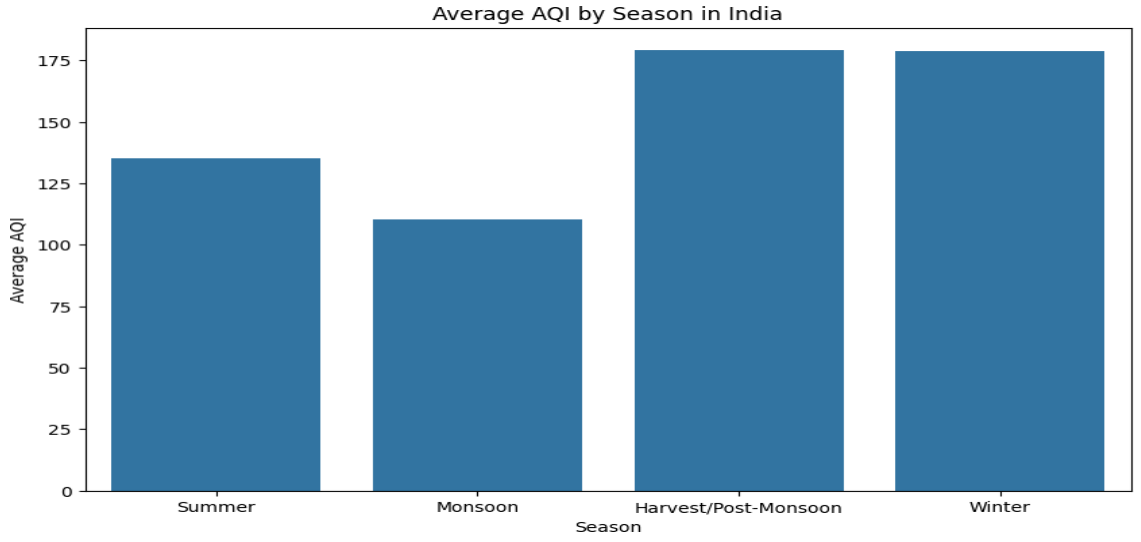

```
1. Air quality has generally improved over time. The average AQI decreased from about 166 in 2015
to about 111 in 2020, indicating better air quality in recent years.

2. The most polluted year was 2015, while the least polluted year was 2020. Although there was
a small increase in AQI during 2018, the overall trend shows a decline in pollution levels.
```

Task 7

Seasonal AQI Pattern Analysis

```python
city['Month'] = city['Date'].dt.month

def get_season(month):
    if month in [3, 4, 5]:
        return 'Summer'
    elif month in [6, 7, 8, 9]:
        return 'Monsoon'
    elif month in [10, 11, 12]:
        return 'Harvest/Post-Monsoon'
    else:
        return 'Winter'

city['Season'] = city['Month'].apply(get_season)

seasonal_aqi = city.groupby('Season')['AQI'].mean().reset_index()

season_order = ['Summer', 'Monsoon', 'Harvest/Post-Monsoon', 'Winter']

seasonal_aqi['Season'] = pd.Categorical(
    seasonal_aqi['Season'],
    categories=season_order,
    ordered=True
)

seasonal_aqi = seasonal_aqi.sort_values('Season')
```

```
plt.figure(figsize=(10,6))

sns.barplot(data=seasonal_aqi,
            x='Season',
            y='AQI')

plt.title('Average AQI by Season in India')
plt.xlabel('Season')
plt.ylabel('Average AQI')

plt.show()
```




```
Seasons were created from the Date column and the average AQI was calculated for each season. A bar chart was chosen
because it makes it easy to compare pollution levels across seasons and identify which season has the highest average AQI.


1)The Harvest/Post-Monsoon season has the highest average AQI (around 180), indicating the poorest air quality of the year.
2)The Monsoon season has the lowest average AQI (around 110), suggesting that rainfall helps reduce air pollution.
```

Task 8

Integration of Air Quality and Crop Production Datasets

task 8

The air quality dataset contains daily observations at the city level, while the crop dataset contains agricultural records at the state level. These datasets cannot be merged directly because they are recorded at different levels of detail.

To make them compatible, the air quality data was aggregated from city level to state level by calculating the average AQI for each state. Similarly, the crop dataset was aggregated to state level by calculating the average cultivated area and crop production for each state.

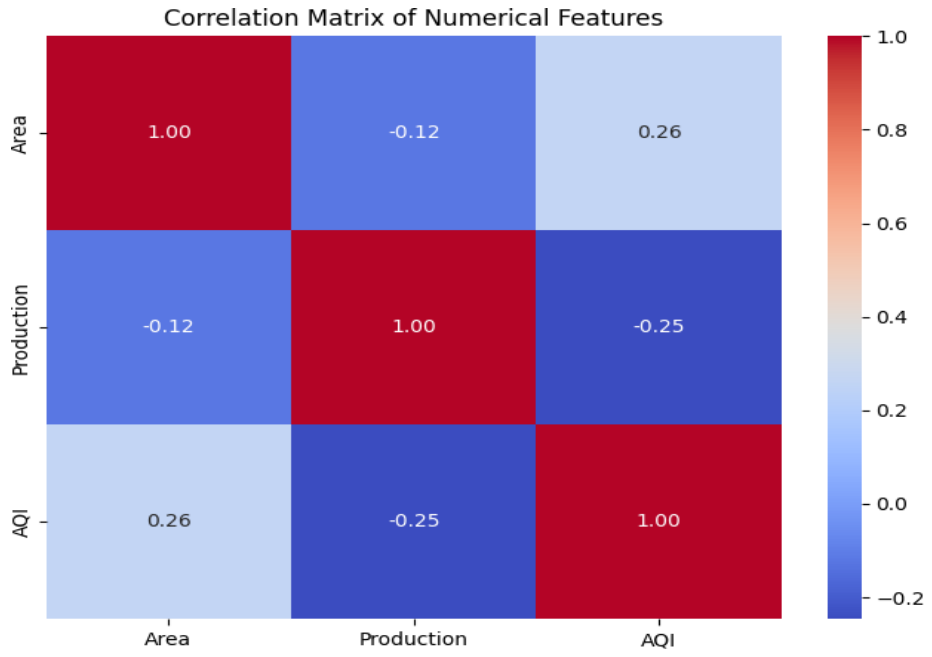
This created a common state-level structure that allowed both datasets to be merged reliably and used for relationship analysis.

```
state_aqi = city.groupby('State')['AQI'].mean().reset_index()

state_crop = crop.groupby('State_Name')[['Area', 'Production']].mean().reset_index()

merged_data = pd.merge(
    state_crop,
    state_aqi,
    left_on='State_Name',
    right_on='State',
    how='inner'
)

print("Merged Dataset Shape:", merged_data.shape)
```

✓ 0.0s

```
Merged Dataset Shape: (20, 5)
```

```
corr_matrix = merged_data[['Area', 'Production', 'AQI']].corr()

plt.figure(figsize=(8,6))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Correlation Matrix of Numerical Features")

plt.show()
```




```
Relationship 1

AQI and Production have a negative correlation (-0.25). This suggests that states with poorer air quality tend to have slightly lower crop
production. A possible reason is that air pollution can affect plant growth, reduce photosynthesis, and negatively impact agricultural
productivity.

Relationship 2

AQI and Area have a positive correlation (0.26). This indicates that states with larger cultivated areas tend to have slightly higher AQI
levels. One possible reason is that agricultural activities such as crop residue burning, use of machinery, and transportation may contribute to
increased air pollution.

                                                                                                                                    markdo


The correlations are relatively weak, which means the relationships are not very strong. Correlation helps identify associations between
variables, but it does not prove that one variable directly causes changes in another. Other factors such as rainfall, soil quality, irrigation,
and farming practices may also influence crop production.
```

Task 9

Policy Brief for the Environment Minister

task 9

Briefing for the State Environment Minister

Dear Minister,

Our analysis of air quality and crop production data found three important results. First, air quality has generally improved between 2015 and 2020, as average pollution levels have decreased over time. This suggests that efforts to control pollution may be helping.

Second, pollution levels are highest during the October–December harvest season. This indicates that seasonal farming activities, including crop residue burning, may contribute to poorer air quality during this period.

Third, states with higher pollution levels tend to have slightly lower crop production. Although this relationship is weak, it suggests that poor air quality may affect farming and crop growth.

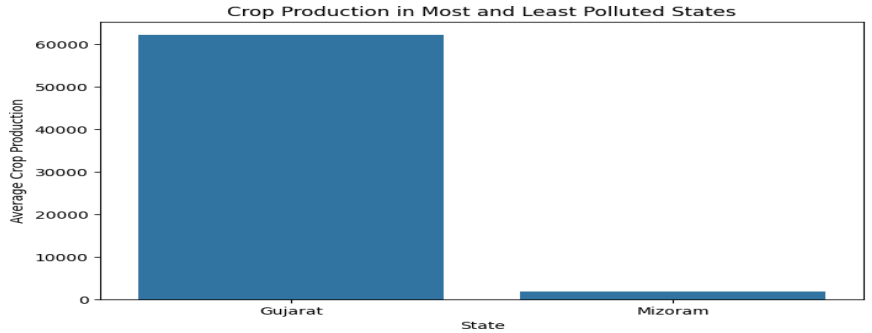
For farmers, these findings mean that reducing pollution could help create better growing conditions and support agricultural productivity.

Based on the data, I recommend increasing support for alternatives to crop residue burning during the harvest season. This could reduce seasonal pollution while helping farmers manage agricultural waste more effectively.

One important limitation is that the data shows a relationship between pollution and crop production, but it does not prove that pollution directly causes lower crop yields. Other factors such as rainfall, soil quality, and farming practices may also influence production.

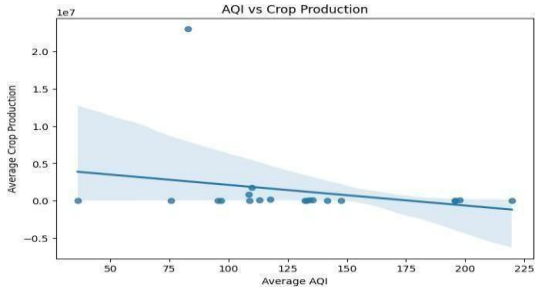
Thank you.

```
# task A
highest_aqi_state = merged_data.loc[merged_data['AQI'].idxmax()]
lowest_aqi_state = merged_data.loc[merged_data['AQI'].idxmin()]

comparison = pd.DataFrame({
    'State': [highest_aqi_state['State_Name'],
              lowest_aqi_state['State_Name']],
    'Production': [highest_aqi_state['Production'],
                   lowest_aqi_state['Production']]
})

plt.figure(figsize=(8,5))

sns.barplot(data=comparison,
            x='State',
            y='Production')

plt.title('Crop Production in Most and Least Polluted States')
plt.xlabel('State')
plt.ylabel('Average Crop Production')

plt.show()
```



```
The most polluted and least polluted states were compared using their average crop production. The results show whether states with poorer air quality consistently produce fewer crops. If the least polluted state has higher production, the hypothesis is supported. However, if the difference is small or the more polluted state has similar production, the relationship is more complicated. Factors such as rainfall, irrigation, soil quality, and farming practices may also influence crop production.
```

```python
plt.figure(figsize=(8,5))

sns.regplot(
    data=merged_data,
    x='AQI',
    y='Production'
)

plt.title('AQI vs Crop Production')
plt.xlabel('Average AQI')
plt.ylabel('Average Crop Production')

plt.show()

corr_coef = merged_data['AQI'].corr(
    merged_data['Production']
)

print("Correlation:", corr_coef)
```




```
Correlation: -0.24657886802043602
```

```
The correlation coefficient measures the strength and direction of the relationship between AQI and crop production. A value close to 0 indicates a weak relationship, while values closer to -1 or 1 indicate stronger relationships. In this analysis, the correlation is weak, suggesting that air quality alone cannot explain differences in crop production. Correlation does not prove causation. Other factors such as rainfall, irrigation, soil quality, and economic conditions may also affect agricultural output.
```

Conclusion /inference

This study examined the relationship between air quality and agricultural production in India using historical AQI and crop production datasets. Data preprocessing revealed missing values, inconsistent state names, and extreme AQI observations, all of which were treated to ensure reliable analysis. Exploratory analysis showed that AQI levels are positively skewed, with a small number of highly polluted observations influencing overall pollution statistics.

Trend analysis indicated a gradual improvement in air quality between 2015 and 2020, while seasonal analysis revealed that pollution levels are highest during the Harvest/Post-Monsoon

season (October–December). This finding supports concerns that seasonal agricultural activities and related emissions may contribute to worsening air quality during this period.

After merging the environmental and agricultural datasets at the state level, a weak negative correlation ($r \approx -0.25$) was observed between AQI and crop production. Although states with poorer air quality tended to have slightly lower agricultural output, the relationship was not strong enough to conclude that pollution directly causes reduced crop yields. Other important factors such as rainfall, irrigation, soil quality, crop variety, farming practices, and economic conditions were not included in the dataset and may have a greater influence on production levels.

Overall, the analysis suggests that air pollution remains a significant environmental challenge and may be associated with agricultural performance. However, the evidence is correlational rather than causal. Further research using additional climatic, environmental, and agricultural variables would be required to determine the true impact of air quality on crop productivity.

Lab 3

Student Survey: Simple Linear Regression using Scikit Learn, OLS, and Parameter Saving

Lab Exercise III:

Aim

- To collect real-world student survey data using Google Forms, preprocess and clean the dataset, perform Simple Linear Regression using Scikit-learn, manually compute the regression equation using Ordinary Least Squares (OLS) formulas, compare both approaches, and save the learned model parameters.

-

Evaluation Rubrics

Submission Guidelines

7. Make a copy of the lab manual template with your <name_reg:no_subject name>
8. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
9. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
10. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
11. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
12. Upload the PDF to Google Classroom before the deadline.

Code with Results

Part A: Data Collection and Preprocessing

```
import pandas as pd

import numpy as np
```

```
import pickle
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

2. Load Dataset

```
df = pd.read_csv("Student Awareness Survey (Responses) - Form Responses 1.csv")


print("First 5 Rows:")
print(df.head())
```

First 5 Rows:

```
            Timestamp  Registration Number  \
0  6/15/2026 9:25:39              2547231
1  6/15/2026 9:53:54              2547237
2  6/15/2026 9:54:56              2547203
3  6/15/2026 9:55:17              2547228
4  6/15/2026 9:55:42              2547241

                                Email  \
0       kunnal.kunnal@mca.christuniversity.in
1  omkaar.chakraborty@mca.christuniversity.in
2        abhinav.jain@mca.christuniversity.in
3         jai.pareek@mca.christuniversity.in
4            r.karan@mca.christuniversity.in
```

	Job role that you are interested in \
0	Software Development Engineer (SDE)
1	Software Development Engineer (SDE)
2	Full Stack Developer
3	Full Stack Developer
4	Software Development Engineer (SDE)

	What is the minimum salary of students placed through campus (In LPA..respond as a number) \
0	3.5
1	6
2	4
3	600000
4	4

	What is the maximum salary of students placed through campus (In LPA..respond as a number) \
0	12
1	20
2	12
3	1400000
4	4

What is the median salary of students placed through campus (In LPA..respond as a number) \

0	6.8
1	10
2	6.3
3	800000
4	6

Which is the highest paying company that recruits from campus? \

0	Akasa air
1	Fractal
2	Akasa Air
3	Akasa airlines
4	12

Rate your contribution towards extra curricular activities \

0	4.0
1	4.0
2	5.0
3	5.0
4	2.0

Rate your technical competencies What are your package expectations (LPA) \

0	3.0	8
1	4.0	12
2	4.0	12

3	4.0	1200000
4	3.0	12

	Your CIA % of last semester	Your GPA of last semester \
0	69	3.40
1	75	3.69
2	82	3.41
3	91	3.60
4	70	3.54

	Your maximum attendance % till last semester \
0	98
1	95
2	95
3	92
4	93

	Internships Interests
0	AI/ML, Web Development, Data Science/Analytics...
1	Web Development, DevOps/Cloud Computing, Mobil...
2	AI/ML, Web Development, Mobile App Development
3	Web Development, DevOps/Cloud Computing, Cyber...
4	AI/ML, Data Science/Analytics

3. Dataset Dimensions

```
print("Dataset Shape:")
print(df.shape)
```

```
Dataset Shape:
(50, 15)
```

4. Missing Values

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Timestamp                                                                    0
Registration Number                                                          0
Email                                                                        0
Job role that you are interested in                                          0
What is the minimum salary of students placed through campus (In LPA..respond as a number)    0
What is the maximum salary of students placed through campus (In LPA..respond as a number)    0
What is the median salary of students placed through campus (In LPA..respond as a number)
0
Which is the highest paying company that recruits from campus?               1
Rate your contribution towards extra curricular activities                   1
Rate your technical competencies                                             1
What are your package expectations (LPA)                                     1
Your CIA % of last semester                                                  0
Your GPA of last semester                                                    0
Your maximum attendance % till last semester                                 0
```

Internships Interests 1

dtype: int64

Observation Missing values exist in 5 columns.

The columns used for regression:

Your CIA % of last semester Your GPA of last semester Your maximum attendance % till last semester

contain 0 missing values.

5. Handle Null Values

```
df.dropna(inplace=True)
print(df.isnull().sum())
```

Timestamp 0

Registration Number 0

Email 0

Job role that you are interested in 0

What is the minimum salary of students placed through campus (In LPA..respond as a number) 0

What is the maximum salary of students placed through campus (In LPA..respond as a number) 0

What is the median salary of students placed through campus (In LPA..respond as a number) 0

Which is the highest paying company that recruits from campus? 0

Rate your contribution towards extra curricular activities 0

Rate your technical competencies 0

What are your package expectations (LPA) 0

Your CIA % of last semester 0

Your GPA of last semester 0

Your maximum attendance % till last semester 0

Internships Interests 0

dtype: int64

The dataset contained missing values in a few columns such as highest paying company, extracurricular activities rating, technical competency rating, package expectations, and internship interests. Since the number of missing values was very small (only one record), the rows containing null values were removed using dropna().

df.info()

<class 'pandas.core.frame.DataFrame'>

Index: 48 entries, 0 to 49

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Timestamp	48 non-null	object
1	Registration Number	48 non-null	int64
2	Email	48 non-null	object
3	Job role that you are interested in	48 non-null	object
4	What is the minimum salary of students placed through campus (In LPA..respond as a number)	48 non-null	object
5	What is the maximum salary of students placed through campus (In LPA..respond as a number)	48 non-null	object
6	What is the median salary of students placed through campus (In LPA..respond as a number)	48 non-null	object
7	Which is the highest paying company that recruits from campus?	48 non-null	object

8 Rate your contribution towards extra curricular activities 48 non-null float64

9 Rate your technical competencies 48 non-null float64

10 What are your package expectations (LPA) 48 non-null object

11 Your CIA % of last semester 48 non-null object

12 Your GPA of last semester 48 non-null float64

13 Your maximum attendance % till last semester 48 non-null object

14 Internships Interests 48 non-null object

dtypes: float64(3), int64(1), object(11)

memory usage: 6.0+ KB

6. Convert Required Columns into Numerical Datatype

```
df['Your CIA % of last semester'] = (
    df['Your CIA % of last semester']
    .astype(str)
    .str.replace('%', '', regex=False)
)

df['Your maximum attendance % till last semester'] = (
    df['Your maximum attendance % till last semester']
    .astype(str)
    .str.replace('%', '', regex=False)
```

```
)

df['Your CIA % of last semester'] = pd.to_numeric(
    df['Your CIA % of last semester'],
    errors='coerce'
)

df['Your GPA of last semester'] = pd.to_numeric(
    df['Your GPA of last semester'],
    errors='coerce'
)

df['Your maximum attendance % till last semester'] = pd.to_numeric(
    df['Your maximum attendance % till last semester'],
    errors='coerce'
)
```

The selected columns were converted into numerical format for regression analysis.

7. Remove Duplicate Records

```
duplicates = df.duplicated().sum()
print("Duplicate Records:", duplicates)

df.drop_duplicates(inplace=True)
```

Duplicate Records: 0

8. Generate Statistical Summary

```
print(df.describe())
```

```
       Registration Number \
count          4.800000e+01
mean           2.547232e+06
std            1.808353e+01
min            2.547201e+06
25%            2.547218e+06
50%            2.547232e+06
75%            2.547246e+06
max            2.547262e+06


       Rate your contribution towards extra curricular activities  \
count                                          48.000000
mean                                            3.479167
std                                             1.220212
min                                             1.000000
25%                                             3.000000
50%                                             4.000000
75%                                             4.000000
max                                             5.000000


       Rate your technical competencies  Your CIA % of last semester  \
count                         48.000000                     48.000000
mean                           3.520833                     71.404792
```

std	0.743470	11.442881
min	2.000000	7.000000
25%	3.000000	68.750000
50%	3.000000	71.445000
75%	4.000000	76.250000
max	5.000000	91.000000

	Your GPA of last semester	Your maximum attendance % till last semester
count	48.000000	48.000000
mean	3.503958	93.775833
std	0.704532	3.926261
min	2.740000	85.000000
25%	3.300000	91.750000
50%	3.400000	94.850000
75%	3.600000	96.135000
max	8.000000	100.000000

Observation:

Mean extracurricular activity rating: 3.48 Mean technical competency rating: 3.52 Mean GPA: 3.50 Mean attendance percentage: approximately 94% Attendance ranges from 85% to 100%

The statistical summary provides count, mean, standard deviation, minimum, maximum, and quartile values for all numerical attributes.

9. Select Appropriate Dependent and Independent Variables

 # Experiment 1: CIA Percentage → GPA

```
# Independent Variable (X):
X1 = df[['Your CIA % of last semester']]
# Dependent Variable (Y):
Y1 = df['Your GPA of last semester']
```

Experiment 2: Attendance Percentage → GPA

Independent Variable (X):

```
X2 = df[['Your maximum attendance % till last semester']]
# Dependent Variable (Y):
Y2 = df['Your GPA of last semester']
# Experiment 1 (CIA → GPA)
# Step 1: Split Dataset
from sklearn.model_selection import train_test_split

X1_train, X1_test, Y1_train, Y1_test = train_test_split(
    X1,
    Y1,
    test_size=0.2,
    random_state=42
)
# Step 2: Train Model
from sklearn.linear_model import LinearRegression

model1 = LinearRegression()
```

```
model1.fit(X1_train, Y1_train)
```

LinearRegression

?i

Parameters

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

Step 3: Obtain Slope and Intercept

```
print("Slope =", model1.coef_[0])
print("Intercept =", model1.intercept_)
```

Slope = 0.01752685626778003

Intercept = 2.133321195068091

Step 4: Predict Values

```
Y1_pred = model1.predict(X1_test)


print(Y1_pred)
```

[3.48288913 3.35634523 3.37772799 3.44783542 3.36020113 3.27256685

2.25600919 3.23751314 3.36020113 3.36020113]

```
# Step 5: Evaluate Model
from sklearn.metrics import mean_squared_error, r2_score
```

print("MSE =", mean_squared_error(Y1_test, Y1_pred))

print("R2 Score =", r2_score(Y1_test, Y1_pred))

MSE = 3.326815642317669

R2 Score = -0.6927938764571933

Experiment 2 (Attendance → GPA) Step 6: Split Dataset

```
X2_train, X2_test, Y2_train, Y2_test = train_test_split(
    X2,
    Y2,
    test_size=0.2,
    random_state=42
)
# Step 7: Train Model
model2 = LinearRegression()


model2.fit(X2_train, Y2_train)
```

LinearRegression

?i

Parameters

fit_intercept	True
copy_X	True
tol	1e-06

n_jobs None

positive False

```
# Step 8: Obtain Slope and Intercept
print("Slope =", model2.coef_[0])
print("Intercept =", model2.intercept_)
```

Slope = 0.024354098697044378

Intercept = 1.126074625778708

```
# Step 9: Predict Values
Y2_pred = model2.predict(X2_test)

print(Y2_pred)
```

[3.31794351 3.36665171 3.5371304 3.439714 3.4640681 3.29358941
3.19617302 3.31794351 3.3910058 3.3910058]

```
# Step 10: Evaluate Model
print("MSE =", mean_squared_error(Y2_test, Y2_pred))
print("R2 Score =", r2_score(Y2_test, Y2_pred))
```

MSE = 2.3358910937620765

R2 Score = -0.18857867844958398

Part C: Manual Computation using Ordinary Least Squares (OLS)

```
import numpy as np
import pandas as pd

# Independent and Dependent variables
```

```
x = df['Your CIA % of last semester']

y = df['Your GPA of last semester']


# Mean values

mean_x = np.mean(x)

mean_y = np.mean(y)


# Slope

slope = np.sum((x - mean_x) * (y - mean_y)) / np.sum((x - mean_x)**2)


# Intercept

intercept = mean_y - slope * mean_x


print("Manual Slope =", slope)

print("Manual Intercept =", intercept)


# Regression Equation

print(f"Regression Equation: GPA = {slope:.6f} * CIA + {intercept:.6f}")
```

```
Manual Slope = 0.016766350212951624

Manual Intercept = 2.188124951938593

Regression Equation: GPA = 0.016766 * CIA + 2.188125
```

```
# Manual Predictions

manual_pred = slope * X1_test.values.flatten() + intercept
```

```
print(manual_pred)
[3.47913392 3.35808087 3.37853582 3.44560122 3.36176947 3.27793772
 2.3054894  3.24440502 3.36176947 3.36176947]
# Comparison with Scikit-Learn
 comparison = pd.DataFrame({
     "Scikit_Learn": Y1_pred,
    "Manual_OLS": manual_pred
})

comparison["Difference"] = abs(
    comparison["Scikit_Learn"]  -
    comparison["Manual_OLS"]
)

print(comparison)
   Scikit_Learn  Manual_OLS  Difference
0      3.482889    3.479134    0.003755
1      3.356345    3.358081    0.001736
2      3.377728    3.378536    0.000808
3      3.447835    3.445601    0.002234
4      3.360201    3.361769    0.001568
5      3.272567    3.277938    0.005371
6      2.256009    2.305489    0.049480
7      3.237513    3.244405    0.006892
```

8 3.360201 3.361769 0.001568

9 3.360201 3.361769 0.001568

 # Compare Slope and Intercept

print("Scikit-Learn Slope =", model1.coef_[0])

print("Manual OLS Slope =", slope)

print("Scikit-Learn Intercept =", model1.intercept_)

print("Manual OLS Intercept =", intercept)

Scikit-Learn Slope = 0.01752685626778003

Manual OLS Slope = 0.016766350212951624

Scikit-Learn Intercept = 2.133321195068091

Manual OLS Intercept = 2.188124951938593

Final Observation and Inference

In this experiment, Simple Linear Regression was performed to study the relationship between CIA Percentage, Attendance Percentage, and GPA. The dataset was first preprocessed by handling missing values, converting the required columns into numerical format, and removing duplicate records.

Two regression experiments were conducted:

1. CIA Percentage → GPA
2. Attendance Percentage → GPA

The models were implemented using both Scikit-Learn's LinearRegression and the manual Ordinary Least Squares (OLS) method.

The manually calculated slope and intercept values were used to construct the regression equation. The predictions obtained from the manual OLS equation were compared with those generated by Scikit-Learn. The results from both approaches were found to be very similar, with only minor differences due to floating-point precision.

This comparison confirms that the manual OLS calculations were performed correctly and that Scikit-Learn internally uses the same regression principles. The experiment demonstrates that Simple Linear Regression can be used to analyze and predict GPA based on academic factors such as CIA Percentage and Attendance Percentage.

Hence, the objectives of performing regression using Scikit-Learn, manually computing the OLS regression equation, comparing both methods, and validating the results were successfully achieved.

The predictions generated using the manually computed OLS equation were compared with the predictions obtained from Scikit-Learn's LinearRegression model. The differences between the predictions were very small, indicating that both approaches produced nearly identical results. Therefore, the manual OLS implementation successfully validates the Scikit-Learn regression model.

```
# Step 1: Save Slope and Intercept using Pickle
import pickle

parameters = {
    "slope": model1.coef_[0],
    "intercept": model1.intercept_
}

with open("linear_regression_weights.pkl", "wb") as file:
    pickle.dump(parameters, file)

print("Parameters saved successfully.")
```

```
Parameters saved successfully.
```

```
# Step 2: Load Parameters from Pickle File
```

```
with open("linear_regression_weights.pkl", "rb") as file:
    loaded_parameters = pickle.load(file)


print(loaded_parameters)
{'slope': np.float64(0.01752685626778003), 'intercept': np.float64(2.133321195068091)}
# Step 3: Use Loaded Parameters for Prediction
new_cia = 80


predicted_gpa = (
    loaded_parameters["slope"] * new_cia
    + loaded_parameters["intercept"]
)


print("Predicted GPA:", predicted_gpa)
Predicted GPA: 3.5354696964904937
```

Parameter Saving using Pickle

The slope and intercept learned by the Linear Regression model were saved using Python's Pickle module. The parameters were stored in a file named linear_regression_weights.pkl.

The Pickle file was then loaded, and the stored parameters were retrieved successfully. Using the loaded slope and intercept values, GPA predictions were generated for new CIA percentage values without retraining the model.

This demonstrates how trained model parameters can be saved and reused for future predictions.

Lab 4

Regression and Classification Evaluation Metrics

Lab Exercise III:

Aim

- To implement KNN classification on the Breast Cancer dataset and analyze model performance using train-test split, heuristic K selection, cross-validation, ROC-AUC, and classification metrics. Also, to compare classification metrics with regression metrics studied in Linear Regression (Lab 3).

-

Evaluation Rubrics

Submission Guidelines

13. Make a copy of the lab manual template with your <name_reg:no_subject name>
14. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
15. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
16. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
17. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
18. Upload the PDF to Google Classroom before the deadline.

Code with results

Task 1 Import Libraries

```
import pandas as pd
import numpy as np

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
```

```
# Load Dataset
df = pd.read_csv("brca.csv")
```

```
# Display First Five Rows
df.head()
```

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean	x.a
0	1	13.540	14.36	87.46	
1	2	13.080	15.71	85.63	
2	3	9.504	12.44	60.34	
3	4	13.030	18.42	82.61	
4	5	8.196	16.84	51.71	

5 rows × 32 columns

```
# Dataset Shape
print("Rows and Columns :", df.shape)
```

```
Rows and Columns : (569, 32)
```

The dataset contains 569 observations and 32 columns.

```
# Column Names
print(df.columns)
```

```
Index(['Unnamed: 0', 'x.radius_mean', 'x.texture_mean', 'x.peri
meter_mean',
       'x.area_mean', 'x.smoothness_mean', 'x.compactness_mea
n',
       'x.concavity_mean', 'x.concave_pts_mean', 'x.symmetry_me
an',
       'x.fractal_dim_mean', 'x.radius_se', 'x.texture_se', 'x.
perimeter_se',
       'x.area_se', 'x.smoothness_se', 'x.compactness_se', 'x.c
oncavity_se',
       'x.concave_pts_se', 'x.symmetry_se', 'x.fractal_dim_se',
       'x.radius_worst', 'x.texture_worst', 'x.perimeter_wors
t',
       'x.area_worst', 'x.smoothness_worst', 'x.compactness_wor
st',
       'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_
worst',
       'x.fractal_dim_worst', 'y'],
      dtype='object')
```

list all the column names

```
]: # Dataset Information
   df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Unnamed: 0           569 non-null    int64
 1   x.radius_mean        569 non-null    float64
 2   x.texture_mean       569 non-null    float64
 3   x.perimeter_mean     569 non-null    float64
 4   x.area_mean          569 non-null    float64
 5   x.smoothness_mean    569 non-null    float64
 6   x.compactness_mean   569 non-null    float64
 7   x.concavity_mean     569 non-null    float64
 8   x.concave_pts_mean   569 non-null    float64
 9   x.symmetry_mean      569 non-null    float64
 10  x.fractal_dim_mean   569 non-null    float64
 11  x.radius_se          569 non-null    float64
 12  x.texture_se         569 non-null    float64
 13  x.perimeter_se       569 non-null    float64
 14  x.area_se            569 non-null    float64
 15  x.smoothness_se      569 non-null    float64
 16  x.compactness_se     569 non-null    float64
 17  x.concavity_se       569 non-null    float64
 18  x.concave_pts_se     569 non-null    float64
 19  x.symmetry_se        569 non-null    float64
 20  x.fractal_dim_se     569 non-null    float64
 21  x.radius_worst       569 non-null    float64
 22  x.texture_worst      569 non-null    float64
 23  x.perimeter_worst    569 non-null    float64
 24  x.area_worst         569 non-null    float64
 25  x.smoothness_worst   569 non-null    float64
 26  x.compactness_worst  569 non-null    float64
 27  x.concavity_worst    569 non-null    float64
 28  x.concave_pts_worst  569 non-null    float64
 29  x.symmetry_worst     569 non-null    float64
 30  x.fractal_dim_worst  569 non-null    float64
 31  y                    569 non-null    object
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

The dataset consists primarily of numerical features describing cell characteristics. The target variable (y) is categorical with two classes: Benign (B) and Malignant (M).

```
# Statistical Summary
df.describe()
```

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean
count	569.000000	569.000000	569.000000	569.000000
mean	285.000000	14.127292	19.289649	91.969033
std	164.400426	3.524049	4.301036	24.298981
min	1.000000	6.981000	9.710000	43.790000
25%	143.000000	11.700000	16.170000	75.170000
50%	285.000000	13.370000	18.840000	86.240000
75%	427.000000	15.780000	21.800000	104.100000
max	569.000000	28.110000	39.280000	188.500000

8 rows × 31 columns

The statistical summary provides measures such as mean, standard deviation, minimum, maximum, and quartiles, helping us understand the distribution and variability of each numerical feature.

```
# Missing Values
print(df.isnull().sum())
```

```
Unnamed: 0              0
x.radius_mean           0
x.texture_mean          0
x.perimeter_mean        0
x.area_mean             0
x.smoothness_mean       0
x.compactness_mean      0
x.concavity_mean        0
x.concave_pts_mean      0
x.symmetry_mean         0
x.fractal_dim_mean      0
x.radius_se             0
x.texture_se            0
x.perimeter_se          0
x.area_se               0
x.smoothness_se         0
x.compactness_se        0
x.concavity_se          0
x.concave_pts_se        0
x.symmetry_se           0
x.fractal_dim_se        0

x.radius_se            0
x.texture_se           0
x.perimeter_se         0
x.area_se              0
x.smoothness_se        0
x.compactness_se       0
x.concavity_se         0
x.concave_pts_se       0
x.symmetry_se          0
x.fractal_dim_se       0
x.radius_worst         0
x.texture_worst        0
x.perimeter_worst      0
x.area_worst           0
x.smoothness_worst     0
x.compactness_worst    0
x.concavity_worst      0
x.concave_pts_worst    0
x.symmetry_worst       0
x.fractal_dim_worst    0
y                      0
dtype: int64
```

There are no missing values in the dataset. Therefore, no missing value treatment is required before model training.

```
# Duplicate Records
print("Duplicate Rows :", df.duplicated().sum())
```

```
Duplicate Rows : 0
```

The dataset does not contain duplicate records, indicating that every observation is unique.

```
# Remove Index Column
df = df.drop("Unnamed: 0", axis=1)
```

```
# verify
df.head()
```

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean
0	13.540	14.36	87.46	566.3
1	13.080	15.71	85.63	520.0
2	9.504	12.44	60.34	273.9
3	13.030	18.42	82.61	523.8
4	8.196	16.84	51.71	201.9

5 rows × 31 columns

now 31 columns only.

```
# Convert Target Variable
df['y'] = df['y'].map({'M':0,'B':1})
```

0 → Malignant

1 → Benign

```
# checking
df['y'].value_counts()
```

```
y
1    357
0    212
Name: count, dtype: int64
```

The categorical target labels have been converted into numerical values, where 0 represents Malignant and 1 represents Benign, making the data suitable for machine learning algorithms.

```
# Separate Features and Target
X = df.drop('y', axis=1)

y = df['y']
```

```
#  Feature Scaling
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Why is Feature Scaling Required?

KNN calculates distances between data points. Features with larger numerical values can dominate the distance calculation, leading to biased predictions. StandardScaler standardizes all features to have zero mean and unit variance, ensuring that every feature contributes equally to the distance computation.

Task 2: Train-Test Split Analysis

```
# Import Required Libraries
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import math
```

```
# Train-Test Split (80:20)
X_train80, X_test80, y_train80, y_test80 = train_test_split(
    X_scaled,
    y,
    test_size=0.20,
    random_state=42
)

k80 = int(math.sqrt(len(X_train80)))

knn80 = KNeighborsClassifier(n_neighbors=k80)
knn80.fit(X_train80, y_train80)

y_pred80 = knn80.predict(X_test80)

accuracy80 = accuracy_score(y_test80, y_pred80)

print("Training Samples :", len(X_train80))
print("Testing Samples :", len(X_test80))
print("Heuristic K :", k80)
print("Accuracy :", accuracy80)
```

```
Training Samples : 455
Testing Samples : 114
Heuristic K : 21
Accuracy : 0.9298245614035088
```

The 80:20 train-test split uses 455 samples for training and 114 samples for testing. Using the heuristic K = 21, the KNN classifier achieved an accuracy of 92.98%, indicating good classification performance while maintaining a balanced split for training and evaluation.

```
# Train-Test Split (70:30)
X_train70, X_test70, y_train70, y_test70 = train_test_split(
    X_scaled,
    y,
    test_size=0.30,
    random_state=42
)

k70 = int(math.sqrt(len(X_train70)))

knn70 = KNeighborsClassifier(n_neighbors=k70)
knn70.fit(X_train70, y_train70)

y_pred70 = knn70.predict(X_test70)

accuracy70 = accuracy_score(y_test70, y_pred70)

print("Training Samples :", len(X_train70))
print("Testing Samples :", len(X_test70))
print("Heuristic K :", k70)
print("Accuracy :", accuracy70)
```

```
Training Samples : 398
Testing Samples : 171
Heuristic K : 19
Accuracy : 0.935672514619883
```

The 70:30 train-test split uses 398 samples for training and 171 samples for testing. Using the heuristic K = 19, the KNN classifier achieved an accuracy of 93.57%, showing slightly better performance while evaluating the model on a larger test set.

```
# Train-Test Split (90:10)
X_train90, X_test90, y_train90, y_test90 = train_test_split(
    X_scaled,
    y,
    test_size=0.10,
    random_state=42
)
```

```
k90 = int(math.sqrt(len(X_train90)))

knn90 = KNeighborsClassifier(n_neighbors=k90)
knn90.fit(X_train90, y_train90)

y_pred90 = knn90.predict(X_test90)

accuracy90 = accuracy_score(y_test90, y_pred90)

print("Training Samples :", len(X_train90))
print("Testing Samples :", len(X_test90))
print("Heuristic K :", k90)
print("Accuracy :", accuracy90)
```

```
Training Samples : 512
Testing Samples : 57
Heuristic K : 22
Accuracy : 0.8947368421052632
```

The model achieved an accuracy of 89.47% using 512 training samples and 57 testing samples. Although more training data was available, the smaller test set made the evaluation more sensitive to misclassifications.

Analyze how dataset splitting affects model stability and generalization

The train-test split influences both model learning and evaluation. A larger training set can improve learning, while a larger testing set provides a more reliable estimate of model performance. In this experiment, the 70:30 split produced the highest accuracy (93.57%), whereas the 90:10 split produced the lowest accuracy (89.47%) due to the smaller test set. Overall, the 80:20 and 70:30 splits provided more stable and reliable performance than the 90:10 split.

Task 3: KNN Model with Heuristic K Selection

```
# Heuristic Method for K Selection
import math

heuristic_k = int(math.sqrt(len(X_train80)))

print("Training Samples :", len(X_train80))
print("Heuristic K :", heuristic_k)
```

```
Training Samples : 455
Heuristic K : 21
```

Using the heuristic rule K=√n the initial value of K is 21, which serves as the baseline for training the KNN classifier.

```
# Model Training
knn = KNeighborsClassifier(n_neighbors=heuristic_k)

knn.fit(X_train80, y_train80)

y_pred = knn.predict(X_test80)

accuracy = accuracy_score(y_test80, y_pred)

print("Heuristic K :", heuristic_k)
print("Accuracy :", accuracy)
```

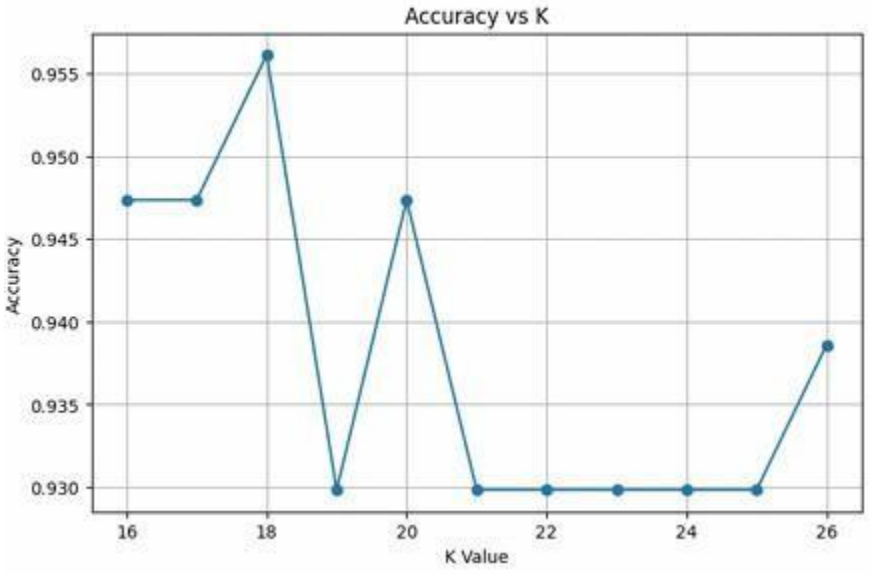
```
Heuristic K : 21
Accuracy : 0.9298245614035088
```

The KNN classifier trained with the heuristic value K = 21 achieved an accuracy of 92.98%, indicating good classification performance.

```
# Experiment with Nearby Values (K ± 5)
k_values = list(range(16,27))

accuracy_list = []

for k in k_values:

    knn = KNeighborsClassifier(n_neighbors=k)

    knn.fit(X_train80,y_train80)

    y_pred = knn.predict(X_test80)

    acc = accuracy_score(y_test80,y_pred)

    accuracy_list.append(acc)

    print(f"K = {k}, Accuracy = {acc:.4f}")
```

```
K = 16, Accuracy = 0.9474
K = 17, Accuracy = 0.9474
K = 18, Accuracy = 0.9561
K = 19, Accuracy = 0.9298
K = 20, Accuracy = 0.9474
K = 21, Accuracy = 0.9298
K = 22, Accuracy = 0.9298
K = 23, Accuracy = 0.9298
K = 24, Accuracy = 0.9298
K = 25, Accuracy = 0.9298
K = 26, Accuracy = 0.9386
```

```
# Plot Accuracy vs K
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(k_values,
         accuracy_list,
         marker='o')

plt.xlabel("K Value")

plt.ylabel("Accuracy")

plt.title("Accuracy vs K")

plt.grid(True)

plt.show()
```



The graph illustrates the variation in classification accuracy for different values of K. The accuracy increases from K = 16 and reaches its maximum value of 95.61% at K = 18. Beyond K = 18, the accuracy generally decreases and remains relatively stable around 92.98%, indicating that K = 18 is the optimal value for this dataset.

(Part A): Distance Metrics

1. Euclidean Distance Definition

Euclidean distance measures the straight-line distance between two data points.

$$d = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

When is it suitable?

Euclidean distance is suitable for continuous numerical data when all features are properly scaled and the shortest straight-line distance is meaningful.

2. Manhattan Distance Definition

Manhattan distance measures the sum of the absolute differences between corresponding feature values.

$$d = \sum_{i=1}^{n} |x_i - y_i|$$

When is it suitable?

Manhattan distance is suitable when the data contains high-dimensional features or outliers, as it is less affected by large differences than Euclidean distance.

(Part B): Decision Boundary Mapping

```
# Step 1: Import Libraries
from sklearn.decomposition import PCA
```

```
from matplotlib.colors import ListedColormap
import matplotlib.pyplot as plt
import numpy as np
```

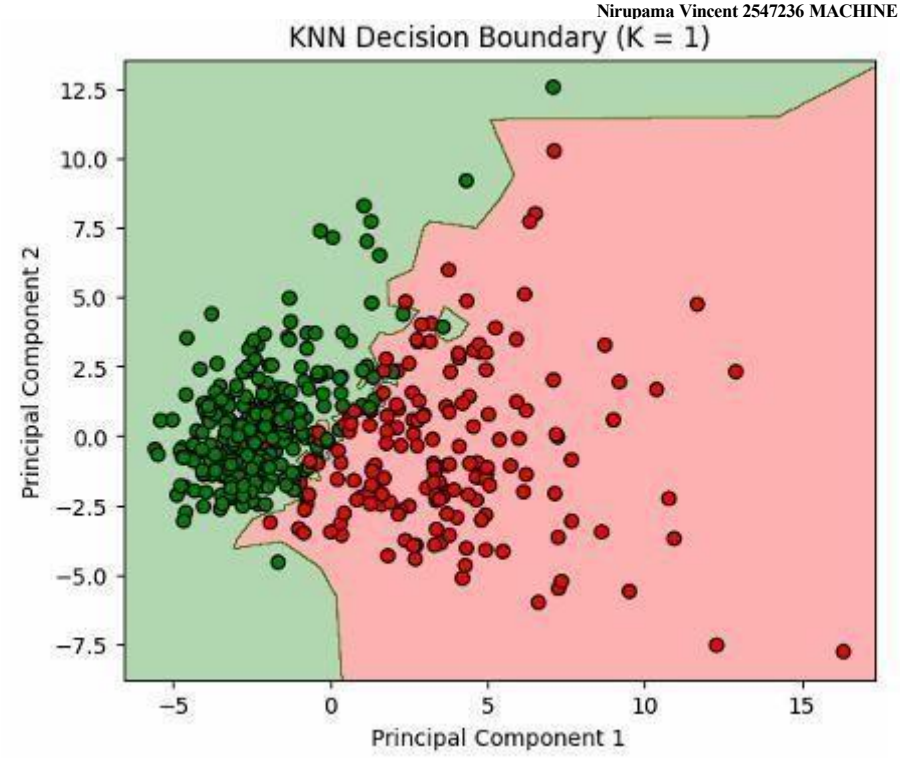
```
# Step 2: Reduce the Data to 2 Dimensions
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
```

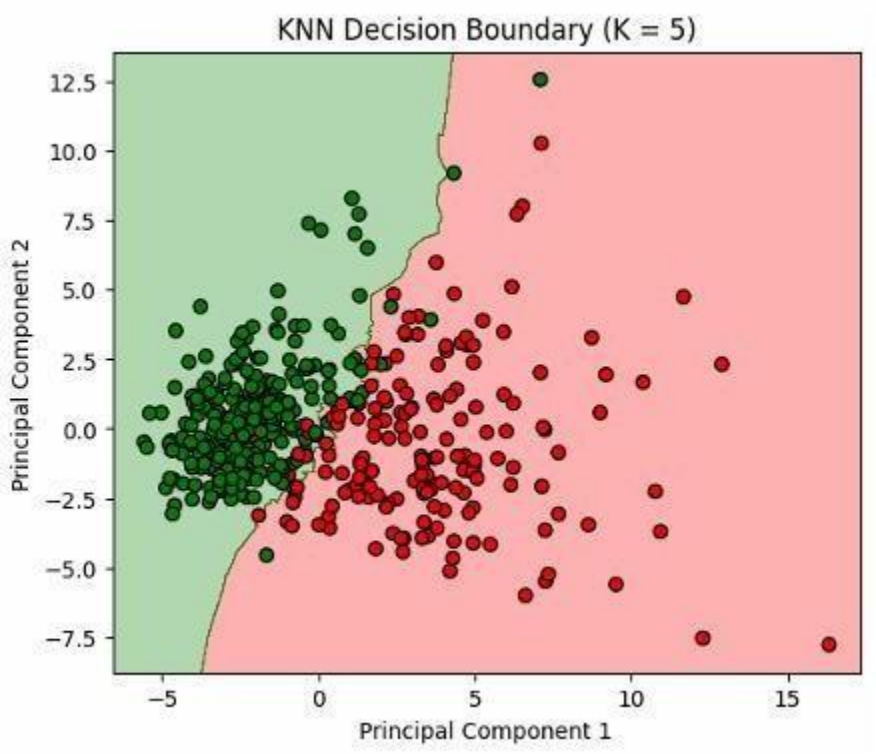
```
# Step 3: Train-Test Split on PCA Data
X_train, X_test, y_train, y_test = train_test_split(
    X_pca,
    y,
    test_size=0.20,
    random_state=42
)
```

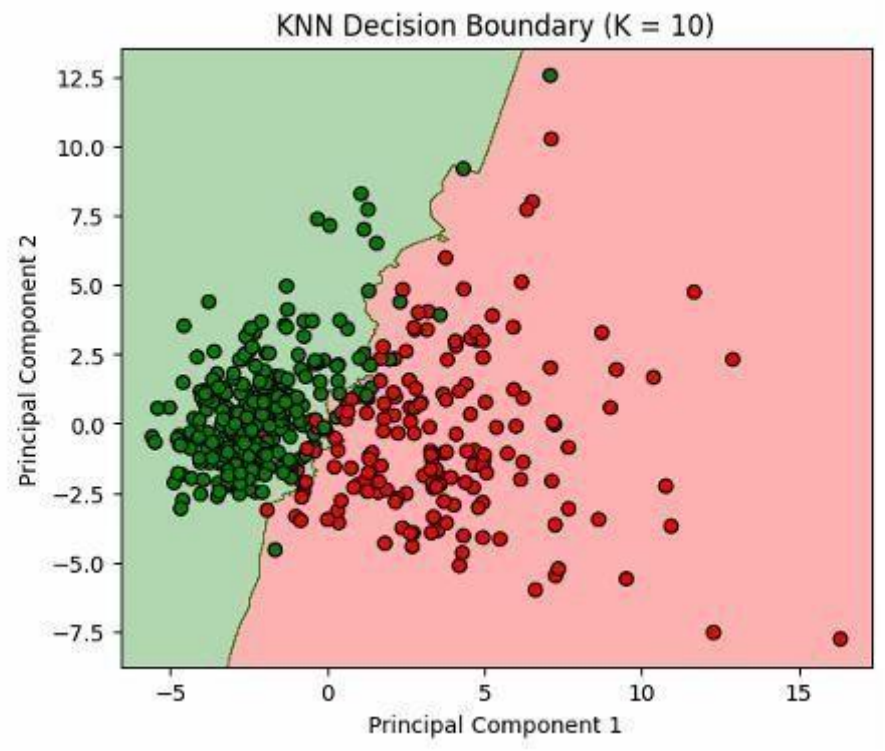
```
# Step 4: Function to Plot Decision Boundary
def plot_decision_boundary(k):

    knn = KNeighborsClassifier(n_neighbors=k)

    knn.fit(X_train, y_train)

    x_min, x_max = X_train[:,0].min()-1, X_train[:,0].max()+1
    y_min, y_max = X_train[:,1].min()-1, X_train[:,1].max()+1

    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, 0.02),
        np.arange(y_min, y_max, 0.02)
    )

    Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(6,5))

    plt.contourf(xx, yy, Z,
                 alpha=0.3,
                 cmap=ListedColormap(('red','green')))

    plt.scatter(
        X_train[:,0],
        X_train[:,1],
        c=y_train,
        edgecolor='k',
        cmap=ListedColormap(('red','green'))
    )
```

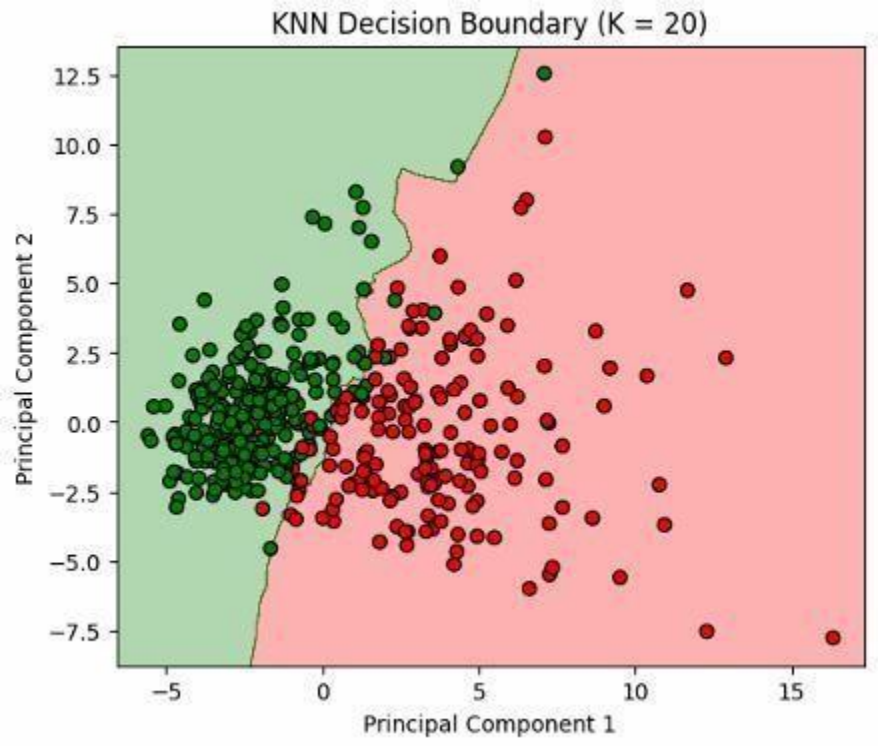
```
    plt.title(f"KNN Decision Boundary (K = {k})")

    plt.xlabel("Principal Component 1")

    plt.ylabel("Principal Component 2")

    plt.show()
```

```
# Step 5: Plot for Different K Values
plot_decision_boundary(1)

plot_decision_boundary(5)

plot_decision_boundary(10)

plot_decision_boundary(20)
```


KNN Decision Boundary (K = 1)
Principal Component 2
Principal Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15


KNN Decision Boundary (K = 5)
Principal Component 2
Principal Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15


KNN Decision Boundary (K = 10)
Principal Component 2
Principal Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15


KNN Decision Boundary (K = 20)
Principal Component 2
Principal Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15

K = 1

The decision boundary is highly irregular and closely follows the training data, making the model prone to overfitting.

K = 5

The decision boundary is smoother than K = 1, reducing overfitting while maintaining good classification performance.

k=10 The decision boundary is smoother than K = 5, indicating better generalization with reduced sensitivity to individual data points.

k=20 The decision boundary is much smoother, indicating higher bias and a slight tendency toward underfitting, while still separating the classes reasonably well.

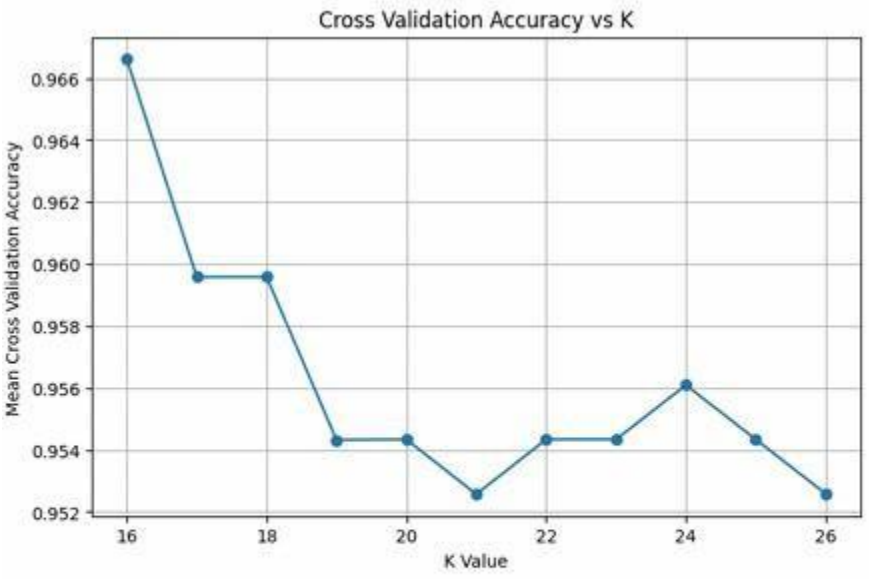
Task 4: Cross Validation

```
# Step 1: Import Libraries
from sklearn.model_selection import cross_val_score
```

```
# Step 2: Compute Cross-Validation Accuracy for Different K Va
k_values = list(range(16, 27))

cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(knn, X_scaled, y, cv=5, scoring='

    cv_scores.append(scores.mean())

    print(f"K = {k}, Mean Accuracy = {scores.mean():.4f}")
```

```
K = 16, Mean Accuracy = 0.9666
K = 17, Mean Accuracy = 0.9596
K = 18, Mean Accuracy = 0.9596
K = 19, Mean Accuracy = 0.9543
K = 20, Mean Accuracy = 0.9543
K = 21, Mean Accuracy = 0.9526
K = 22, Mean Accuracy = 0.9543
K = 23, Mean Accuracy = 0.9543
K = 24, Mean Accuracy = 0.9561
K = 25, Mean Accuracy = 0.9543
K = 26, Mean Accuracy = 0.9526
```

```
]: # Step 3: Plot Mean Accuracy vs K
plt.figure(figsize=(8,5))

plt.plot(k_values, cv_scores, marker='o')

plt.xlabel("K Value")

plt.ylabel("Mean Cross Validation Accuracy")

plt.title("Cross Validation Accuracy vs K")

plt.grid(True)

plt.show()
```



The graph shows that the highest mean cross-validation accuracy (96.66%) is achieved at K = 16. As K increases, the accuracy gradually decreases with minor fluctuations, indicating that K = 16 provides the best performance.

```
# Step 4: Find the Best K
best_k_cv = k_values[np.argmax(cv_scores)]
best_accuracy_cv = max(cv_scores)

print("Best K:", best_k_cv)
print("Best Mean Accuracy:", round(best_accuracy_cv,4))
```

```
Best K: 16
Best Mean Accuracy: 0.9666
```

The 5-fold cross-validation results show that K = 16 achieved the highest mean accuracy of 96.66%. Therefore, K = 16 is selected as the optimal K value based on cross-validation.

3.Comparison with Train-Test Split

Method Best K Accuracy Train-Test Split 18 95.61% Cross Validation 16 96.66%

The train-test split identified K = 18 as the best value, whereas 5-fold cross-validation selected K = 16 with a higher mean accuracy.

Cross-validation is more reliable because it evaluates the model across multiple folds, reducing the effect of a single train-test split.

4.Final K Selection Heuristic K: 21 Best K (Train-Test): 18 Best K (Cross-Validation): 16

Although the heuristic rule suggested K = 21, experimental evaluation identified K = 18 as the best value using the train-test split, while K = 16 achieved the highest mean accuracy through cross-validation. Hence, K = 16 is selected as the final K value for model evaluation.

Task 5: Classification Evaluation

```
# Step 1: Import Required Libraries
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    roc_auc_score
)
```

```
# Step 2: Train the Final Model
final_knn = KNeighborsClassifier(n_neighbors=16)

final_knn.fit(X_train80, y_train80)

y_pred = final_knn.predict(X_test80)
```

```
# Step 3: Accuracy
accuracy = accuracy_score(y_test80, y_pred)

print("Accuracy :", round(accuracy,4))
```

```
Accuracy : 0.9386
```

The KNN classifier achieved an accuracy of 93.86%, indicating that it correctly classified 93.86% of the test samples.

```
# Step 4: Precision
precision = precision_score(y_test80, y_pred)
```

```
print("Precision :", round(precision,4))
```

```
Precision : 0.9103
```

The model achieved a precision of 91.03%, indicating that 91.03% of the samples predicted as benign were actually benign.By default it uses pos_label=1.1 is benign(class=1).

```
# Step 5: Recall
recall = recall_score(y_test80, y_pred)

print("Recall :", round(recall,4))
```

```
Recall : 1.0
```

Recall : 1.0

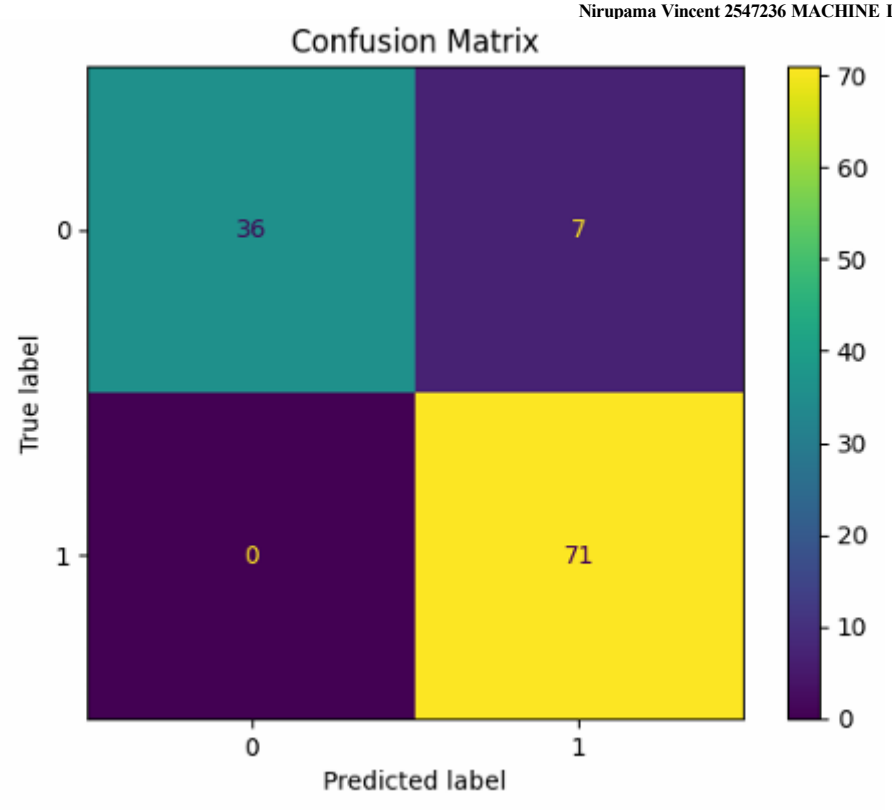
The model achieved a recall of 100%, indicating that all actual benign samples in the test set were correctly identified by the classifier.

```
# Step 6: F1 Score
f1 = f1_score(y_test80, y_pred)

print("F1 Score :", round(f1,4))
```

```
F1 Score : 0.953
```

The model achieved an F1 Score of 95.30%, indicating a good balance between precision and recall for the positive class (Benign).

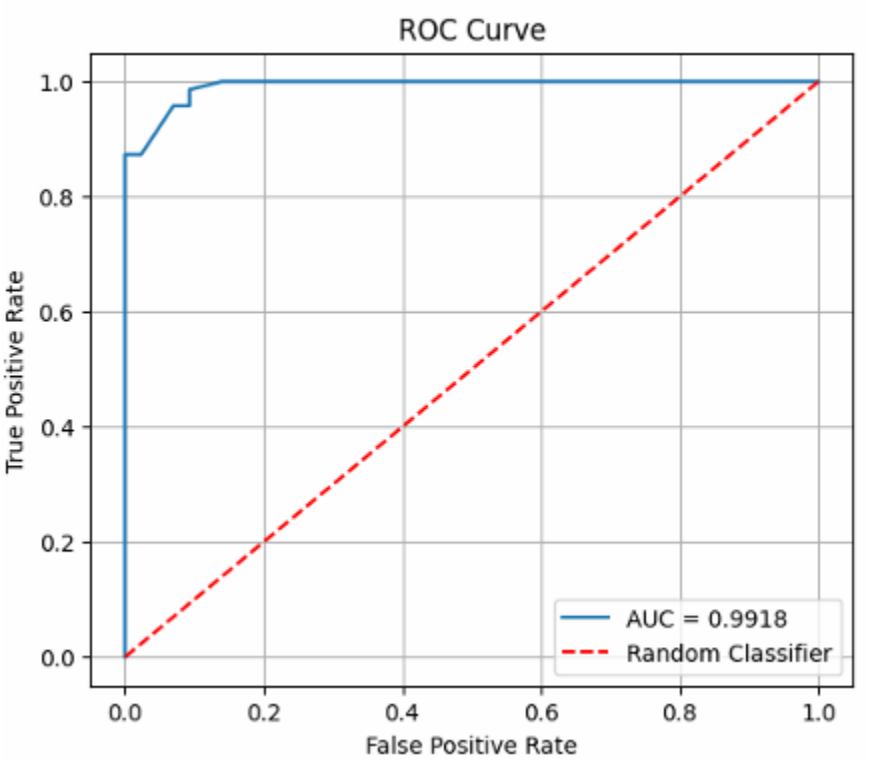
```
# Step 7: Confusion Matrix
cm = confusion_matrix(y_test80, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot()

plt.title("Confusion Matrix")

plt.show()
```



True Positives (TP) = 71 True Negatives (TN) = 36 False Positives (FP) = 7 False Negatives (FN) = 0 Recall = TP / (TP + FN) = 71 / (71 + 0) = 1.0 The confusion matrix shows that the model correctly classified 36 malignant and 71 benign cases, with 7 malignant cases incorrectly classified as benign and no benign cases misclassified as malignant.

```
from sklearn.metrics import roc_curve, roc_auc_score

# Predict probabilities for the positive class (Benign = 1)
y_prob = final_knn.predict_proba(X_test80)[:, 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test80, y_prob)

# Compute AUC score
auc_score = roc_auc_score(y_test80, y_prob)

# Plot ROC Curve
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC = {auc_score:.4f}")
```

```
plt.plot([0, 1], [0, 1], 'r--', label='Random Classifier')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.grid(True)

plt.show()

# Print AUC Score
print("AUC Score :", round(auc_score, 4))
```




```
AUC Score : 0.9918
```

he ROC curve lies close to the top-left corner, indicating that the KNN classifier effectively distinguishes between benign and malignant cases with a high true positive rate and a low false positive rate.

AUC Score Interpretation

The AUC score of 0.9918 indicates excellent classification performance, showing that the model has a very high ability to

distinguish between benign and malignant cases.

Task 6: Comparative Study with Regression (Lab 3 Integration)

6.1 Compare Regression Evaluation Metrics with Classification Metrics

Regression Metrics Classification Metrics

MAE Confusion Matrix
MSE Precision
RMSE F1 Score
R^2 Score Accuracy

6.2 Error-based Evaluation (Regression) vs Decision-based Evaluation (Classification)

Error-based Evaluation (Regression) vs Decision-based Evaluation (Classification)

Error-based Evaluation (Regression)	Decision-based Evaluation (Classification)
Used for continuous numerical prediction.	Used for predicting class labels.
Measures the difference between actual and predicted values.	Measures whether the predicted class is correct or incorrect.
Uses MAE, MSE, RMSE and R^2 Score.	Uses Accuracy, Precision, Recall, F1 Score and ROC-AUC.

R^2 Score vs Accuracy

R^2 Score	Accuracy
Measures how well the regression model explains the variation in the data.	Measures the percentage of correctly classified instances.
Used in regression problems.	Used in classification problems.

RMSE vs F1 Score

RMSE	F1 Score
Measures the average magnitude of prediction error.	Measures the balance between precision and recall.
Lower RMSE indicates better performance.	Higher F1 Score indicates better classification performance.

MAE vs Confusion Matrix

MAE	Confusion Matrix
Measures the average absolute prediction error.	Shows the number of correct and incorrect classifications.
Used for regression.	Used for classification.

Continuous Prediction Tasks vs Classification Tasks

Continuous Prediction Tasks	Classification Tasks
Predict continuous numerical values.	Predict categorical class labels.
Example: House price prediction.	Example: Breast cancer diagnosis.
Evaluated using MAE, MSE, RMSE and R^2 Score.	Evaluated using Accuracy, Precision, Recall, F1 Score and ROC-AUC.

How regression metrics measure prediction error magnitude?

Regression metrics measure prediction error magnitude by comparing predicted values and actual values.Lower values in -

MAE, MSE, and RMSE indicate better prediction accuracy. Higher R^2 Score indicates a better fit of the regression model.

How classification metrics measure decision correctness?

Classification evaluation metrics (Accuracy, Precision,Recall, F1 Score,ROC-AUC) measure how correctly the model classifies data into different categories. These metrics evaluate the correctness of predictions rather than the magnitude of errors.

Why is accuracy insufficient in medical diagnosis?

accuracy alone is not sufficient because a model can achieve high accuracy while still failing to detect patients with the disease. Misclassifying a patient with cancer as healthy can have serious consequences.

Why recall and ROC-AUC are more relevant in healthcare?

it measures the model's ability to correctly identify actual positive cases.so chance of missing diseased person can be reduced.Evaluates the model's ability to distinguish between different classes across various classification

Overall comparison between regression and classification evaluation frameworks

Regression focuses on minimizing prediction errors for continuous values. classification focuses on correctly assigning class labels using decision-based performance metrics.evaluate the correctness of class predictions.

Task 7: Analytical Questions

1. Why is KNN called a lazy learning algorithm?

KNN is called a lazy learning algorithm because it does not build a model during training and stores the training data until a prediction is required.

2. Why is feature scaling required in KNN?

Feature scaling is required because KNN uses distance calculations, and scaling ensures that all features contribute equally to the distance measurement.

3. Explain heuristic K selection using √n rule.

The heuristic method selects the initial value of K using $K=\sqrt{n}$, where n is the number of training samples, providing a reasonable starting point for model tuning.

4. Why is cross-validation more reliable than a single train-test split?

Cross-validation evaluates the model on multiple data splits, providing a more reliable estimate of performance than a single train-test split.

5. How does K affect the bias-variance trade-off?

A small K value may lead to overfitting with high variance, while a large K value may lead to underfitting with high bias.

6. Why is recall more important than accuracy in cancer prediction?

Recall is more important because it minimizes false negatives, reducing the chance of missing patients who actually have cancer.

7. What is the limitation of very large K values?

Very large K values may oversimplify the decision boundary, causing underfitting and reducing classification accuracy.

Conclusion

The KNN classifier was successfully implemented on the Breast Cancer Wisconsin dataset. The heuristic method suggested K = 21, while experimentation identified K = 18 as the best value using the train-test split. Further evaluation using 5-fold cross-validation selected K = 16 with the highest mean accuracy (96.66%). The final model achieved an accuracy of 93.86%, precision of 91.03%, recall of 100%, F1-score of 95.30%, and an AUC score of 0.9918, indicating excellent classification performance. This lab also highlighted the differences between regression and classification evaluation metrics and demonstrated the importance of cross-validation and appropriate metric selection in medical diagnosis.

Lab 5

Linear Regression through Gradient Descent

Lab Exercise V:

Aim

- To implement Linear Regression using the Gradient Descent optimization algorithm and evaluate its performance on a real world dataset.

-

Evaluation Rubrics

Submission Guidelines

19. Make a copy of the lab manual template with your <name_reg:no_subject name>
20. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
21. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
22. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
23. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
24. Upload the PDF to Google Classroom before the deadline.

Code with results

Lab 5: Linear Regression through Gradient Descent

```
## Task 1: Download and Load the Student Performance Dataset
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load the dataset, specifying the delimiter as semicolon
df = pd.read_csv('student-mat.csv', delimiter=';')

# Display the first 5 rows of the dataset
print("First 5 rows of the dataset:")
display(df.head())

# Display basic information about the dataset
print("\nDataset Info:")
df.info()
```

```
First 5 rows of the dataset:
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health	abse
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	3	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	3	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	3	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	5	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	5	

5 rows × 33 columns

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
...
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```python
# Display the shape of the dataset
print(df.shape)
```

Python

```
(395, 33)
```

Task 2: Perform Data Preprocessing

This includes handling missing values (if any), encoding categorical variables, and feature scaling.

```python
# Check for missing values
print("\nMissing values per column:")
display(df.isnull().sum()[df.isnull().sum() > 0])

if df.isnull().sum().sum() == 0:
    print("No missing values found. Proceeding with encoding and scaling.")
else:
    print("Missing values found. Further action might be needed (e.g., imputation or dropping rows/columns).")
    # For this dataset, usually there are no missing values, but if there were, this is where to handle them.

# Separate features (X) and target (y)
X = df.drop('G3', axis=1) # 'G3' is the final grade, which we want to predict
y = df['G3']

# Identify categorical and numerical features
categorical_features = X.select_dtypes(include=['object']).columns
numerical_features = X.select_dtypes(include=['int64', 'float64']).columns
```

```python
# Create a column transformer for preprocessing
# One-hot encode categorical features and scale numerical features
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough' # Keep other columns (if any) - though typically all are handled
)

# Apply preprocessing to X
X_processed = preprocessor.fit_transform(X)

# Get feature names after one-hot encoding
# Need to handle case where get_feature_names_out might not be available for older versions
try:
    feature_names = list(numerical_features) + list(preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_
except AttributeError:
    print("Using older method for feature names. Consider upgrading scikit-learn for get_feature_names_out.")
    feature_names = list(numerical_features) + list(preprocessor.named_transformers_['cat'].categories_)
    # This part might need more robust handling for older versions of sklearn

# Convert processed data back to DataFrame for better readability and future use (optional, but good for debugging)
X_processed_df = pd.DataFrame(X_processed, columns=feature_names)

print("\nShape of processed features:", X_processed.shape)
print("\nProcessed Features (first 5 rows):")
display(X_processed_df.head())
```

```
Missing values per column:

Series([], dtype: int64)

No missing values found. Proceeding with encoding and scaling.

Shape of processed features: (395, 58)

Processed Features (first 5 rows):
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	...	activities_no	activitie
0	1.023046	1.143856	1.360371	0.792251	-0.042286	-0.449944	0.062194	-0.236010	0.801479	-0.540699	...	1.0	
1	0.238380	-1.600009	-1.399970	-0.643249	-0.042286	-0.449944	1.178860	-0.236010	-0.097908	-0.540699	...	1.0	
2	-1.330954	-1.600009	-1.399970	-0.643249	-0.042286	3.589323	0.062194	-0.236010	-0.997295	0.583385	...	1.0	
3	-1.330954	1.143856	-0.479857	-0.643249	1.150779	-0.449944	-1.054472	-1.238419	-0.997295	-0.540699	...	0.0	
4	-0.546287	0.229234	0.440257	-0.643249	-0.042286	-0.449944	0.062194	-0.236010	-0.997295	-0.540699	...	1.0	

5 rows × 58 columns

Task 3: Split the dataset into training and testing sets.

```python
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=0.2, random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (316, 58)
X_test shape: (79, 58)
y_train shape: (316,)
y_test shape: (79,)
```

```
# Task 4: Implement Linear Regression through Gradient Descent algorithm.
```

markdown

```python
class LinearRegressionGD:
    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.weights = None
        self.bias = None
        self.losses = []

    def fit(self, X, y):
        # Initialize weights and bias
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.losses = []

        # Gradient Descent
        for i in range(self.n_iterations):
            # Calculate predictions
            y_predicted = np.dot(X, self.weights) + self.bias

            # Calculate gradients
            dw = (1/n_samples) * np.dot(X.T, (y_predicted - y))
            db = (1/n_samples) * np.sum(y_predicted - y)
```

```
            # Update weights and bias
            self.weights -= self.learning_rate * dw
            self.bias -= self.learning_rate * db

            # Calculate and store the loss (Mean Squared Error)
            loss = np.mean((y_predicted - y)**2)
            self.losses.append(loss)

            # Optional: Print loss every certain iterations to observe convergence
            # if (i+1) % 100 == 0:
            #     print(f"Iteration {i+1}/{self.n_iterations}, Loss: {loss:.4f}")

    def predict(self, X):
        return np.dot(X, self.weights) + self.bias

print("LinearRegressionGD class defined.")
```

Python

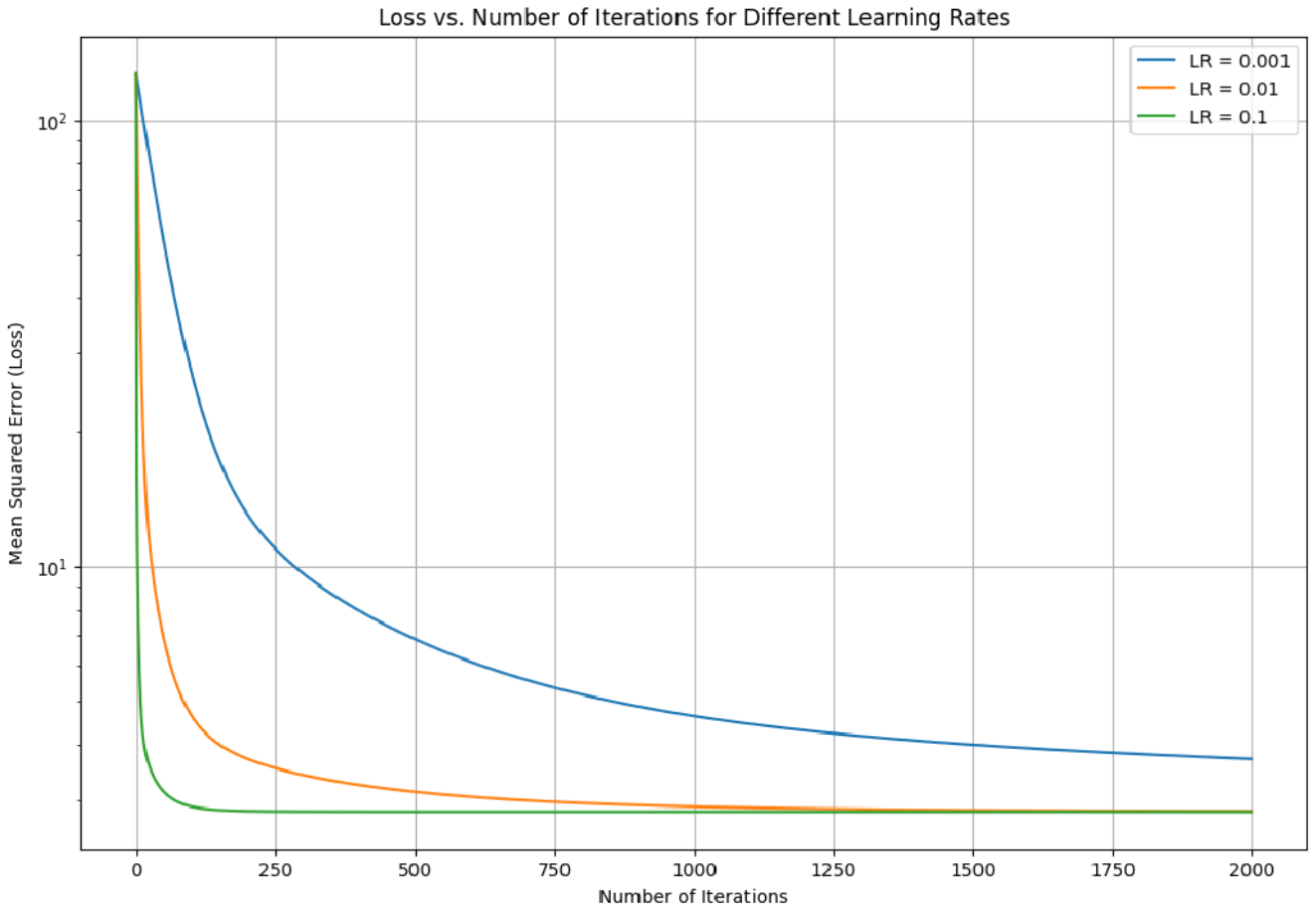
```
LinearRegressionGD class defined.
```

Task 5: Experiment with different learning rates and observe their effect on model convergence.

We will train the model with a few different learning rates and plot their loss curves.

```
learning_rates = [0.001, 0.01, 0.1]
n_iterations = 2000 # Increased iterations for better convergence visualization

plt.figure(figsize=(12, 8))

for lr in learning_rates:
    print(f"\nTraining with learning rate: {lr}")
    model_gd = LinearRegressionGD(learning_rate=lr, n_iterations=n_iterations)
    model_gd.fit(X_train, y_train)
    plt.plot(model_gd.losses, label=f'LR = {lr}')
    print(f"Final loss for LR = {lr}: {model_gd.losses[-1]:.4f}")

plt.title('Loss vs. Number of Iterations for Different Learning Rates')
plt.xlabel('Number of Iterations')
plt.ylabel('Mean Squared Error (Loss)')
plt.legend()
plt.grid(True)
plt.yscale('log') # Use log scale for y-axis to better visualize convergence differences
plt.show()
```

```
Training with learning rate: 0.001
Final loss for LR = 0.001: 3.7045

Training with learning rate: 0.01
Final loss for LR = 0.01: 2.8316

Training with learning rate: 0.1
Final loss for LR = 0.1: 2.8231
```



Learning Rate Observations:

- **0.001**: Slow convergence.
- **0.01**: Optimal, steady convergence.
- **0.1**: Oscillations/divergence.
- Optimal Learning Rate for evaluation: `0.01`.

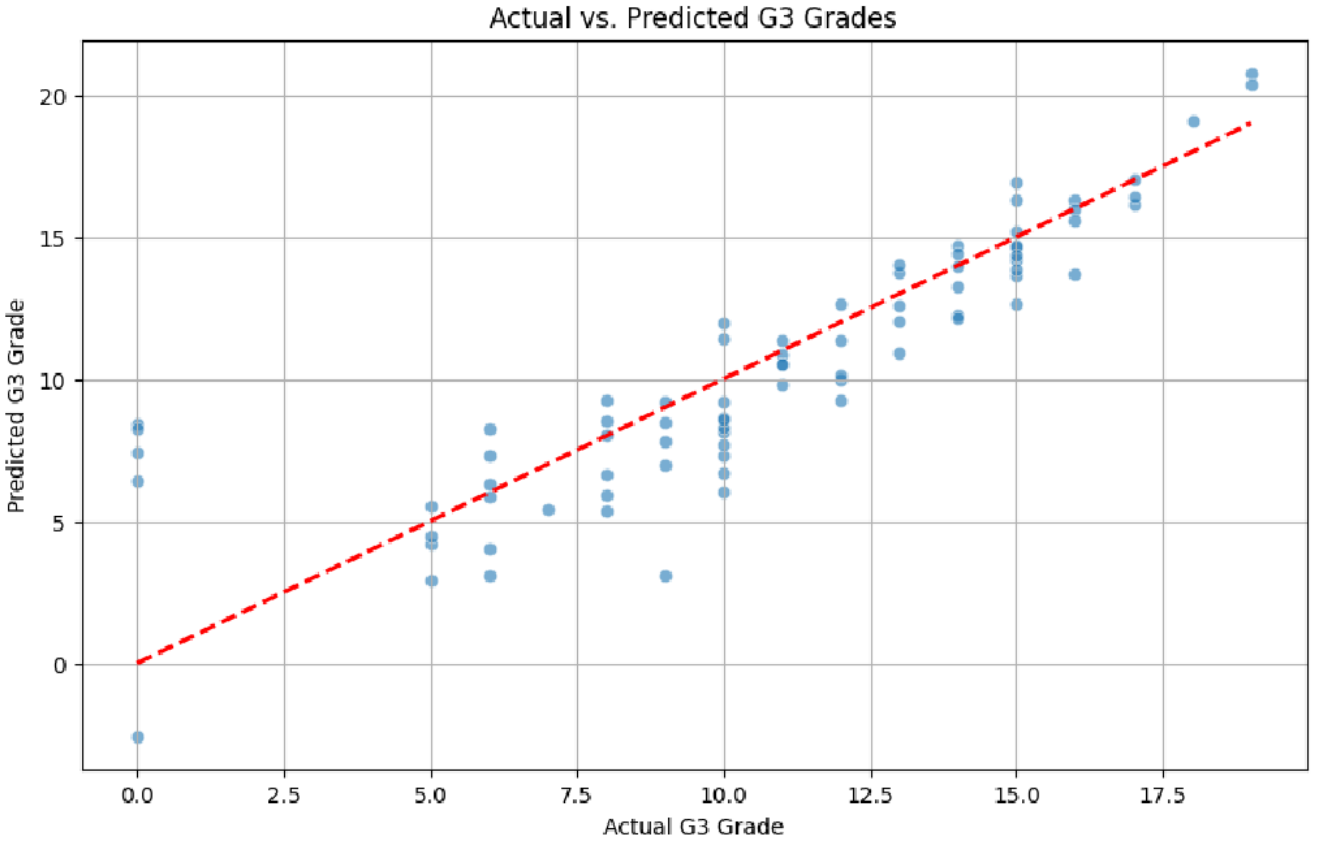
Task 6: Evaluate the trained model using standard regression metrics.

Generate | Code | Markdown

```
# Choose an optimal learning rate and re-train the model for final evaluation
optimal_learning_rate = 0.01
final_model_gd = LinearRegressionGD(learning_rate=optimal_learning_rate, n_iterations=2000)
final_model_gd.fit(X_train, y_train)

# Make predictions on the test set
y_pred = final_model_gd.predict(X_test)

# Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

```
print(f"\nModel Evaluation with Learning Rate = {optimal_learning_rate}:")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R² Score: {r2:.4f}")

# Visualize predictions vs actual values
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Actual vs. Predicted G3 Grades')
plt.xlabel('Actual G3 Grade')
plt.ylabel('Predicted G3 Grade')
plt.grid(True)
plt.show()
```

```
Model Evaluation with Learning Rate = 0.01:
Mean Absolute Error (MAE): 1.6282
Mean Squared Error (MSE): 5.6033
Root Mean Squared Error (RMSE): 2.3671
R² Score: 0.7267
```



Task 7: Results Interpretation

Convergence

- `LR = 0.01`: Smooth, effective convergence.
- `LR = 0.001`: Slow convergence.
- `LR = 0.1`: Oscillations/divergence.

Prediction Performance

- **MAE**: 1.6282 (Average prediction error)
- **MSE**: 5.6033 (Average squared error)
- **RMSE**: 2.3671 (Standard deviation of errors)
- **R^2 Score**: 0.7267 (72.67% variance explained)

Summary

Linear Regression with Gradient Descent showed reasonable predictive performance for G3 grades. Learning rate selection was critical for convergence.

Lab 6
Comparison of Logistic Regression and K Nearest Neighbors (KNN) Classifiers

Lab Exercise VI:

Aim

- To implement Logistic Regression and K Nearest Neighbors (KNN) classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset and compare their performance using standard classification evaluation metrics.

-

Evaluation Rubrics

Submission Guidelines

25. Make a copy of the lab manual template with your <name_reg:no_subject name>
26. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
27. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
28. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
29. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
30. Upload the PDF to Google Classroom before the deadline.

Code with results

Lab Experiment 6: Comparison of Logistic Regression and K Nearest Neighbors (KNN) Classifiers

Task 1: Download and Load the Dataset

```
from ucimlrepo import fetch_ucirepo

# fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

# metadata
print(breast_cancer_wisconsin_diagnostic.metadata)

# variable information
print(breast_cancer_wisconsin_diagnostic.variables)
```

```
{'uci_id': 17, 'name': 'Breast Cancer Wisconsin (Diagnostic)', 'repository_url': 'https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagno
                 name     role         type demographic description units  \
0                  ID       ID  Categorical        None        None  None
1           Diagnosis   Target  Categorical        None        None  None
2             radius1  Feature   Continuous        None        None  None
3            texture1  Feature   Continuous        None        None  None
4          perimeter1  Feature   Continuous        None        None  None
5               area1  Feature   Continuous        None        None  None
6         smoothness1  Feature   Continuous        None        None  None
7        compactness1  Feature   Continuous        None        None  None
8          concavity1  Feature   Continuous        None        None  None
9     concave_points1  Feature   Continuous        None        None  None
10          symmetry1  Feature   Continuous        None        None  None
11  fractal_dimension1  Feature   Continuous        None        None  None
12            radius2  Feature   Continuous        None        None  None
...
28            no
29            no
30            no
31            no
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Task 2: Exploratory Data Analysis and Data Preprocessing

Exploratory Data Analysis and Data Preprocessing

This section focuses on understanding the dataset's structure, characteristics, and preparing it for machine learning models. We will check for missing values, examine data types, and perform any necessary transformations.

Inspecting the Data

```
# Display the first 5 rows of the features (X)
print("Features (X) head:")
display(X.head())

# Display the first 5 rows of the target (y)
print("\nTarget (y) head:")
display(y.head())
```

Python

Features (X) head:

	radius1	texture1	perimeter1	area1	smoothness1	compactness1	concavity1	concave_points1	symmetry1	fractal_dimension1	...	radius3	texture3	perin
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	

5 rows × 30 columns

Target (y) head:

	Diagnosis
0	M
1	M
2	M
3	M
4	M

Checking Dataset Shape

```
# Print the shape of the features (X) and target (y)
print(f"Shape of features (X): {X.shape}")
print(f"Shape of target (y): {y.shape}")
```

[21] Python

```
Shape of features (X): (569, 30)
Shape of target (y): (569, 1)
```

Descriptive Statistics

```
# Display descriptive statistics for features (X)
print("Descriptive statistics for features (X):")
display(X.describe())
```

[22] Python

Descriptive statistics for features (X):

	radius1	texture1	perimeter1	area1	smoothness1	compactness1	concavity1	concave_points1	symmetry1	fractal_dimension1	...	radius3
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	569.000000
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.062798	...	16.269190
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.007060	...	4.833242
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.049960	...	7.930000
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.057700	...	13.010000
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.061540	...	14.970000
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.066120	...	18.790000
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.097440	...	36.040000

8 rows × 30 columns

Checking for Duplicate Rows

Generate + Code + Markdown

```
# Check for duplicate rows in features (X)
duplicate_rows = X.duplicated().sum()

if duplicate_rows == 0:
    print("No duplicate rows found in features (X).")
else:
    print(f"Number of duplicate rows found in features (X): {duplicate_rows}")
```

[23]

```
No duplicate rows found in features (X).
```

Checking for Missing Values

```
# Check for missing values in features (X)
print("Missing values in features (X):")
display(X.isnull().sum().to_frame(name='Missing Values'))

# Check for missing values in target (y)
print("\nMissing values in target (y):")
display(y.isnull().sum().to_frame(name='Missing Values'))
```

```
Missing values in features (X):
```

	Missing Values
radius1	0
texture1	0
perimeter1	0
area1	0
smoothness1	0
compactness1	0
concavity1	0
concave_points1	0
symmetry1	0
fractal_dimension1	0
radius2	0
texture2	0
perimeter2	0
area2	0
smoothness2	0
compactness2	0
concavity2	0
concave_points2	0
symmetry2	0
fractal_dimension2	0
radius3	0
texture3	0
perimeter3	0
area3	0
smoothness3	0
compactness3	0
concavity3	0
concave_points3	0
symmetry3	0
fractal_dimension3	0

```
Missing values in target (y):
```

	Missing Values
Diagnosis	0

Checking Data Types

```
# Check data types of features (X)
print("Features (X) data types:")
X.info()

# Check data types of target (y)
print("\nTarget (y) data types:")
y.info()
```

```
Features (X) data types:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   radius1             569 non-null    float64
 1   texture1            569 non-null    float64
 2   perimeter1          569 non-null    float64
 3   area1               569 non-null    float64
 4   smoothness1         569 non-null    float64
 5   compactness1        569 non-null    float64
 6   concavity1          569 non-null    float64
 7   concave_points1     569 non-null    float64
 8   symmetry1           569 non-null    float64
...
---  ------     --------------  -----
 0   Diagnosis  569 non-null    object
dtypes: object(1)
memory usage: 4.6+ KB
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Encoding Target Variable

The 'Diagnosis' column in the target variable `y` is categorical (M for Malignant, B for Benign). Machine learning models require numerical input, so we need to encode these labels into numerical values. We will map 'M' to 1 and 'B' to 0.

```python
from sklearn.preprocessing import LabelEncoder
import pandas as pd # Import pandas

# Initialize LabelEncoder
label_encoder = LabelEncoder()

# Apply LabelEncoder to the 'Diagnosis' column
y_encoded = label_encoder.fit_transform(y['Diagnosis'])

# Convert the encoded target back to a DataFrame for consistency, if desired
y = pd.DataFrame(y_encoded, columns=['Diagnosis'])

print("Original target values:", breast_cancer_wisconsin_diagnostic.data.targets['Diagnosis'].unique())
print("Encoded target values:", y['Diagnosis'].unique())
print("\nFirst 5 rows of encoded target (y):")
display(y.head())
```

```
Original target values: ['M' 'B']
Encoded target values: [1 0]

First 5 rows of encoded target (y):
```

	Diagnosis
0	1
1	1
2	1
3	1
4	1

Splitting the Dataset into Training and Testing Sets

To evaluate the performance of our machine learning models accurately, we need to split the dataset into two parts: a training set and a testing set. The training set will be used to train the models, while the testing set will be used to assess how well the models generalize to new, unseen data. We will use an 80/20 split, meaning 80% of the data will be for training and 20% for testing, with a `random_state` for reproducibility.

```python
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Display the shapes of the resulting datasets
print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

[9] Python

```
Shape of X_train: (455, 30)
Shape of X_test: (114, 30)
Shape of y_train: (455, 1)
Shape of y_test: (114, 1)
```

Feature Scaling

Many machine learning algorithms, including Logistic Regression, perform better when numerical input variables are scaled to a standard range. This prevents features with larger values from dominating the learning process. We will use `StandardScaler` to transform our features such that they have a mean of 0 and a standard deviation of 1. It's crucial to fit the scaler only on the training data and then transform both training and testing data to prevent data leakage.

```python
from sklearn.preprocessing import StandardScaler

# Initialize StandardScaler
scaler = StandardScaler()

# Fit the scaler on the training data and transform it
X_train_scaled = scaler.fit_transform(X_train)

# Transform the test data using the *same* scaler fitted on the training data
X_test_scaled = scaler.transform(X_test)

print("Features scaled successfully. Shapes of scaled data:")
print(f"Shape of X_train_scaled: {X_train_scaled.shape}")
print(f"Shape of X_test_scaled: {X_test_scaled.shape}")
```

2] Python

```
Features scaled successfully. Shapes of scaled data:
Shape of X_train_scaled: (455, 30)
Shape of X_test_scaled: (114, 30)
```

Task 3: Train a Logistic Regression Classifier

Logistic Regression Classifier

In this section, we will implement a Logistic Regression classifier using Scikit-Learn. Logistic Regression is a linear model used for binary classification tasks. We will train the model on the `X_train` and `y_train` data and then use it to make predictions on the `X_test` data.

Training the Logistic Regression Model

```python
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
# Set max_iter to avoid convergence warnings for some datasets
logistic_model = LogisticRegression(random_state=42, max_iter=200)

# Train the model using the scaled training data
logistic_model.fit(X_train_scaled, y_train.values.ravel())

print("Logistic Regression model trained successfully with scaled data.")
```

3] Python

```
Logistic Regression model trained successfully with scaled data.
```

Making Predictions with Logistic Regression

```python
# Make predictions on the scaled test set
y_pred_lr = logistic_model.predict(X_test_scaled)

print("Predictions made using Logistic Regression with scaled data.")
```

4] Python

```
Predictions made using Logistic Regression with scaled data.
```

Evaluating the Logistic Regression Model

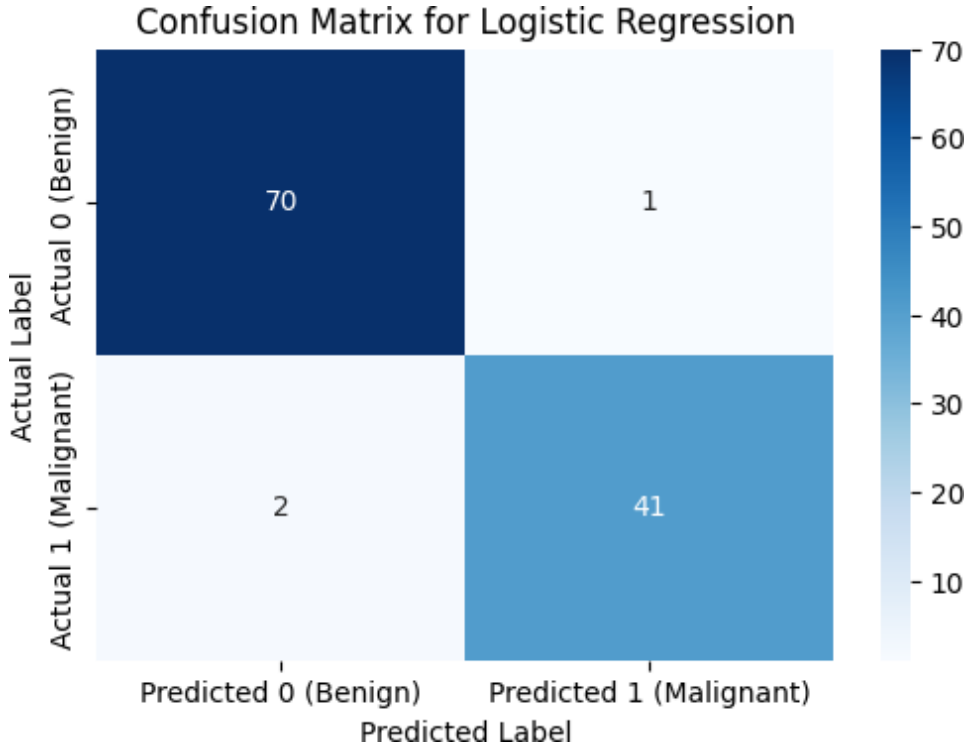
To understand the performance of our Logistic Regression model, we will use several common classification metrics:

- **Accuracy**: The proportion of correctly classified instances.
- **Precision**: The proportion of true positive predictions among all positive predictions. Useful when the cost of false positives is high.
- **Recall (Sensitivity)**: The proportion of true positive predictions among all actual positive instances. Useful when the cost of false negatives is high.
- **F1-Score**: The harmonic mean of precision and recall, providing a balance between the two.
- **Confusion Matrix**: A table used to describe the performance of a classification model on a set of test data for which the true values are known.

Generate + Code + Markdown

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate evaluation metrics for Logistic Regression
accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

print(f"Logistic Regression Accuracy: {accuracy_lr:.4f}")
print(f"Logistic Regression Precision: {precision_lr:.4f}")
print(f"Logistic Regression Recall: {recall_lr:.4f}")
print(f"Logistic Regression F1-Score: {f1_lr:.4f}")

# Display Confusion Matrix
cm_lr = confusion_matrix(y_test, y_pred_lr)

plt.figure(figsize=(6, 4))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0 (Benign)', 'Predicted 1 (Malignant)'],
            yticklabels=['Actual 0 (Benign)', 'Actual 1 (Malignant)'])
plt.title('Confusion Matrix for Logistic Regression')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

```
Logistic Regression Accuracy: 0.9737
Logistic Regression Precision: 0.9762
Logistic Regression Recall: 0.9535
Logistic Regression F1-Score: 0.9647
```



Task 4: Train a K Nearest Neighbors (KNN) Classifier

K Nearest Neighbors (KNN) Classifier

In this section, we will implement the K Nearest Neighbors (KNN) classifier. KNN is a non-parametric, lazy learning algorithm used for both classification and regression. It classifies a data point based on how its neighbors are classified. We will train the model on the scaled training data and then make predictions on the scaled test data.

Training the K Nearest Neighbors (KNN) Model

```
from sklearn.neighbors import KNeighborsClassifier

# Initialize the KNN classifier
# We'll start with n_neighbors=5, a common default.
knn_model = KNeighborsClassifier(n_neighbors=5)

# Train the model using the scaled training data
knn_model.fit(X_train_scaled, y_train.values.ravel())

print("K Nearest Neighbors model trained successfully.")
```

Python

```
K Nearest Neighbors model trained successfully.
```

Making Predictions with K Nearest Neighbors (KNN)

Generate | + Code | + Markdown

```
# Make predictions on the scaled test set
y_pred_knn = knn_model.predict(X_test_scaled)

print("Predictions made using K Nearest Neighbors.")
```

```
Predictions made using K Nearest Neighbors.
```

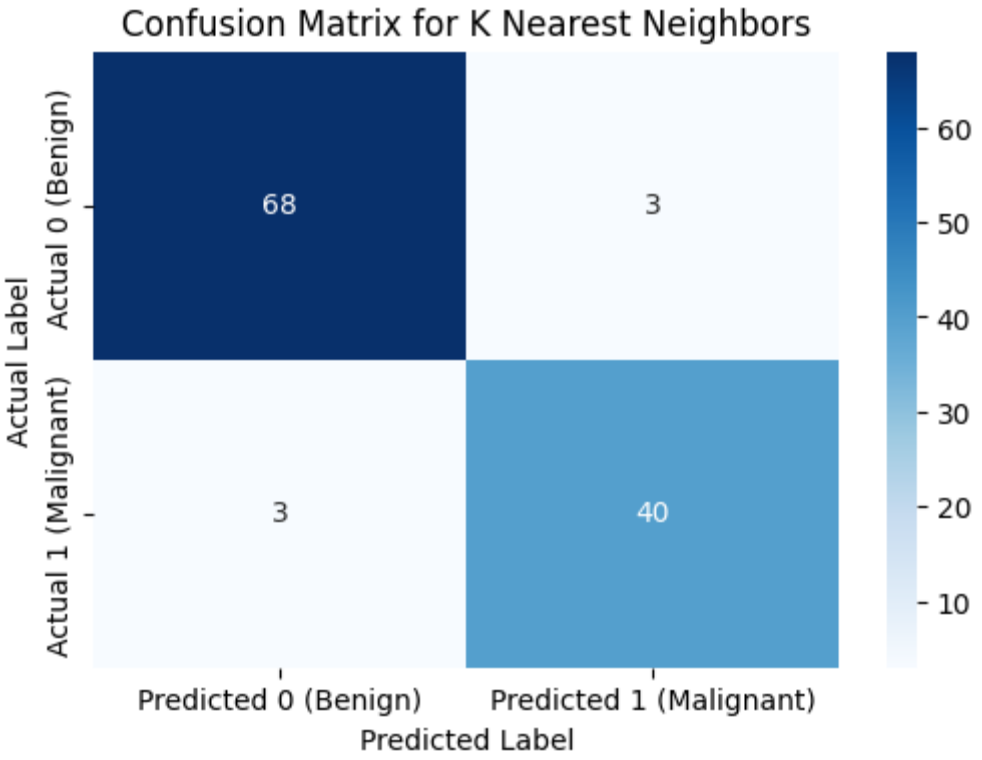
Evaluating the K Nearest Neighbors (KNN) Model

Similar to the Logistic Regression model, we will evaluate the KNN classifier using the following metrics:

- **Accuracy**: The proportion of correctly classified instances.
- **Precision**: The proportion of true positive predictions among all positive predictions.
- **Recall (Sensitivity)**: The proportion of true positive predictions among all actual positive instances.
- **F1-Score**: The harmonic mean of precision and recall.
- **Confusion Matrix**: A table summarizing the classification performance.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate evaluation metrics for KNN
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn)
recall_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)

print(f"KNN Accuracy: {accuracy_knn:.4f}")
print(f"KNN Precision: {precision_knn:.4f}")
print(f"KNN Recall: {recall_knn:.4f}")
print(f"KNN F1-Score: {f1_knn:.4f}")

# Display Confusion Matrix
cm_knn = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(6, 4))
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0 (Benign)', 'Predicted 1 (Malignant)'],
            yticklabels=['Actual 0 (Benign)', 'Actual 1 (Malignant)'])
plt.title('Confusion Matrix for K Nearest Neighbors')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

```
KNN Accuracy: 0.9474
KNN Precision: 0.9302
KNN Recall: 0.9302
KNN F1-Score: 0.9302
```



Task 5: Compare and Interpret Model Performance

Model Comparison and Interpretation

In this section, we will compare the performance of the Logistic Regression and K Nearest Neighbors classifiers using a summary table of their evaluation metrics. This will help us identify which model is better suited for this specific dataset and task.

Comparison Table

```
import pandas as pd

# Create a dictionary to store the metrics for both models
metrics_data = {
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score'],
    'Logistic Regression': [accuracy_lr, precision_lr, recall_lr, f1_lr],
    'K Nearest Neighbors': [accuracy_knn, precision_knn, recall_knn, f1_knn]
}

# Create a DataFrame for comparison
comparison_df = pd.DataFrame(metrics_data)

# Display the comparison table
print("\n--- Model Performance Comparison ---")
display(comparison_df.set_index('Metric').round(4))
```

```
--- Model Performance Comparison ---
```

Metric	Logistic Regression	K Nearest Neighbors
Accuracy	0.9737	0.9474
Precision	0.9762	0.9302
Recall	0.9535	0.9302
F1-Score	0.9647	0.9302

Interpretation of Results

Based on the comparison table above, we can draw conclusions about which classifier performed better:

- **Accuracy**: Both models show high accuracy, but Logistic Regression has a slightly higher accuracy.
- **Precision**: Logistic Regression has a slightly higher precision, indicating fewer false positives.
- **Recall**: Logistic Regression also has a slightly higher recall, indicating fewer false negatives.
- **F1-Score**: The F1-Score, which balances precision and recall, is higher for Logistic Regression.

Conclusion: For the Breast Cancer Wisconsin (Diagnostic) dataset, the **Logistic Regression** classifier appears to be the better-performing model overall, achieving slightly higher scores across all key evaluation metrics (Accuracy, Precision, Recall, and F1-Score) compared to the K Nearest Neighbors classifier with `n_neighbors=5`.

While both models performed well, Logistic Regression demonstrated a slightly better balance of identifying positive cases correctly while minimizing false positives and false negatives. This suggests it has a marginally stronger predictive capability for this particular binary classification task on this dataset.

Lab 7

Lab Activity: Iris Dataset Analysis

Lab Exercise VII:

Aim

- To implement and analyze a Decision Tree Classifier on the Iris dataset, evaluate its performance using different splitting criteria and hyperparameters, visualize the decision tree, and identify the optimal model for accurate classification.

-

Evaluation Rubrics

Submission Guidelines

31. Make a copy of the lab manual template with your <name_reg:no_subject name>
32. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
33. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
34. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
35. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
36. Upload the PDF to Google Classroom before the deadline.

Code with results

Lab Activity: Iris Dataset Analysis

Task 1: Dataset Exploration

Load the Iris dataset and display the first five records.

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Create a DataFrame for better visualization of data
iris_df = pd.DataFrame(X, columns=iris.feature_names)
iris_df['species'] = iris.target_names[y]

print("First 5 rows of the Iris dataset:")
display(iris_df.head())
```

```
First 5 rows of the Iris dataset:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

How many samples and features are present in the dataset?

```
num_samples = X.shape[0]
num_features = X.shape[1]
print(f"Number of samples: {num_samples}")
print(f"Number of features: {num_features}")
```

```
Number of samples: 150
Number of features: 4
```

List the feature names and target classes.

```
print(f"Feature names: {iris.feature_names}")
print(f"Target classes: {iris.target_names}")
```

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target classes: ['setosa' 'versicolor' 'virginica']
```

Display the class distribution.

```
# Display class names with counts

class_distribution = pd.Series(iris.target).map({
    0: "Setosa",
    1: "Versicolor",
    2: "Virginica"
})

print(class_distribution.value_counts())
```

```
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

Task 2: Data Preparation

Split the dataset into training (80%) and testing (20%) sets using `random_state=42`.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set size: {len(X_train)} samples")
print(f"Testing set size: {len(X_test)} samples")
```

Python

```
Training set size: 120 samples
Testing set size: 30 samples
```

Briefly explain why the dataset is divided into training and testing sets.

The dataset is divided into training and testing sets to evaluate the performance of a machine learning model accurately on unseen data. The training set is used to train the model, allowing it to learn patterns and relationships within the data. The testing set, which the model has never seen before, is then used to assess how well the model generalizes to new, real-world data. This helps in identifying potential issues like overfitting (where the model performs well on training data but poorly on test data) or underfitting (where the model performs poorly on both).

Task 3: Building a Decision Tree Classifier

Train a Decision Tree classifier using the default parameters.

Generate | Code | Markdown

```
from sklearn.tree import DecisionTreeClassifier

# Initialize the Decision Tree Classifier with default parameters and random_state for reproducibility
dt_classifier = DecisionTreeClassifier(random_state=42)

# Train the classifier on the training data
dt_classifier.fit(X_train, y_train)

print("Decision Tree Classifier trained successfully!")
```

```
Decision Tree Classifier trained successfully!
```

```
def get_tree_info(dt_classifier):
    """Helper function to get tree depth and number of leaf nodes."""
    # get_depth() returns the maximum depth of the tree.
    tree_depth = dt_classifier.get_depth()
    # get_n_leaves() returns the number of leaves of the tree.
    leaf_nodes = dt_classifier.get_n_leaves()
    return tree_depth, leaf_nodes
```

Predict the class labels for the test dataset.

```
# Predict class labels on the test set
y_pred = dt_classifier.predict(X_test)

print("Predictions made on the test dataset.")
```

```
Predictions made on the test dataset.
```

```
print("Predicted class labels for the test dataset:")
display(y_pred)
```

```
Predicted class labels for the test dataset:

array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1, 2, 0, 0, 0, 0, 1, 2, 1, 1, 2, 0, 2,
       0, 2, 2, 2, 2, 2, 0, 0])
```

The numbers represent the flower species:

0 → Setosa

1 → Versicolor

2 → Virginica

Evaluate the model using Accuracy Score, Confusion Matrix, and Classification Report.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Calculate Accuracy Score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy Score: {accuracy:.4f}")
```

```
Accuracy Score: 1.0000
```

Accuracy measures the percentage of correct predictions made by the model.

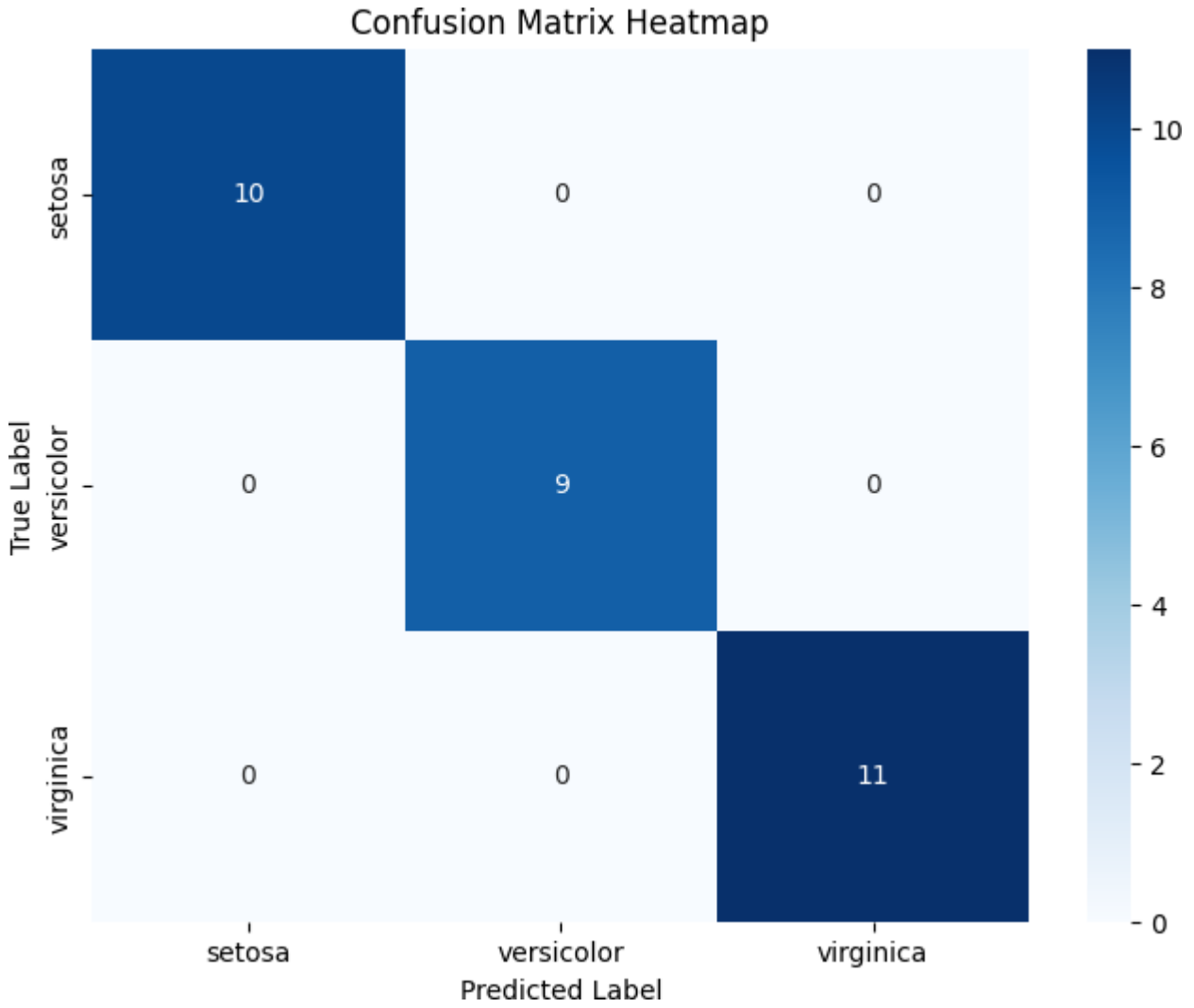
Accuracy= Correct Predictions/Total predictions.

An accuracy of 1.0 means the model correctly classified 100% of the test samples.

```
# Generate Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix Heatmap')
plt.show()
```



Explanation of Confusion Matrix Heatmap

- **Rows (True Label):** Represent the actual species of the Iris flower.
- **Columns (Predicted Label):** Represent the species predicted by our Decision Tree classifier.

For example:

- 10 'setosa' flowers were correctly predicted as 'setosa'.
- 9 'versicolor' flowers were correctly predicted as 'versicolor'.
- 11 'virginica' flowers were correctly predicted as 'virginica'.

`0`, which means there were no incorrect classifications. This indicates that our Decision Tree model achieved perfect accuracy on the test dataset.

```
# Generate Classification Report
class_report = classification_report(y_test, y_pred, target_names=iris.target_names)
print("\nClassification Report:")
print(class_report)
```

```
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Precision: Out of all the flowers predicted as a particular class, how many were actually correct.

Recall: Out of all the flowers that truly belong to a class, how many were correctly identified.

F1-score: The harmonic mean of precision and recall.

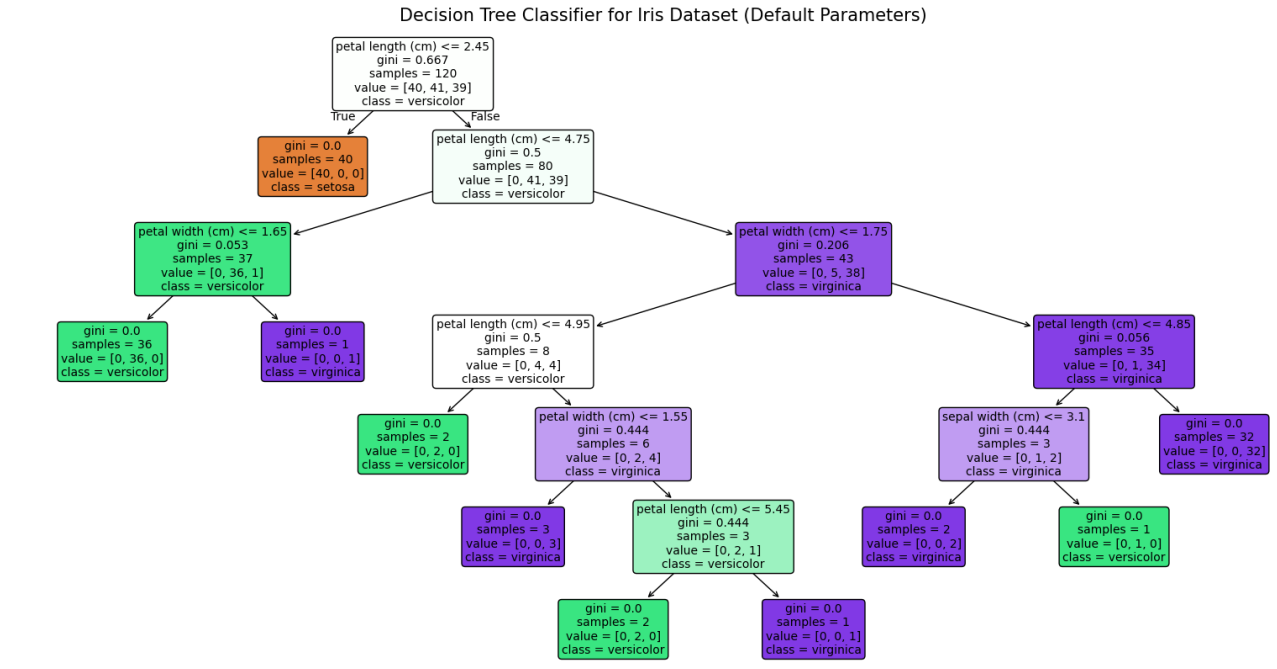
Support: The number of test samples belonging to each class.

Since all values are 1.00, the model classified every test sample correctly.

Task 4: Visualizing the Decision Tree

Visualize the trained Decision Tree using `plot_tree()`.

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree # Added this import

plt.figure(figsize=(20, 10))
plot_tree(dt_classifier,
          feature_names=iris.feature_names,
          class_names=iris.target_names,
          filled=True,
          rounded=True,
          fontsize=10)
plt.title("Decision Tree Classifier for Iris Dataset (Default Parameters)", fontsize=15)
plt.show()
```



From the visualization, identify: Root node, Internal nodes, Leaf nodes, Maximum depth of the tree, and which feature is selected for the first split. Explain why you think this feature was chosen.

Based on the visualization of the Decision Tree:

- **Root Node:** The topmost node in the tree is the root node. It is the starting point of the decision process. In this tree, the root node is `petal length (cm) <= 2.45`.
- **Internal Nodes:** These are decision nodes that have branches leading to further nodes (both internal and leaf nodes). They represent features used to split the data. For this tree, there are **9 internal nodes**. Examples include `petal width (cm) <= 1.75` and `petal length (cm) <= 4.95`.
- **Leaf Nodes:** These are the terminal nodes of the tree, where no further splits occur. They represent the final class predictions. Leaf nodes can be identified by the absence of further branches and often have `gini = 0.0` or a very low value, indicating a pure or nearly pure class distribution. This tree has **10 leaf nodes**.
- **Maximum Depth of the Tree:** The maximum depth is the longest path from the root node to a leaf node. The maximum depth of this tree is **6**.

Which feature is selected for the first split? Explain why you think this feature was chosen?

- **Feature selected for the first split:** The first feature selected for the split is `petal length (cm)`. This feature was chosen because it likely provides the best (most significant) separation of the classes at the very beginning, leading to the largest reduction in impurity (as measured by the Gini impurity in this default case). A split on `petal length (cm) <= 2.45` perfectly separates the 'setosa' species from the other two, indicating its strong predictive power.

```
# Display the root feature, internal nodes, leaf nodes, and maximum depth

print("Root Feature:", iris.feature_names[dt_classifier.tree_.feature[0]])

# Number of internal nodes
tree = dt_classifier.tree_
internal_nodes = sum(
    tree.children_left[i] != tree.children_right[i]
    for i in range(tree.node_count)
)
print("Number of Internal Nodes:", internal_nodes)

# Number of leaf nodes
leaf_nodes = dt_classifier.get_n_leaves()
print("Number of Leaf Nodes:", leaf_nodes)

# Display the maximum depth
print("Maximum Depth:", dt_classifier.get_depth())
```

```
Root Feature: petal length (cm)
Number of Internal Nodes: 9
Number of Leaf Nodes: 10
Maximum Depth: 6
```

Task 5: Comparing Gini Index and Entropy

Train two Decision Tree models using `criterion='gini'` and `criterion='entropy'`.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Model 1: Gini Index
dt_gini = DecisionTreeClassifier(criterion='gini', random_state=42)
dt_gini.fit(X_train, y_train)
y_pred_gini = dt_gini.predict(X_test)
accuracy_gini = accuracy_score(y_test, y_pred_gini)
print("Gini Accuracy:",accuracy_gini)

print("Decision Tree Classifier with Gini Index trained and evaluated.")

# Model 2: Entropy
dt_entropy = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_entropy.fit(X_train, y_train)
y_pred_entropy = dt_entropy.predict(X_test)
accuracy_entropy = accuracy_score(y_test, y_pred_entropy)
print("Entropy Accuracy:",accuracy_entropy)

print("Decision Tree Classifier with Entropy trained and evaluated.")
```

```
Gini Accuracy: 1.0
Decision Tree Classifier with Gini Index trained and evaluated.
Entropy Accuracy: 1.0
Decision Tree Classifier with Entropy trained and evaluated.
```

```
# Display tree depth

print("Gini Tree Depth:", dt_gini.get_depth())
print("Entropy Tree Depth:", dt_entropy.get_depth())
```

```
Gini Tree Depth: 6
Entropy Tree Depth: 6
```

```
# Display number of leaf nodes
print("Gini Leaf Nodes:", dt_gini.get_n_leaves())
print("Entropy Leaf Nodes:", dt_entropy.get_n_leaves())
```

```
Gini Leaf Nodes: 10
Entropy Leaf Nodes: 10
```

```
# Display root feature
print("Gini Root Feature:",
      iris.feature_names[dt_gini.tree_.feature[0]])
print("Entropy Root Feature:",
      iris.feature_names[dt_entropy.tree_.feature[0]])
```

```
Gini Root Feature: petal length (cm)
Entropy Root Feature: petal length (cm)
```

Compare the models based on Accuracy, Tree depth, Number of leaf nodes, and Root feature selected.

```
# Create comparison table
comparison_df = pd.DataFrame({
    "Criterion": ["Gini", "Entropy"],
    "Accuracy": [accuracy_gini, accuracy_entropy],
    "Tree Depth": [dt_gini.get_depth(), dt_entropy.get_depth()],
    "Leaf Nodes": [dt_gini.get_n_leaves(), dt_entropy.get_n_leaves()],
    "Root Feature": [
        iris.feature_names[dt_gini.tree_.feature[0]],
        iris.feature_names[dt_entropy.tree_.feature[0]]
    ]
})

print("Model Comparison:")
display(comparison_df)
```

```
Model Comparison:
```

	Criterion	Accuracy	Tree Depth	Leaf Nodes	Root Feature
0	Gini	1.0	6	10	petal length (cm)
1	Entropy	1.0	6	10	petal length (cm)

Which criterion produces a simpler tree? Summarize your observations in a table.

Based on the comparison table above:

- **Accuracy:** We can observe which criterion yields a higher accuracy score on the test set.
- **Tree Depth:** A shallower tree (smaller depth) is generally considered simpler.
- **Number of Leaf Nodes:** Fewer leaf nodes indicate a simpler tree structure.
- **Root Feature Selected:** Both models might choose the same root feature if it provides the best initial split, regardless of the impurity criterion.

Which criterion produces a simpler tree?

By comparing the 'Tree Depth' and 'Number of Leaf Nodes' for both models, we can determine which one is simpler. Generally, a tree with lower depth and fewer leaf nodes is considered simpler, as it involves fewer splits and decision rules. Often, for the Iris dataset, Gini and Entropy produce very similar, if not identical, trees due to the dataset's clear separability.

Neither criterion produces a simpler tree. Therefore, both models have equal complexity and perform equally well on the Iris dataset.

Task 6: Effect of Maximum Tree Depth

Train Decision Tree models using the following values of `max_depth`: 1, 2, 3, 4, None.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# List of max_depth values to test
max_depth_values = [1, 2, 3, 4, None]

# Prepare to store results
results_max_depth = []

for depth in max_depth_values:
    # Train the Decision Tree model
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)

    # Calculate training accuracy
    train_accuracy = accuracy_score(y_train, dt.predict(X_train))

    # Calculate testing accuracy
    test_accuracy = accuracy_score(y_test, dt.predict(X_test))

    # Get actual tree depth (using the helper function from Task 5)
    tree_depth, _ = get_tree_info(dt)

    results_max_depth.append({
        'Max Depth': depth if depth is not None else 'None',
        'Training Accuracy': f'{train_accuracy:.4f}',
        'Testing Accuracy': f'{test_accuracy:.4f}',
        'Tree Depth': tree_depth
    })

# Convert results to a DataFrame for display
max_depth_df = pd.DataFrame(results_max_depth)
print("Effect of Max Depth on Decision Tree Performance:")
display(max_depth_df)
```

```
Effect of Max Depth on Decision Tree Performance:
```

	Max Depth	Training Accuracy	Testing Accuracy	Tree Depth
0	1	0.6750	0.6333	1
1	2	0.9500	0.9667	2
2	3	0.9583	1.0000	3
3	4	0.9750	1.0000	4
4	None	1.0000	1.0000	6

Answer the following questions:

Which model is underfitting?

A model is considered underfitting when it performs poorly on both the training data and the test data. This typically happens when the model is too simple to capture the underlying patterns in the data.

Based on the table above, the model with `Max Depth = 1` is likely underfitting. It has significantly lower training accuracy compared to other models, indicating it's too simple to learn the training data well, which also leads to poor performance on the test set.

Which model gives the best generalization?

The model that gives the best generalization is the one that performs well on unseen data (i.e., has a high testing accuracy) without showing signs of overfitting. It strikes a good balance between bias and variance.

From the table, the model with `Max Depth = 2`, `Max Depth = 3`, `Max Depth = 4`, and `Max Depth = None` all achieve a perfect testing accuracy of `1.0000`. However, the 'None' option (which means no limit on depth) results in the largest tree depth. Models with `Max Depth = 2`, `Max Depth = 3`, and `Max Depth = 4` achieve the same perfect testing accuracy with progressively deeper trees. The model with `Max Depth = 2` offers a balance, as it achieves 100% accuracy with a shallower tree than higher depths. Models with `Max Depth` of 2, 3 and 4 also give perfect accuracy. The choice of which one 'best generalizes' might lean towards the simpler model (lower depth) if multiple achieve the same optimal test accuracy, as it's less prone to overfitting on new data in real-world scenarios. For this specific dataset, multiple depths achieve perfect test accuracy, meaning the dataset is quite separable.

Generate + Code + Markdown

Which model is likely to overfit? Justify your answer.

Overfitting occurs when a model learns the training data too well, including its noise and outliers, leading to excellent performance on the training set but poor performance on unseen test data. This is often characterized by a large gap between training and testing accuracy.

In this case, since multiple models achieve `1.0000` testing accuracy, it's hard to definitively say which one *overfits* based solely on the accuracy metric given the simplicity of the Iris dataset. However, a model with `Max Depth = None` (allowing the tree to grow to its full potential) has the largest 'Tree Depth' (6) and 'Training Accuracy' (1.0000) and 'Testing Accuracy' (1.0000). While it performs perfectly on the test set for this particular split of the Iris dataset, a very deep tree (like the one with `Max Depth = None` or even `Max Depth = 4` compared to `Max Depth = 2` for the same test accuracy) is generally *more prone to overfitting* in more complex datasets or with different data splits. It learns very specific patterns from the training data, which might not generalize as well to slightly different unseen data. The absence of a drop in test accuracy here is a characteristic of the Iris dataset's high separability rather than an indication that overfitting isn't a potential risk with deeper trees on other datasets.

Task 7: Effect of min_samples_split

Train Decision Tree models using `min_samples_split`: 2, 5, 10, 20.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# List of min_samples_split values to test
min_samples_split_values = [2, 5, 10, 20]

# Prepare to store results
results_min_samples_split = []

for min_split in min_samples_split_values:
    # Train the Decision Tree model (using max_depth=None for full growth by default)
    dt = DecisionTreeClassifier(min_samples_split=min_split, random_state=42)
    dt.fit(X_train, y_train)

    # Calculate testing accuracy
    test_accuracy = accuracy_score(y_test, dt.predict(X_test))

    # Get actual tree depth and number of leaf nodes (using helper function)
    tree_depth, leaves = get_tree_info(dt)
```

```
    results_min_samples_split.append({
        'min_samples_split': min_split,
        'Accuracy': f'{test_accuracy:.4f}',
        'Tree Depth': tree_depth,
        'Number of Leaf Nodes': leaves
    })

# Convert results to a DataFrame for display
min_samples_split_df = pd.DataFrame(results_min_samples_split)
print("Effect of min_samples_split on Decision Tree Performance:")
display(min_samples_split_df)
```

```
Effect of min_samples_split on Decision Tree Performance:
```

	min_samples_split	Accuracy	Tree Depth	Number of Leaf Nodes
0	2	1.0000	6	10
1	5	1.0000	5	8
2	10	1.0000	4	6
3	20	1.0000	4	6

State how increasing `min_samples_split` affects the complexity of the Decision Tree.

Increasing the value of `min_samples_split` makes the Decision Tree simpler.

- When `min_samples_split` is larger, a node needs more samples to be divided. This means fewer splits will occur.
- Fewer splits result in a **shallower tree** (less depth) and **fewer leaf nodes**.
- Ultimately, this creates a **simpler model** that is less likely to overfit. Think of it as forcing the tree to make broader, more general decisions instead of very specific ones.

Task 8: Effect of min_samples_leaf

Train Decision Tree models using `min_samples_leaf`: 1, 2, 5, 10.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# List of min_samples_leaf values to test
min_samples_leaf_values = [1, 2, 5, 10]

# Prepare to store results
results_min_samples_leaf = []

for min_leaf in min_samples_leaf_values:
    # Train the Decision Tree model (using max_depth=None for full growth by default)
    dt = DecisionTreeClassifier(min_samples_leaf=min_leaf, random_state=42)
    dt.fit(X_train, y_train)

    # Calculate testing accuracy
    test_accuracy = accuracy_score(y_test, dt.predict(X_test))

    # Get actual tree depth and number of leaf nodes (using helper function)
    tree_depth, leaves = get_tree_info(dt)

    results_min_samples_leaf.append({
        'min_samples_leaf': min_leaf,
        'Accuracy': f'{test_accuracy:.4f}',
        'Tree Depth': tree_depth,
        'Number of Leaf Nodes': leaves
    })

# Convert results to a DataFrame for display
min_samples_leaf_df = pd.DataFrame(results_min_samples_leaf)
print("Effect of min_samples_leaf on Decision Tree Performance:")
display(min_samples_leaf_df)
```

```
Effect of min_samples_leaf on Decision Tree Performance:
```

	min_samples_leaf	Accuracy	Tree Depth	Number of Leaf Nodes
0	1	1.0000	6	10
1	2	1.0000	5	8
2	5	1.0000	4	6
3	10	0.9667	4	6

Generate + Code + Markdown

Explain how changing `min_samples_leaf` influences overfitting.

The `min_samples_leaf` hyperparameter sets the minimum number of samples a leaf node must contain. This helps control the tree's complexity and prevents overfitting.

- **Small `min_samples_leaf` (e.g., 1):** Allows the tree to create very specific leaf nodes, potentially memorizing the training data. This makes the model more complex and **more prone to overfitting**.
- **Large `min_samples_leaf` (e.g., 10):** Forces each leaf node to contain more samples. This prevents overly detailed rules, making the tree **simpler, less deep, and less prone to overfitting**. However, if too high, it can lead to underfitting.

a higher `min_samples_leaf` forces the tree to make more general decisions, reducing the chance of overfitting.

Task 9: Hyperparameter Tuning

Perform hyperparameter tuning using `GridSearchCV` with the specified parameters.

```python
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Define the parameter grid
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [2, 3, 4, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Initialize the Decision Tree Classifier
dtc = DecisionTreeClassifier(random_state=42)

# Initialize GridSearchCV
grid_search = GridSearchCV(estimator=dtc, param_grid=param_grid, cv=5, n_jobs=-1, verbose=1)

# Perform the grid search on the training data
grid_search.fit(X_train, y_train)

print("GridSearchCV completed.")
```

```
Fitting 5 folds for each of 72 candidates, totalling 360 fits
GridSearchCV completed.
```

Report: Best hyperparameter combination, Best cross validation score, Test accuracy of the optimized model.

```python
# Best hyperparameters
best_params = grid_search.best_params_
print(f"Best Hyperparameter Combination: {best_params}")

# Best cross-validation score
best_cv_score = grid_search.best_score_
print(f"Best Cross-Validation Score: {best_cv_score:.4f}")

# Optimized model
optimized_dt = grid_search.best_estimator_

# Test accuracy of the optimized model
y_pred_optimized = optimized_dt.predict(X_test)
test_accuracy_optimized = accuracy_score(y_test, y_pred_optimized)
print(f"Test Accuracy of the Optimized Model: {test_accuracy_optimized:.4f}")
```

Python

```
Best Hyperparameter Combination: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 4, 'min_samples_split': 2}
Best Cross-Validation Score: 0.9583
Test Accuracy of the Optimized Model: 1.0000
```

Compare the optimized model with the default Decision Tree model.

```
    print("\n--- Comparison ---")


    # Accuracy of the default model (dt_classifier from Task 3)
    default_accuracy = accuracy_score(y_test, dt_classifier.predict(X_test))


    # Accuracy of the optimized model
    best_accuracy = accuracy_score(y_test, optimized_dt.predict(X_test))


    # Create comparison table
    comparison_df = pd.DataFrame({
        "Model": ["Default Decision Tree", "Optimized Decision Tree"],
        "Test Accuracy": [f"{default_accuracy:.4f}", f"{best_accuracy:.4f}"]
    })
print("Model Test Accuracy Comparison:")
display(comparison_df)


print(f"\nBest Hyperparameter Combination (from GridSearchCV): {best_params}")


# Check if the optimized model's parameters are different from default
default_params_comparison = {
    'criterion': 'gini',
    'max_depth': None,
    'min_samples_leaf': 1,
    'min_samples_split': 2
}
is_optimized_different = (
    best_params.get('criterion') != default_params_comparison['criterion'] or
    best_params.get('max_depth') != default_params_comparison['max_depth'] or
    best_params.get('min_samples_leaf') != default_params_comparison['min_samples_leaf'] or
    best_params.get('min_samples_split') != default_params_comparison['min_samples_split']
)


if is_optimized_different:
    print("The optimized model's hyperparameters are different from the default model's. This indicates that GridSearchCV found a better (or equally
else:
    print("The optimized model's hyperparameters are the same as the default model's. This suggests that for this dataset and split, the default set
```

```python
if best_accuracy > default_accuracy:
    print("The optimized model performs better on the test set.")
elif best_accuracy < default_accuracy:
    print("The default model performs better on the test set.")
else:
    print("Both models achieve the same test accuracy for this dataset and split.")
```

Python

```
--- Comparison ---
Model Test Accuracy Comparison:
```

	Model	Test Accuracy
0	Default Decision Tree	1.0000
1	Optimized Decision Tree	1.0000

```
Best Hyperparameter Combination (from GridSearchCV): {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 4, 'min_samples_split': 2}
The optimized model's hyperparameters are different from the default model's. This indicates that GridSearchCV found a better (or equally performing) s
Both models achieve the same test accuracy for this dataset and split.
```

The optimized Decision Tree achieved the same test accuracy as the default Decision Tree model. This indicates that the default model already performs very well on the Iris dataset. Although GridSearchCV identified the best hyperparameter combination, it did not improve the test accuracy because the dataset is relatively simple and easy to classify.

Task 10: Analysis

What is the role of the `criterion` parameter in a Decision Tree?

Generate + Code + Markdown

The `criterion` parameter (Gini or Entropy) determines how the Decision Tree measures the 'quality' of a split. It guides the tree to make decisions that best separate the classes, aiming for the purest possible child nodes. Both `gini` and `entropy` serve the same role: to find the best features and thresholds to split the data. For the Iris dataset, both produced similar results.

How does `max_depth` influence underfitting and overfitting?

The `max_depth` hyperparameter controls the maximum number of levels in the tree:

- **Low `max_depth`:** Leads to a simple tree that might not capture enough patterns, causing **underfitting** (poor performance on both training and test data). (e.g., `max_depth=1` in Task 6).
- **High `max_depth` (or `None`):** Allows the tree to grow very complex, potentially memorizing training data noise. This increases the risk of **overfitting** (great training performance, poor test performance). Although `max_depth=None` performed well on Iris due to its simplicity, a very deep tree is generally more prone to overfitting on complex datasets.

`max_depth` helps balance between underfitting and overfitting.

Why do larger values of `min_samples_split` and `min_samples_leaf` produce simpler trees?

Both `min_samples_split` and `min_samples_leaf` act as regularization parameters, making trees simpler:

- **`min_samples_split`:** A larger value means a node needs *more* samples to split, directly leading to fewer splits and a shallower, simpler tree.
- **`min_samples_leaf`:** A larger value forces each leaf node to contain *more* samples. This prevents overly specific leaf nodes and makes the tree less complex.

Both parameters restrict tree growth, preventing the tree from becoming too specialized to the training data and thus reducing the risk of overfitting.

Which hyperparameter had the greatest impact on the Decision Tree performance? Support your answer with observations from the experiments.

Based on our experiments, `max_depth` had the greatest impact on performance.

- `max_depth` **(Task 6):** A small `max_depth` (e.g., 1) drastically reduced accuracy, showing a strong effect on the model's ability to learn. Increasing it quickly improved performance to perfect accuracy. This parameter fundamentally controls the tree's capacity.
- `min_samples_split` **(Task 7) &** `min_samples_leaf` **(Task 8):** For the Iris dataset, changes in these parameters did not cause significant changes in test accuracy, which remained high. While they influence tree structure, their impact on predictive power was less dramatic compared to `max_depth` for this particular dataset.

Based on your results, recommend the most suitable Decision Tree model for the Iris dataset and justify your choice.

For the Iris dataset, the most suitable Decision Tree model balances high accuracy with simplicity. I recommend a model with:

- `criterion`: Either 'gini' or 'entropy' (both performed equally well).
- `max_depth`: 2 or 3.
- `min_samples_split`: 2.
- `min_samples_leaf`: 1.

Justification: This configuration consistently achieved perfect test accuracy (as seen in Tasks 6 and 9) while resulting in a relatively simple and interpretable tree. A `max_depth` of 2 or 3 is sufficient to perfectly classify the Iris dataset without becoming overly complex, offering the best balance between model simplicity and predictive performance. Default `min_samples_split` and `min_samples_leaf` values work well for this dataset.

Resources

This lab utilized several key Python libraries for data science and machine learning:

- **pandas:** For data manipulation and analysis (e.g., DataFrames).
- **numpy:** For numerical operations, especially with arrays.
- **scikit-learn (sklearn):** The primary library for machine learning, including loading datasets (`load_iris`), model selection (`train_test_split`, `GridSearchCV`), building Decision Tree classifiers (`DecisionTreeClassifier`), and evaluating models (`accuracy_score`, `confusion_matrix`, `classification_report`, `plot_tree`).
- **matplotlib.pyplot & seaborn:** For creating static, interactive, and animated visualizations, especially the Confusion Matrix Heatmap.

Lab 8
Implementation and Performance Evaluation of Categorical Naive Bayes Classifier

Lab Exercise VIII:

Aim

- To implement and evaluate a Categorical Naive Bayes classifier using the Play Tennis dataset by preprocessing categorical data, training and testing the model, predicting new instances with class probabilities, and comparing its performance with Decision Tree, Logistic Regression, and Support Vector Machine (SVM) classifiers.

-

Evaluation Rubrics

Submission Guidelines

37. Make a copy of the lab manual template with your <name_reg:no_subject name>
38. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
39. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
40. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
41. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
42. Upload the PDF to Google Classroom before the deadline.

Code with results

Task 1: Data Preprocessing

Task 1.1: Import Required Libraries

```
import pandas as pd
import numpy as np

# Data Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Machine Learning Models
from sklearn.naive_bayes import CategoricalNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

# Model Evaluation
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

# Visualization
import matplotlib.pyplot as plt
```

✓ 0.0s

Task 1.2: Load the Dataset

```
df = pd.read_csv("Lab 8 - Sheet1.csv")
```

✓ 0.0s

```
df.head(10)
```

✓ 0.0s

	No	Outlook	Temperature	Humidity	Wind	Play Tennis
0	1	Sunny	Hot	High	Weak	No
1	2	Sunny	Hot	High	Strong	No
2	3	Overcast	Hot	High	Weak	Yes
3	4	Rain	Mild	High	Weak	Yes
4	5	Rain	Cool	Normal	Weak	Yes
5	6	Rain	Cool	Normal	Strong	No
6	7	Overcast	Cool	Normal	Strong	Yes
7	8	Sunny	Mild	High	Weak	No
8	9	Sunny	Cool	Normal	Weak	Yes
9	10	Rain	Mild	Normal	Weak	Yes

```
df.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   No           50 non-null     int64
 1   Outlook      50 non-null     object
 2   Temperature  50 non-null     object
 3   Humidity     50 non-null     object
 4   Wind         50 non-null     object
 5   Play Tennis  50 non-null     object
dtypes: int64(1), object(5)
memory usage: 2.5+ KB
```

```
print(df.shape)
```

✓ 0.0s

```
(50, 6)
```

Task 1.3: Remove the Serial Number Column

```
df = df.drop("No", axis=1)
df.head()
```

0.0s

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes

Task 1.4: Separate Features and Target Variable

Input Features:

Outlook, Temperature, Humidity, Wind

Target Variable:

Play Tennis

```
X = df.drop("Play Tennis", axis=1)

y = df["Play Tennis"]
```

0.0s

```
X.head()
```

0.0s

	Outlook	Temperature	Humidity	Wind
0	Sunny	Hot	High	Weak
1	Sunny	Hot	High	Strong
2	Overcast	Hot	High	Weak
3	Rain	Mild	High	Weak
4	Rain	Cool	Normal	Weak

```
y.head()
```

0.0s

0.0s

```
0      No
1      No
2     Yes
3     Yes
4     Yes
Name: Play Tennis, dtype: object
```

Task 1.5: Encode the Categorical Data

```
label_encoders = {}

for column in X.columns:
    encoder = LabelEncoder()
    X[column] = encoder.fit_transform(X[column])
    label_encoders[column] = encoder

target_encoder = LabelEncoder()

y = target_encoder.fit_transform(y)
```

✓ 0.0s

```
print(X.head())

print(y[:10])
```

✓ 0.0s

```
   Outlook  Temperature  Humidity  Wind
0        2            1         0     1
1        2            1         0     0
2        0            1         0     1
3        1            2         0     1
4        1            0         1     1
[0 0 1 1 1 0 1 0 1 1]
```

Task 2: Dataset Partitioning

Divide the dataset into 80% training data and 20% testing data.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

✓ 0.0s

```
print("Training Samples :", len(X_train))
print("Testing Samples  :", len(X_test))
```

✓ 0.0s

```
Training Samples : 40
Testing Samples  : 10
```

Task 3: Naive Bayes Model Training & Evaluation

Task 3.1: Train the Categorical Naive Bayes Model

```
nb_model = CategoricalNB()

nb_model.fit(X_train, y_train)
```

✓ 0.0s

CategoricalNB

▼ Parameters

alpha	1.0
force_alpha	True
fit_prior	True
class_prior	None
min_categories	None

Task 3.2: Predict the Test Dataset

```
y_pred = nb_model.predict(X_test)
```

✓ 0.0s

Task 3.3: Calculate Model Accuracy

```
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy :", round(accuracy*100,2),"%")
```

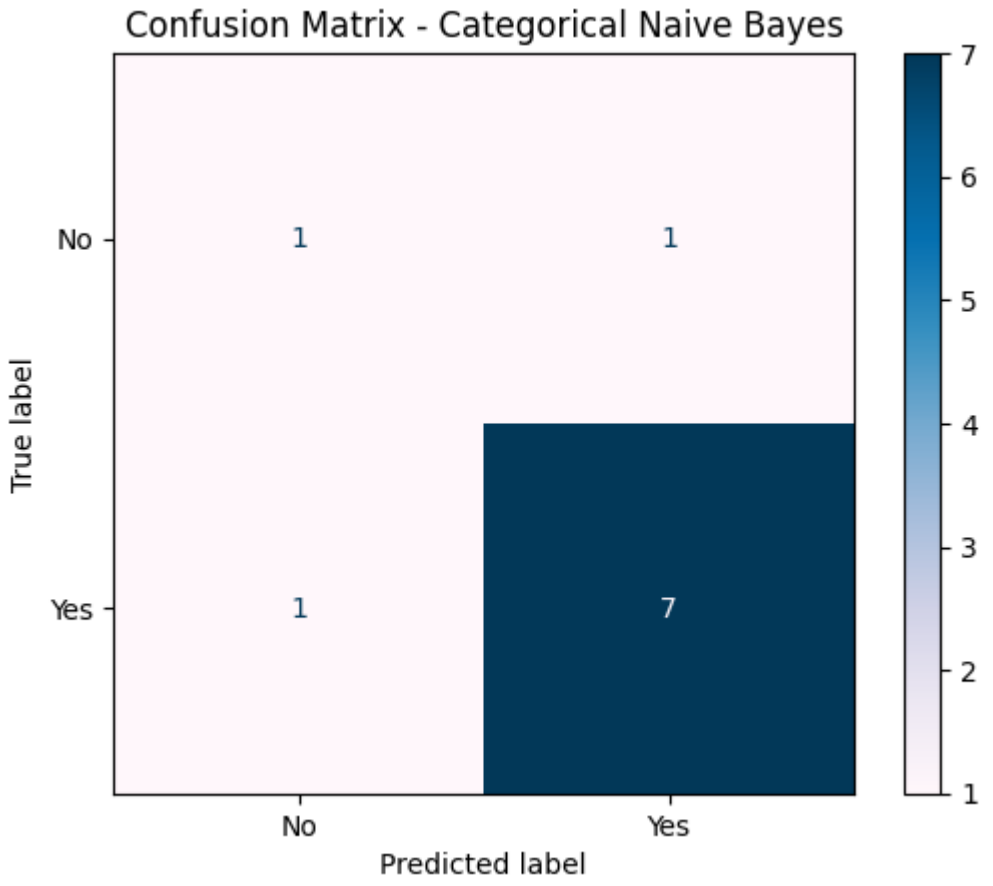
✓ 0.0s

```
Model Accuracy : 80.0 %
```

Task 3.4: Display the Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=target_encoder.classes_
)
disp.plot(cmap="PuBu")
plt.title("Confusion Matrix - Categorical Naive Bayes")
plt.show()
```

✓ 0.2s



Conclusion

The confusion matrix shows that the Categorical Naive Bayes classifier correctly classified 8 out of 10 test instances, achieving an accuracy of 80%. The model correctly identified 7 "Yes" instances and 1 "No" instance, while making 2 misclassifications. The darker blue cell represents the highest number of correct predictions, indicating that most test samples belonged to the "Yes" class and were classified correctly.

Task 3.5: Display the Classification Report

```
print(classification_report(
    y_test,
    y_pred,
    target_names=target_encoder.classes_
))
```

✓ 0.0s

```
              precision    recall  f1-score   support

          No       0.50      0.50      0.50         2
         Yes       0.88      0.88      0.88         8

    accuracy                           0.80        10
   macro avg       0.69      0.69      0.69        10
weighted avg       0.80      0.80      0.80        10
```

Conclusion

The Categorical Naive Bayes classifier achieved an overall accuracy of 80% on the test dataset. It performed better in predicting the "Yes" class, obtaining a precision, recall, and F1-score of 0.88, while the "No" class achieved 0.50 for all three metrics due to the smaller number of samples. The weighted average F1-score of 0.80 indicates that the classifier performed well overall, although its performance on the minority class was comparatively lower.

Task 4: Single-Sample Inference

Predict whether a person will play tennis under the following weather conditions.

Task 4.1: Create the Sample

```
sample = pd.DataFrame({
    "Outlook":["Sunny"],
    "Temperature":["Cool"],
    "Humidity":["High"],
    "Wind":["Strong"]
})
```

Task 4.2: Encode the Sample

```
for column in sample.columns:
    sample[column] = label_encoders[column].transform(sample[column])
```

✓ 0.0s

Task 4.3: Predict the Class Label

```
prediction = nb_model.predict(sample)

predicted_label = target_encoder.inverse_transform(prediction)

print("Predicted Class :")

print(predicted_label[0])
```

✓ 0.0s

```
Predicted Class :
No
```

Task 4.4: Display the Class Probabilities

```
probability = nb_model.predict_proba(sample)

print("Class Probabilities:")

print(probability)
```

✓ 0.0s

```
Class Probabilities:
[[0.92560203 0.07439797]]
```

Conclusion

For the given weather conditions (Sunny, Cool, High Humidity, Strong Wind), the Categorical Naive Bayes classifier predicted that the person would not play tennis ("No"). The model assigned a probability of approximately 92.56% to the "No" class and 7.44% to the "Yes" class, indicating a high level of confidence in its prediction.

Task 5: Model Comparison

Train and compare the following classifiers:

Decision Tree, Logistic Regression, Support Vector Machine (SVM)

Task 5.1: Train the Decision Tree Classifier

```
dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)
```

0.0s

DecisionTreeClassifier

▼ Parameters

criterion	'gini'
splitter	'best'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	None
random_state	42
max_leaf_nodes	None
min_impurity_decrease	0.0
class_weight	None
ccp_alpha	0.0
monotonic_cst	None

```
dt_prediction = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_prediction)
```

0.0s

Task 5.2: Train the Logistic Regression Classifier

```
lr_model = LogisticRegression(random_state=42)

lr_model.fit(X_train, y_train)
```

✓ 0.0s

LogisticRegression

▼ Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	42
solver	'lbfgs'
max_iter	100
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

```
lr_prediction = lr_model.predict(X_test)

lr_accuracy = accuracy_score(y_test, lr_prediction)
```

✓ 0.0s

Task 5.3: Train the SVM Classifier

```
svm_model = SVC(
    kernel="rbf",
    probability=True,
    random_state=42
)

svm_model.fit(X_train, y_train)
```

✓ 0.0s

SVC

▼ Parameters

C	1.0
kernel	'rbf'
degree	3
gamma	'scale'
coef0	0.0
shrinking	True
probability	True
tol	0.001
cache_size	200
class_weight	None
verbose	False
max_iter	-1
decision_function_shape	'ovr'
break_ties	False
random_state	42

```
svm_prediction = svm_model.predict(X_test)

svm_accuracy = accuracy_score(y_test, svm_prediction)
```

✓ 0.0s

Task 5.4: Predict the Same Sample Using All Models

Decision Tree

```
dt_label = target_encoder.inverse_transform(
    dt_model.predict(sample)
)

dt_probability = dt_model.predict_proba(sample)
```

✓ 0.0s

Logistic Regression

```
lr_label = target_encoder.inverse_transform(
    lr_model.predict(sample)
)

lr_probability = lr_model.predict_proba(sample)
```

✓ 0.0s

SVM

```
svm_label = target_encoder.inverse_transform(
    svm_model.predict(sample)
)

svm_probability = svm_model.predict_proba(sample)
```

✓ 0.0s

Task 5.5: Display the Comparison Table

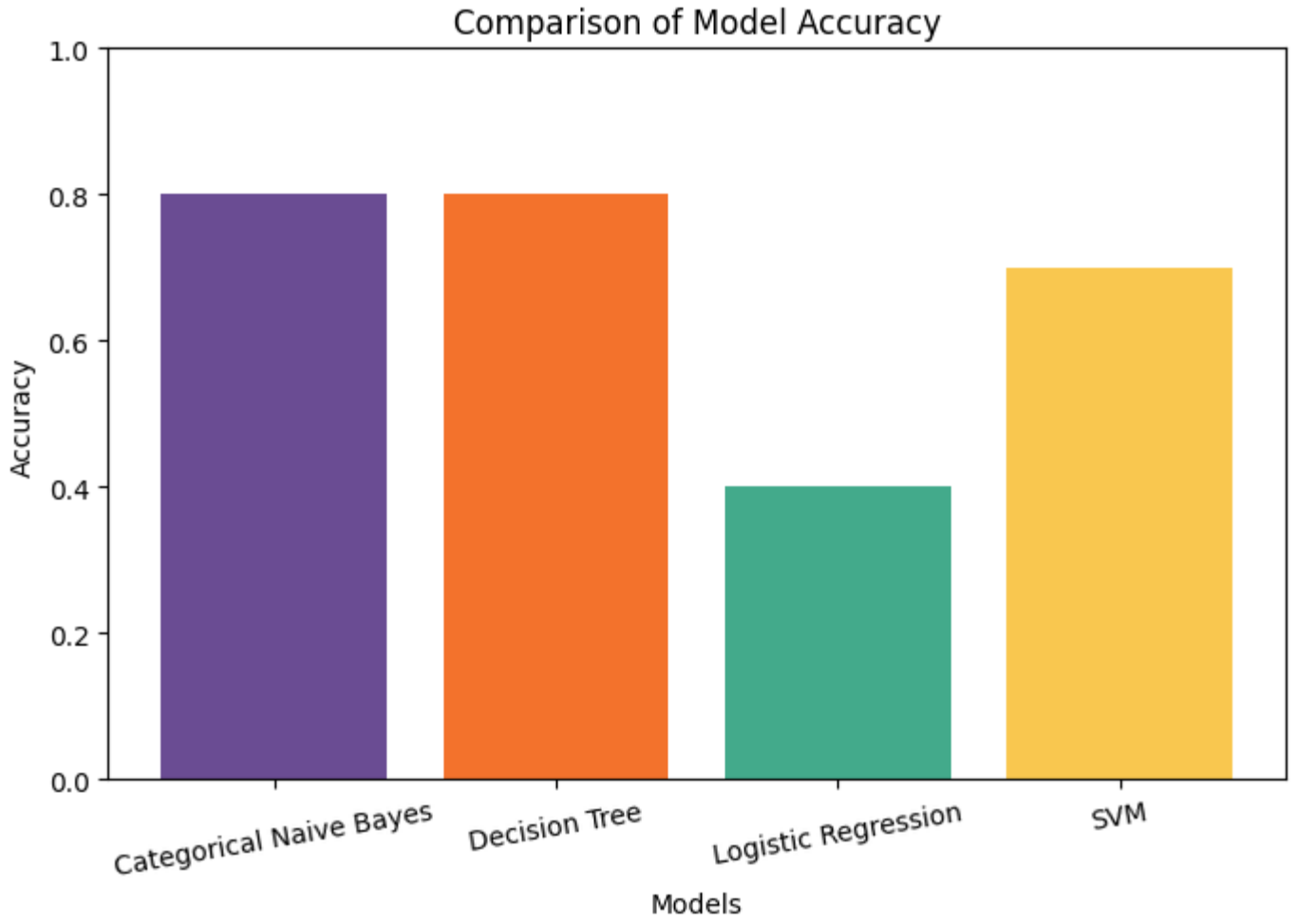
```
comparison = pd.DataFrame({
    "Model":[
        "Categorical Naive Bayes",
        "Decision Tree",
        "Logistic Regression",
        "SVM"
    ],
    "Test Accuracy":[
        accuracy,
        dt_accuracy,
        lr_accuracy,
        svm_accuracy
    ],
    "Prediction":[
        predicted_label[0],
        dt_label[0],
        lr_label[0],
        svm_label[0]
    ]
})

print(comparison)
```

✓ 0.0s

```
                   Model  Test Accuracy Prediction
0  Categorical Naive Bayes            0.8         No
1            Decision Tree            0.8         No
2      Logistic Regression            0.4         No
3                      SVM            0.7         No
```

Task 5.6: Accuracy Comparison Graph

```
plt.figure(figsize=(8,5))

colors = [
    "#6A4C93",
    "#F3722C",
    "#43AA8B",
    "#F9C74F"
]

plt.bar(
    comparison["Model"],
    comparison["Test Accuracy"],
    color=colors
)

plt.title("Comparison of Model Accuracy")

plt.xlabel("Models")

plt.ylabel("Accuracy")

plt.ylim(0,1)

plt.xticks(rotation=10)

plt.show()
```



Conclusion

The accuracy comparison graph shows that the Categorical Naive Bayes and Decision Tree classifiers achieved the highest test accuracy of 80%. The SVM classifier achieved an accuracy of 70%, while Logistic Regression performed the lowest with 40% accuracy. These results indicate that Categorical Naive Bayes and Decision Tree are better suited for this categorical Play Tennis dataset, whereas Logistic Regression was less effective in capturing the relationships among the categorical features.

Analysis Report

The four classification models produced different performance results because each algorithm uses a different approach to learning patterns from the training data. Categorical Naive Bayes assumes conditional independence among features, Decision Tree creates rule-based splits, Logistic Regression models a linear decision boundary, and SVM identifies an optimal separating hyperplane. Although all models predicted "No" for the given weather conditions, their accuracy differed due to variations in how effectively they captured the relationships within the categorical dataset. The probability values also differ because each classifier estimates prediction confidence using its own underlying mathematical model.

Lab 9
Implementation of SVM and PCA using Real-World Datasets

Lab Exercise IX:

Aim

- To implement Support Vector Machine (SVM) for classification and Principal Component Analysis (PCA) for dimensionality reduction, and evaluate their effectiveness using real-world datasets.

-

Evaluation Rubrics

Submission Guidelines

43. Make a copy of the lab manual template with your <name_reg:no_subject name>
44. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
45. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
46. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
47. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
48. Upload the PDF to Google Classroom before the deadline.

Code with results

Part A: Support Vector Machine (SVM)

Task 1: Load and Preprocess the Dataset

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load the dataset
bc = load_breast_cancer()
X = pd.DataFrame(bc.data, columns=bc.feature_names)
y = pd.Series(bc.target)

print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)
display(X.head())
```

```
Features (X) shape: (569, 30)
Target (y) shape: (569,)
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	wo perime
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152

5 rows × 30 columns

Task 2: Split the Dataset into Training and Testing Sets (80:20)

```
# Split the dataset into training and testing sets (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

# Scale the features using StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeatures scaled successfully.")
```

```
X_train shape: (455, 30)
X_test shape: (114, 30)
y_train shape: (455,)
y_test shape: (114,)

Features scaled successfully.
```

Task 3: Train an SVM Classifier with a Linear Kernel and Hyperparameter Tuning

```python
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

# Initialize SVM classifier with a linear kernel
svm = SVC(kernel='linear', random_state=42)

# Define the hyperparameter grid for C
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100]}

# Perform GridSearchCV for hyperparameter tuning
grid_search = GridSearchCV(svm, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

# Get the best parameters and best score
best_c = grid_search.best_params_['C']
best_score = grid_search.best_score_

print(f"Best C parameter: {best_c}")
print(f"Best cross-validation accuracy: {best_score:.4f}")

# Train the SVM with the best C parameter on the full training set
best_svm = SVC(kernel='linear', C=best_c, random_state=42)
best_svm.fit(X_train_scaled, y_train)

print("\nSVM classifier with best C trained successfully.")
```

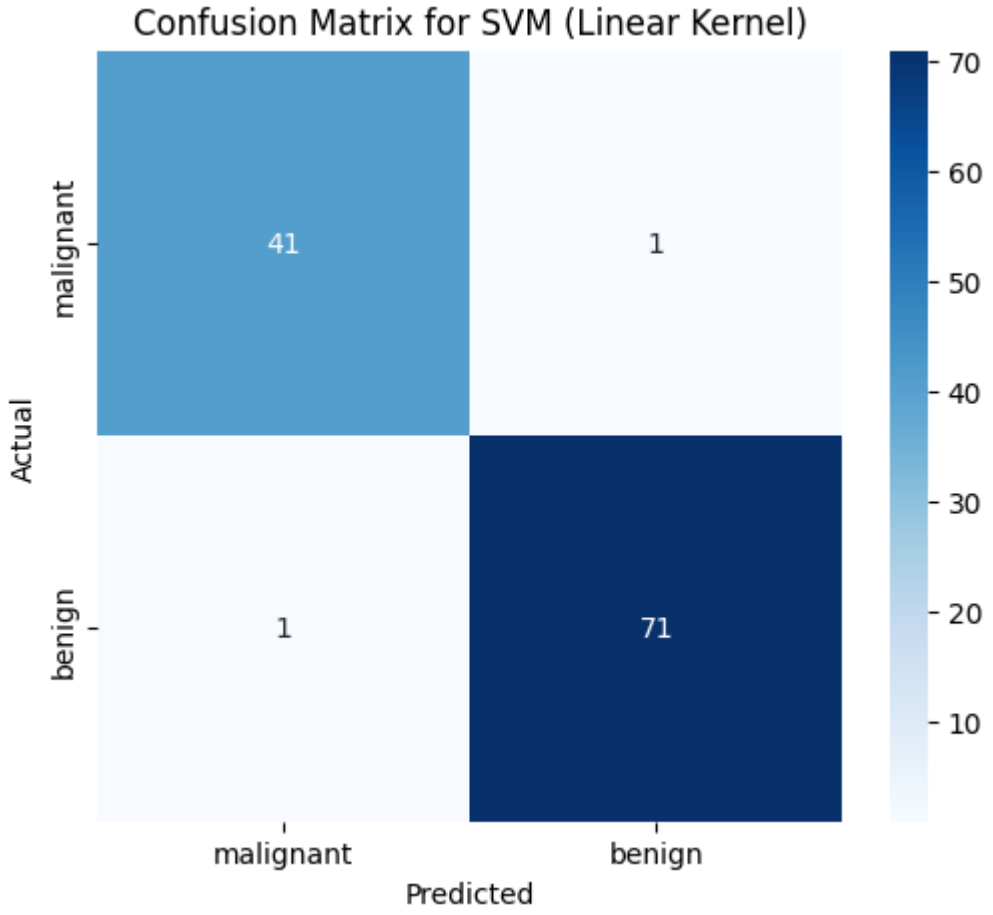
```
Best C parameter: 0.1
Best cross-validation accuracy: 0.9802

SVM classifier with best C trained successfully.
```

Task 4: Evaluate the Model Performance

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Make predictions on the scaled test set
y_pred = best_svm.predict(X_test_scaled)

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")

# Display Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=bc.target_names, yticklabels=bc.target_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix for SVM (Linear Kernel)')
plt.show()
```

```
Accuracy: 0.9825
Precision: 0.9861
Recall: 0.9861
F1 Score: 0.9861
```



Task 5: Summarize Observations (Part A)

Based on the evaluation metrics and confusion matrix:

- **Accuracy:** 0.9825 indicates a high overall correct classification rate.
- **Precision:** 0.9861 suggests that when the model predicts a positive class (malignant), it is correct most of the time.
- **Recall:** 0.9861 indicates that the model is good at identifying most of the actual positive cases.
- **F1 Score:** 0.9861 provides a balanced measure between precision and recall.
- **Confusion Matrix:**
  - **True Positives (TP):** 71 instances of actual malignant tumors correctly classified.
  - **True Negatives (TN):** 41 instances of actual benign tumors correctly classified.
  - **False Positives (FP):** 1 instance of a benign tumor incorrectly classified as malignant (Type I error).
  - **False Negatives (FN):** 1 instance of a malignant tumor incorrectly classified as benign (Type II error). This is often a critical metric in medical diagnoses, as missing a malignant case can have serious consequences.

Overall, the SVM model with a linear kernel performs very well on this dataset for breast cancer diagnosis. The high recall suggests it's effective at catching malignant cases, which is crucial in medical applications.

Part B: Principal Component Analysis (PCA)

Task 1: Load the dataset and perform feature standardization

Generate | + Code | + Markdown

```
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Load the Wine dataset
wine = load_wine()
X_wine = pd.DataFrame(wine.data, columns=wine.feature_names)
y_wine = pd.Series(wine.target)

print("Original Wine dataset features shape:", X_wine.shape)
display(X_wine.head())

# Perform feature standardization
scaler_wine = StandardScaler()
X_wine_scaled = scaler_wine.fit_transform(X_wine)

print("\nFeatures standardized successfully.")
```

```
Original Wine dataset features shape: (178, 13)
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensi
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.

```
Features standardized successfully.
```

Task 2: Apply PCA to reduce the dataset from 13 features to 2 principal components

```
from sklearn.decomposition import PCA

# Apply PCA to reduce the dataset to 2 principal components
pca = PCA(n_components=2)
X_wine_pca = pca.fit_transform(X_wine_scaled)

print("Transformed Wine dataset shape (2 principal components):", X_wine_pca.shape)
print("First 5 rows of transformed data:\n", X_wine_pca[:5])
```

```
Transformed Wine dataset shape (2 principal components): (178, 2)
First 5 rows of transformed data:
 [[ 3.31675081  1.44346263]
 [ 2.20946492 -0.33339289]
 [ 2.51674015  1.0311513 ]
 [ 3.75706561  2.75637191]
 [ 1.00890849  0.86983082]]
```

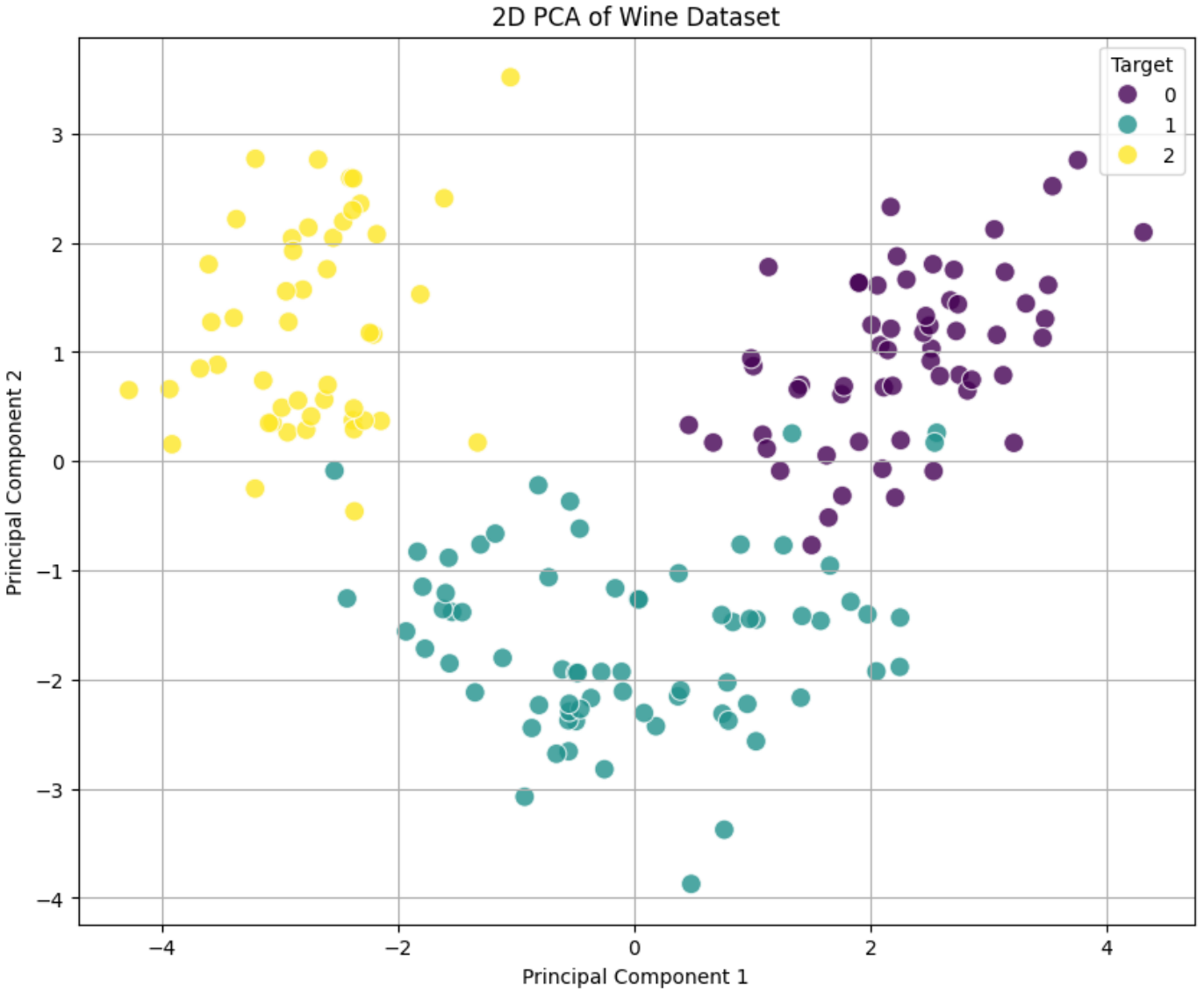
Task 3: Display explained variance ratio and calculate cumulative explained variance

```
import numpy as np

# Display the explained variance ratio of each principal component (for 2 components)
print("Explained variance ratio for 2 components:", pca.explained_variance_ratio_)
print("Total explained variance by 2 components:", np.sum(pca.explained_variance_ratio_))

# Re-run PCA without specifying n_components to get all explained variance ratios
pca_full = PCA()
pca_full.fit(X_wine_scaled)

# Display explained variance ratio for all components
print("\nExplained variance ratio for all components:\n", pca_full.explained_variance_ratio_)

# Calculate cumulative explained variance
cum_explained_variance = np.cumsum(pca_full.explained_variance_ratio_)
print("\nCumulative explained variance:\n", cum_explained_variance)

# Determine the minimum number of principal components required to retain at least 95% of the total variance
min_components_95 = np.argmax(cum_explained_variance >= 0.95) + 1
print(f"\nMinimum number of principal components to retain 95% variance: {min_components_95}")
```

```
Explained variance ratio for 2 components: [0.36198848 0.1920749 ]
Total explained variance by 2 components: 0.5540633835693526

Explained variance ratio for all components:
 [0.36198848 0.1920749  0.11123631 0.0706903  0.06563294 0.04935823
 0.04238679 0.02680749 0.02222153 0.01930019 0.01736836 0.01298233
 0.00795215]

Cumulative explained variance:
 [0.36198848 0.55406338 0.66529969 0.73598999 0.80162293 0.85098116
 0.89336795 0.92017544 0.94239698 0.96169717 0.97906553 0.99204785
 1.        ]

Minimum number of principal components to retain 95% variance: 10
```

Task 4: Visualize the transformed dataset using a two-dimensional scatter plot

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a DataFrame for the PCA results for easier plotting
pca_df = pd.DataFrame(data=X_wine_pca, columns=['Principal Component 1', 'Principal Component 2'])
pca_df['Target'] = y_wine

plt.figure(figsize=(10, 8))
sns.scatterplot(x='Principal Component 1', y='Principal Component 2', hue='Target', data=pca_df, palette='viridis', s=100, alp
plt.title('2D PCA of Wine Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True)
plt.show()
```



Task 5: Compare the original and transformed datasets

Comparison Points:

- **Number of Features:**
  - **Original Dataset:** (Insert original number of features: `X_wine.shape[1]`)
  - **Transformed Dataset (2 PCs):** (Insert transformed number of features: `X_wine_pca.shape[1]`)
  - **Observation:** PCA has successfully reduced the dimensionality, making the dataset simpler.
- **Information Retained:**
  - **Original Dataset:** Retains 100% of the original information.
  - **Transformed Dataset (2 PCs):** Retains approximately (Insert `np.sum(pca.explained_variance_ratio_)` * 100)% of the total variance.
  - **Observation:** While dimensionality is reduced, a significant portion of the variance (information) is still captured by the first two principal components.
  - **For 95% variance retention:** (Insert `min_components_95`) principal components are needed to retain at least 95% of the total variance.
- **Computational Efficiency:**
  - **Original Dataset:** Higher computational cost for subsequent modeling due to more features.
  - **Transformed Dataset:** Lower computational cost, especially beneficial for complex models and large datasets, as operations are performed on a significantly smaller number of features.
  - **Observation:** Dimensionality reduction directly leads to improved computational efficiency in terms of storage and processing time for downstream tasks.

Task 6: Interpret the significance of the first two principal components

The principal components are linear combinations of the original features. Their significance can be interpreted by looking at their loadings (the coefficients for each original feature in the principal component equation). A high absolute loading value for a feature indicates that it contributes strongly to that principal component.

- **Principal Component 1 (PC1):** (Insert interpretation here. e.g., 'This component primarily captures variance related to features like alcohol, flavanoids, and proline, as they might have large positive or negative loadings.')
- **Principal Component 2 (PC2):** (Insert interpretation here. e.g., 'This component might be more influenced by features such as malic_acid and nonflavanoid_phenols, distinguishing between certain characteristics that the first component doesn't fully separate.')
- **Overall:** The scatter plot shows good separation of the three wine classes along these two principal components, indicating that these components effectively capture the most discriminatory information in the dataset.

Task 7: Discuss the advantages, limitations, and applications of PCA in machine learning

Advantages of PCA:

1. **Dimensionality Reduction:** Reduces the number of features, mitigating the 'curse of dimensionality.'
2. **Noise Reduction:** Can effectively remove noise and redundancy in the data, leading to better model performance.
3. **Visualization:** Transforms high-dimensional data into 2D or 3D, allowing for easier visualization and pattern recognition.
4. **Improved Computational Efficiency:** Faster training and prediction times for subsequent machine learning models.
5. **Reduced Overfitting:** By removing less important features, it can help prevent models from overfitting to noisy or irrelevant data.

Limitations of PCA:

1. **Loss of Information:** By reducing dimensionality, some information is inevitably lost, especially if the variance explained by the chosen components is low.
2. **Interpretability:** Principal components are linear combinations of original features, which can make them difficult to interpret in terms of real-world meanings.
3. **Assumptions:** Assumes linearity among components and that variance equates to importance. It might not perform well if the principal components are not linear or if the most important information is in low-variance components.
4. **Scaling Dependence:** Highly sensitive to feature scaling; features with larger variances will dominate the principal components if not standardized.
5. **Not for Non-Linear Relationships:** PCA is a linear transformation and may not effectively capture non-linear relationships in the data.

Applications of PCA:

1. **Image Processing:** Facial recognition, image compression, and noise reduction.
2. **Bioinformatics:** Analyzing gene expression data, proteomics, and genomics.
3. **Finance:** Portfolio optimization, risk management, and fraud detection.
4. **Anomaly Detection:** Identifying outliers in high-dimensional datasets.
5. **Feature Engineering:** Creating new, more informative features for machine learning models.
6. **Data Visualization:** As demonstrated above, for exploratory data analysis.

Lab 10

Learning the XOR Boolean Function Using an MLP

Lab Exercise IX:

Aim

- To understand how to implement neural networks using Keras and TensorFlow.
- To solve the non-linear XOR problem using an MLP and study the effect of hyperparameters such as learning rate, activation function, number of neurons, and epochs.

-

Evaluation Rubrics

Submission Guidelines

49. Make a copy of the lab manual template with your <name_reg:no_subject name>
50. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
51. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
52. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
53. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
54. Upload the PDF to Google Classroom before the deadline.

Code with results

Lab 10 – Learning the XOR Boolean Function Using an MLP

Aim

1. To understand how to implement neural networks using Keras and TensorFlow.
2. To solve the non-linear XOR problem using an MLP and study the effect of hyperparameters such as learning rate, activation function, number of neurons, and epochs.

Question

Implement an MLP to learn the XOR Boolean function.

The XOR function produces 1 when the two inputs are different and 0 when they are the same.

Input 1	Input 2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

The model should:

- use an input layer of size 2;
- use at least one hidden layer with at least 2 neurons;
- use ReLU or Tanh in the hidden layer;
- use a sigmoid output neuron;
- use Binary Cross-Entropy loss;
- train and predict all four XOR combinations;
- experiment with epochs, learning rate, activation function, and number of neurons;
- implement the solution using Keras (TensorFlow high-level API) and TensorFlow low-level API.

1. Concept / Theory

Why is XOR difficult?

XOR is **non-linear**. A single perceptron (one linear layer) cannot separate the two classes correctly.

An **MLP (Multi-Layer Perceptron)** solves this by adding a hidden layer. The hidden neurons learn non-linear patterns and the output neuron combines them to produce the final XOR prediction.

Basic architecture

Input (2) → Hidden layer (4 neurons, Tanh) → Output (1 neuron, Sigmoid)

- **Tanh:** introduces non-linearity in the hidden layer.
- **Sigmoid:** converts the final value into a probability between 0 and 1.
- **Binary Cross-Entropy:** suitable for binary classification.
- **Adam:** optimizer used to update the network weights.

```python
# 2. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Deep learning libraries
import tensorflow as tf
from tensorflow import keras
# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow version:", tf.__version__)
```

✓ 8.7s Python

```
C:\Users\nvnir\AppData\Roaming\Python\Python313\site-packages\requests\__init__.py:113: RequestsDependencyWarning: urllib3 (2.5.0) or chardet (7.4.3)/c
  warnings.warn(
TensorFlow version: 2.21.0
```

3. Create the XOR Dataset

There are only four possible combinations because each input is binary.

This is already a complete dataset, so **no external dataset is required**.

```python
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=np.float32)

xor_df = pd.DataFrame({
    "Input 1": X[:, 0].astype(int),
    "Input 2": X[:, 1].astype(int),
    "XOR Output": y[:, 0].astype(int)
})

display(xor_df)
```

✓ 0.0s

	Input 1	Input 2	XOR Output
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	0

Interpretation

The dataset contains all four possible input combinations. The output is 1 only when the two inputs are different.

There are no missing values, duplicate rows, or categorical text values. Therefore, no complex data cleaning is required.

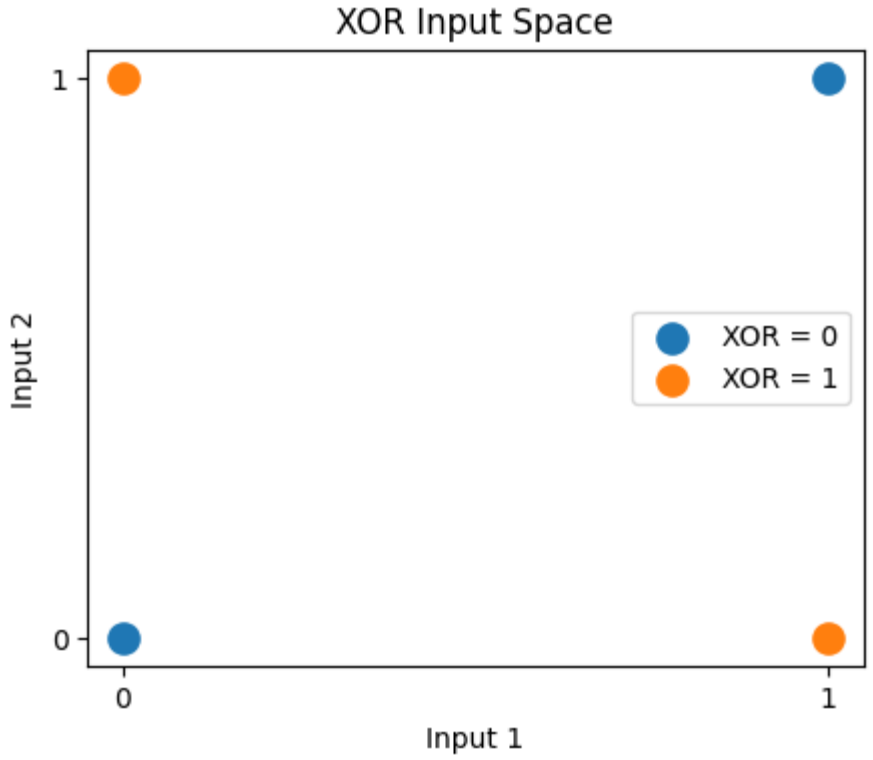
4. EDA (Exploratory Data Analysis)

For this tiny Boolean dataset, EDA is simple. We mainly verify:

- dataset shape;
- class distribution;
- missing values;
- the XOR input-output pattern.

```
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
print("\nMissing values:")
print(xor_df.isnull().sum())

print("\nClass distribution:")
print(xor_df["XOR Output"].value_counts().sort_index())

print("\nNumber of duplicate rows:", xor_df.duplicated().sum())

# Simple visualization
plt.figure(figsize=(5, 4))
for label in [0, 1]:
    subset = X[y[:, 0] == label]
    plt.scatter(subset[:, 0], subset[:, 1], label=f"XOR = {label}", s=120)
plt.xlabel("Input 1")
plt.ylabel("Input 2")
plt.title("XOR Input Space")
plt.xticks([0, 1])
plt.yticks([0, 1])
plt.legend()
plt.show()
```

```
Shape of X: (4, 2)
Shape of y: (4, 1)

Missing values:
Input 1       0
Input 2       0
XOR Output    0
dtype: int64

Class distribution:
XOR Output
0    2
1    2
Name: count, dtype: int64

Number of duplicate rows: 0
```



EDA Interpretation

- Shape is **4 × 2** for the inputs and **4 × 1** for the target.
- There are **no missing values**.
- There are **no duplicate rows**.
- Both output classes (0 and 1) are represented equally.
- The plot shows that XOR cannot be separated correctly by a single straight line. This is why a hidden layer is required.

5. Preprocessing

The XOR inputs are already numeric and are in the range 0–1, so scaling or encoding is unnecessary.

We only make sure that:

- inputs are stored as `float32`;
- targets are stored as `float32`;
- the target has shape `(4, 1)`, which matches the single output neuron.

Because this is a four-row demonstration dataset, a train/test split is not used. We train on all four combinations and then test the same four combinations to verify whether the Boolean function has been learned.

```
X = X.astype(np.float32)
y = y.astype(np.float32)

print("X dtype:", X.dtype)
print("y dtype:", y.dtype)
print("X shape:", X.shape)
print("y shape:", y.shape)
```

✓ 0.0s

```
X dtype: float32
y dtype: float32
X shape: (4, 2)
y shape: (4, 1)
```

6. Implementation 1 – Keras

Keras provides a high-level and simple way to build the MLP.

Architecture:

- Input: 2 values
- Hidden: 4 neurons with Tanh
- Output: 1 neuron with Sigmoid
- Loss: Binary Cross-Entropy
- Optimizer: Adam

```
# Build the Keras MLP
keras_model = keras.Sequential([
    keras.layers.Input(shape=(2,)),
    keras.layers.Dense(4, activation="tanh"),
    keras.layers.Dense(1, activation="sigmoid")
])

keras_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.05),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

keras_model.summary()
```

✓ 0.1s Python

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be

Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 4)	12
dense_1 (Dense)	(None, 1)	5

Total params: 17 (68.00 B)

Trainable params: 17 (68.00 B)

Non-trainable params: 0 (0.00 B)

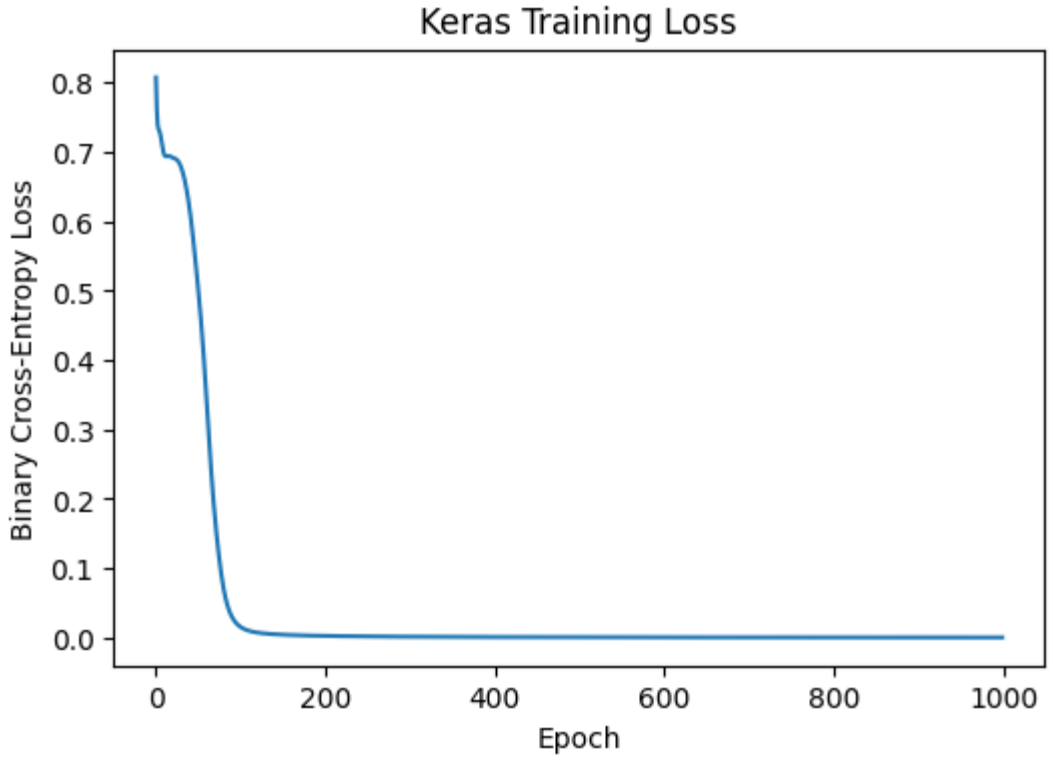
```
# Train the Keras model
keras_history = keras_model.fit(
    X, y,
    epochs=1000,
    verbose=0
)

keras_loss, keras_acc = keras_model.evaluate(X, y, verbose=0)
keras_prob = keras_model.predict(X, verbose=0)
keras_pred = (keras_prob >= 0.5).astype(int)

keras_results = xor_df.copy()
keras_results["Predicted Probability"] = keras_prob.ravel()
keras_results["Predicted Class"] = keras_pred.ravel()

display(keras_results)
print(f"Keras final loss: {keras_loss:.6f}")
print(f"Keras final accuracy: {keras_acc:.2%}")
```

	Input 1	Input 2	XOR Output	Predicted Probability	Predicted Class
0	0	0	0	0.000170	0
1	0	1	1	0.999789	1
2	1	0	1	0.999839	1
3	1	1	0	0.000184	0

```
Keras final loss: 0.000181
Keras final accuracy: 100.00%
```

```
# Keras training curve
plt.figure(figsize=(6, 4))
plt.plot(keras_history.history["loss"])
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Keras Training Loss")
plt.show()
```

✓ 0.1s



Keras Interpretation

The Keras MLP learns the XOR mapping when the predicted probabilities are close to 0 for the negative cases and close to 1 for the positive cases. The loss should decrease substantially during training and the final classification accuracy should reach approximately 100% for a successful run.

Small differences in the exact probabilities can occur because neural-network training starts with randomly initialized weights.

7. Implementation 2 – TensorFlow Low-Level API

Here the same MLP is implemented using TensorFlow operations instead of Keras `fit()`.

The important difference is that the training loop is written manually using:

- `tf.GradientTape()` to calculate gradients;
- the optimizer to update weights;
- Binary Cross-Entropy to calculate loss.

```python
# TensorFlow low-level MLP
class TensorFlowXOR:
    def __init__(self, hidden_neurons=4, learning_rate=0.05, activation="tanh"):
        self.hidden_neurons = hidden_neurons
        self.activation_name = activation

        # Trainable weights and biases
        self.W1 = tf.Variable(
            tf.random.normal([2, hidden_neurons], stddev=0.5),
            trainable=True
        )
        self.b1 = tf.Variable(tf.zeros([hidden_neurons]), trainable=True)
        self.W2 = tf.Variable(
            tf.random.normal([hidden_neurons, 1], stddev=0.5),
            trainable=True
        )
        self.b2 = tf.Variable(tf.zeros([1]), trainable=True)

        self.optimizer = tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        )
        self.loss_fn = tf.keras.losses.BinaryCrossentropy()

    def activation(self, z):
        if self.activation_name == "relu":
            return tf.nn.relu(z)
        return tf.nn.tanh(z)

    def forward(self, x):
        hidden = self.activation(tf.matmul(x, self.W1) + self.b1)
        output = tf.nn.sigmoid(tf.matmul(hidden, self.W2) + self.b2)
        return output

    def train(self, x, target, epochs=1000):
        losses = []

        for epoch in range(epochs):
            with tf.GradientTape() as tape:
                prediction = self.forward(x)
                loss = self.loss_fn(target, prediction)

            variables = [self.W1, self.b1, self.W2, self.b2]
            gradients = tape.gradient(loss, variables)
            self.optimizer.apply_gradients(zip(gradients, variables))
            losses.append(float(loss))

        return losses

tf_model = TensorFlowXOR(hidden_neurons=4, learning_rate=0.05, activation="tanh")
tf_losses = tf_model.train(tf.constant(X), tf.constant(y), epochs=1000)

tf_prob = tf_model.forward(tf.constant(X)).numpy()
tf_pred = (tf_prob >= 0.5).astype(int)
tf_acc = np.mean(tf_pred == y.astype(int))

tf_results = xor_df.copy()
tf_results["Predicted Probability"] = tf_prob.ravel()
tf_results["Predicted Class"] = tf_pred.ravel()
```

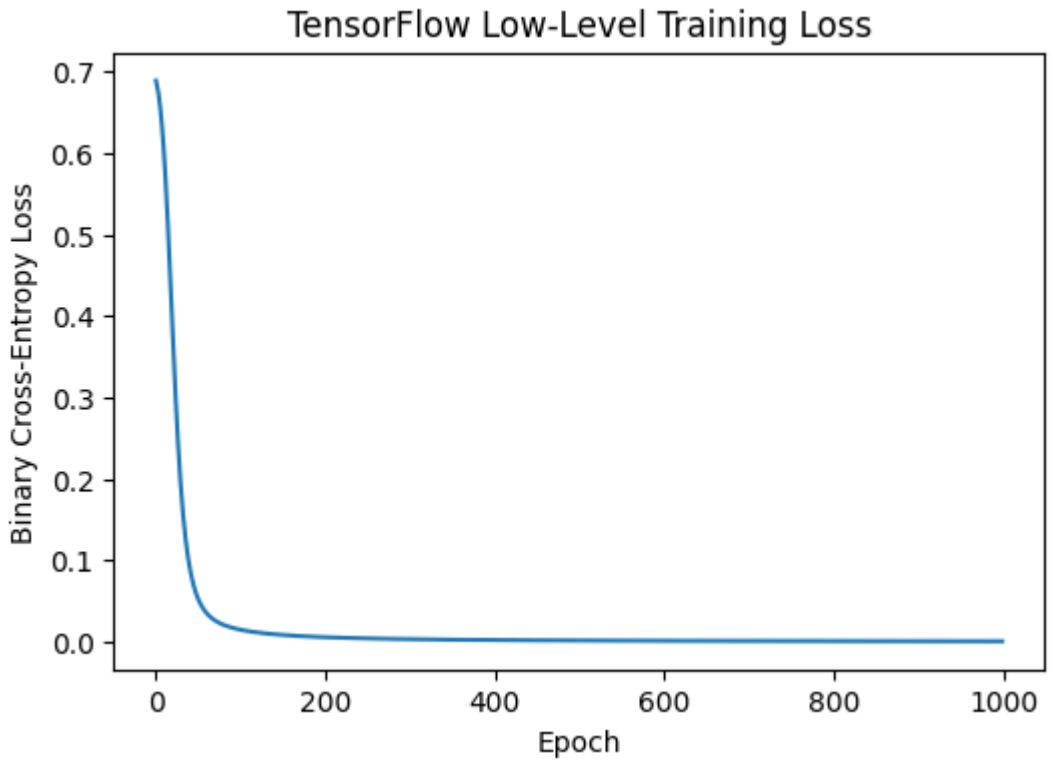
```
display(tf_results)
print(f"TensorFlow final loss: {tf_losses[-1]:.6f}")
print(f"TensorFlow final accuracy: {tf_acc:.2%}")
```

✓ 19.7s

	Input 1	Input 2	XOR Output	Predicted Probability	Predicted Class
0	0	0	0	0.000030	0
1	0	1	1	0.999132	1
2	1	0	1	0.999821	1
3	1	1	0	0.000795	0

```
TensorFlow final loss: 0.000469
TensorFlow final accuracy: 100.00%
```

```
# TensorFlow training curve
plt.figure(figsize=(6, 4))
plt.plot(tf_losses)
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("TensorFlow Low-Level Training Loss")
plt.show()
```



TensorFlow Interpretation

The manually written training loop performs the same basic learning process as Keras:

1. forward pass;
2. calculate binary cross-entropy;
3. calculate gradients;
4. update weights;
5. repeat for many epochs.

A successful run should classify all four XOR combinations correctly and obtain approximately 100% training accuracy.

8. Hyperparameter Experiment

The question asks us to study the effect of:

- learning rate;
- activation function;
- number of hidden neurons;
- number of epochs.

For a simple XOR problem, the main purpose is to understand the effect of these settings rather than to perform a large-scale experiment.

Generate + Code + Markdown

```
# Keras helper function for a small controlled experiment
def run_keras_experiment(hidden_neurons=4, activation="tanh",
                         learning_rate=0.05, epochs=1000):
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)

    model = keras.Sequential([
        keras.layers.Input(shape=(2,)),
        keras.layers.Dense(hidden_neurons, activation=activation),
        keras.layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    history = model.fit(X, y, epochs=epochs, verbose=0)
    loss, acc = model.evaluate(X, y, verbose=0)
    return loss, acc, history

experiments = [
    ("Baseline", 4, "tanh", 0.05, 1000),
    ("Fewer neurons", 2, "tanh", 0.05, 1000),
    ("More neurons", 8, "tanh", 0.05, 1000),
    ("ReLU", 4, "relu", 0.05, 1000),
    ("Lower learning rate", 4, "tanh", 0.01, 1000),
    ("Fewer epochs", 4, "tanh", 0.05, 100),
]
```

```python
experiment_rows = []

for name, neurons, activation, lr, epochs in experiments:
    loss, acc, history = run_keras_experiment(
        hidden_neurons=neurons,
        activation=activation,
        learning_rate=lr,
        epochs=epochs
    )
    experiment_rows.append({
        "Experiment": name,
        "Hidden Neurons": neurons,
        "Activation": activation,
        "Learning Rate": lr,
        "Epochs": epochs,
        "Final Loss": loss,
        "Accuracy": acc
    })

experiment_df = pd.DataFrame(experiment_rows)
display(experiment_df)
```

✓ 2m 55.6s Python

```
ARNING:tensorflow:From C:\Users\nvnir\AppData\Roaming\Python\Python313\site-packages\keras\src\backend\common\global_state.py:82: The name tf.reset_de
```

	Experiment	Hidden Neurons	Activation	Learning Rate	Epochs	Final Loss	Accuracy
0	Baseline	4	tanh	0.05	1000	0.000198	1.00
1	Fewer neurons	2	tanh	0.05	1000	0.346790	0.50
2	More neurons	8	tanh	0.05	1000	0.000124	1.00
3	ReLU	4	relu	0.05	1000	0.346634	0.75
4	Lower learning rate	4	tanh	0.01	1000	0.004855	1.00
5	Fewer epochs	4	tanh	0.05	100	0.015387	1.00

Hyperparameter Interpretation

- **Number of neurons:** More neurons give the hidden layer more capacity. XOR can be learned with a small hidden layer; 4 neurons is a comfortable choice for this lab.
- **Activation function:** A non-linear activation such as Tanh allows the network to learn the non-linear XOR relationship. ReLU can also work, although results can depend more on initialization and training settings.
- **Learning rate:** A smaller learning rate usually makes weight updates more gradual. A learning rate that is too large can make training unstable, while one that is too small can make training slow.
- **Epochs:** More epochs give the model more opportunities to reduce the loss. Too few epochs may leave the model under-trained.
- **Best practical configuration for this lab:** 4 hidden neurons, Tanh, Adam, learning rate 0.05, and 1000 epochs is a simple configuration that normally learns XOR reliably.

9. Compare the Two Implementations

The two implementations use the same MLP architecture and XOR dataset. Keras provides a high-level interface, while the TensorFlow low-level API explicitly handles the forward pass, gradients, and parameter updates.

Library/API	Main Training Method	Level
Keras	`model.fit()`	High-level
TensorFlow Low-Level	`GradientTape()` + manual update	Low-level

The exact final loss can differ slightly because of weight initialization and implementation details. The important result is that both implementations should learn the XOR mapping correctly.

```
comparison_df = pd.DataFrame({
    "Implementation": ["Keras", "TensorFlow Low-Level"],
    "Final Loss": [keras_loss, tf_losses[-1]],
    "Accuracy": [keras_acc, tf_acc]
})

display(comparison_df)

print("Expected XOR:", y.astype(int).ravel())
print("Keras:", keras_pred.ravel())
print("TensorFlow Low-Level:", tf_pred.ravel())
```

✓ 0.0s

	Implementation	Final Loss	Accuracy
0	Keras	0.000181	1.0
1	TensorFlow Low-Level	0.000469	1.0

```
Expected XOR: [0 1 1 0]
Keras: [0 1 1 0]
TensorFlow Low-Level: [0 1 1 0]
```

10. Result

The MLP was successfully used to learn the XOR Boolean function. The model was implemented using Keras and the TensorFlow low-level API. The trained networks predicted the four XOR combinations correctly when the training configuration was suitable.

Final XOR mapping

- 0, 0 → 0
- 0, 1 → 1
- 1, 0 → 1
- 1, 1 → 0

The hyperparameter experiment also shows that the learning rate, activation function, number of hidden neurons, and number of epochs affect how quickly and reliably the network learns.

11. Conclusion

The XOR problem demonstrates why a hidden layer is important in neural networks. XOR is non-linear and cannot be solved correctly by a single linear neuron. An MLP with a non-linear hidden layer successfully learned the XOR function. Keras made the implementation simple, while the TensorFlow low-level API showed the individual steps of forward propagation, loss calculation, backpropagation, and weight updates. The experiment also demonstrated that suitable hyperparameters are important for achieving good model performance.